SISTEMA BANCARIO

Red de cajeros automáticos
Fase II - Cliente / Servidor

C# / WPF / SQL Server / TCP - JSON

Documento técnico, manuales, protocolo, trazabilidad y scripts SQL completos.

Visual Studio 2022 · Entrega preparada el 31 de octubre de 2026

Integrantes, carnés y enlace al video: completar por el grupo antes de presentar.

Estado de validación

Implementación y pruebas locales documentadas. La demostración con tres equipos físicos, el firewall de la red real y el video requieren la participación del grupo. La integración NETGEAR implementa el MD recibido; falta el ensayo físico con otro grupo.

Contenido

SISTEMA BANCARIO 1

Contenido 2

NETGEAR 2.2.0 - arranque, transferencias y conexiones 4

Revisión del servidor - 31/10/2026 6

Correcciones del cajero - NETGEAR 2.1.0 8

Índice para localizar y explicar archivos 9

Alcance y decisiones 11

Datos de demostración 12

Matriz de trazabilidad 13

Arquitectura del sistema 16

Modelo relacional y scripts 17

Diagrama relacional - operación 18

Diagrama relacional - administración 19

Protocolo interno v1 20

Instalación y conexión en red 23

Manual del servidor y dashboard 25

Captura: servidor resumen 27

Captura: servidor cajeros 28

Captura: servidor usuarios 29

Manual del cajero 30

Captura: cajero acceso 32

Captura: cajero token 33

Captura: cajero menu 34

Verificación de la entrega 35

Demostración en equipos físicos y guion del video 36

Integración NETGEAR 37

Ficha NETGEAR 38

Protocolo implementado - NETGEAR 39

Lista para probar la integración futura 41

Entrega al jefe de grupo 42

Anexo - scripts SQL completos 43

NETGEAR 2.2.0 - arranque, transferencias y conexiones

Abrir el cajero

En Windows x64, descargar NETGEAR_Cajero_2.2.0.exe desde Releases y abrirlo. Incluye .NET; no requiere Visual Studio ni SQL Server en el cajero. También puede descargar el ZIP completo, extraerlo y abrir Iniciar-Cajero.cmd. En un clon del repositorio, el CMD descarga y verifica el ejecutable publicado mediante SHA256. Sincronizar-Cajero.cmd descarga otra vez la versión indicada por el lanzador. Cerrar el cajero antes de actualizar. Para versiones futuras: git pull y luego Sincronizar-Cajero.cmd. Los ejecutables se publican como archivos de Releases del mismo repositorio.

Buscar servidor abre una ventana con Escanear servidores, selección de IP/puerto y código asignado. El escaneo combina UDP 5005 y consultas TCP NETGEAR_INFO sin acceso SQL; incluye la IP configurada y las subredes IPv4 locales /24. Solo marque el servidor de su grupo. También puede escribir una IP manualmente. Cada equipo necesita un código distinto dado de alta en el servidor. No usar 127.0.0.1 en otra PC: esa IP apunta a la propia computadora. Si no aparece: misma LAN, servidor en Iniciar servicio, firewall TCP 5000/UDP 5005 y sin aislamiento entre clientes en el router. La configuración y referencias pendientes permanecen locales.

Enviar y recibir

Ingresar tarjeta, PIN y token; pulsar Transferir dinero. Elegir NETGEAR para otra cuenta de nuestro banco, o el banco externo autorizado. Ingresar cuenta o tarjeta destino, Verificar destinatario, monto y concepto; revisar nombre, banco y cuenta antes de confirmar. Mis transferencias muestra envíos y abonos. El destinatario recibe el saldo sin tener una sesión abierta. La transferencia no mueve billetes ni consume el límite de retiro físico. Los bancos externos deben registrar mutuamente código, IP y puerto y autorizar la conexión. BANC02 continúa provisional hasta confirmar el código con los grupos. El cajero usa su servidor; el servidor habla el protocolo IB recibido con el otro banco.

La transferencia interna debita y acredita en una misma transacción SQL. La externa persiste referencia antes del contacto y reserva saldo antes de IB_TRANSFERIR. Los dos bancos conservan el mismo token. Repetir referencia y datos devuelve el resultado sin otro cargo. Si se pierde una respuesta, se bloquean movimientos y se consulta la referencia; no se reenvía dinero automáticamente. IB_ESTADO_TRANSFERENCIA permite conciliar si el otro banco lo implementa. Si no, el administrador debe coordinar la confirmación. No devolver reservas inciertas sin confirmar que el otro banco no acreditó. El nombre del destinatario se verifica otra vez antes de reservar. El usuario origen se deriva de la sesión SQL.

Actualización del servidor

Conservar servidor.local.json y saldos existentes. Aplicar Actualizar-Interbancario.ps1 -Servidor '.' con la instancia real: migraciones aditivas 08/09/10/11. No ejecutar el instalador 01 sobre una base existente. Publicar.ps1 genera portables completos; Publicar-Ejecutables.ps1 genera los EXE únicos. Configurar-Firewall.ps1 permite indicar -Programa para el ejecutable real del servidor. El cliente nunca recibe la cadena SQL.

Estabilidad y diagnóstico del 3 de noviembre

El proceso del servidor permaneció ejecutándose durante los cortes observados. Los cierres simultáneos de varios cajeros coincidieron con eventos de desconexión de la LAN en Windows. Hubo errores de renovación DHCP y una dirección 169.254.x.x en Ethernet; posteriormente recuperó dirección del router. No se observó saturación ni bloqueo SQL persistente en la captura operativa. Se deshabilitó la opción que permite apagar los adaptadores Ethernet y Wi-Fi para ahorrar energía. Esto no equivale a reservar una dirección en el router: aún se deben configurar reservas DHCP distintas por MAC para Ethernet y Wi-Fi. No asignar a mano una IP dentro de un rango DHCP desconocido.

El servidor ahora mantiene lectura TCP mientras procesa solicitudes, con escritura serializada y operaciones limitadas por conexión. Los heartbeats disponen de un pool SQL separado (16 conexiones); operaciones usan hasta 100 conexiones reutilizables. El contexto SQL limitado se restaura con REVERT antes de devolver una conexión al pool. El latido SQL espera como máximo 200 ms ante bloqueo y devuelve estado ocupado; el transporte continúa. La detección de conexión usa llegada de heartbeat en memoria, no una fila SQL atrasada. Un heartbeat perdido aislado no provoca reconexión inmediata; un cajero

sin latidos durante 30 segundos sí se desconecta. Se registran causas SocketError, operaciones lentas y cambios de interfaces para distinguir red y SQL. TCP KeepAlive complementa el heartbeat. Estos cambios no evitan cortes físicos, fallos DHCP o aislamiento del router.

Verificación reproducible

Pruebas.dll --transferencias crea dos bases y dos bancos TCP aislados. 37 comprobaciones: débito/crédito interno, banco externo real, recepción sin sesión, deduplicación, respuesta perdida y conciliación, cupos, concurrencia, rollback, heartbeat y búsqueda durante un bloqueo SQL, identidad y reutilización del pool y desconexión de un cliente silencioso. No modifica la base operativa. Ejecutar con CAJERO_SQL_SERVIDOR apuntando a la instancia de pruebas. Mantener también --integracion, --revision-cajero y --revision-servidor. La prueba física con otros grupos y sus implementaciones IB sigue siendo necesaria.

Validación completada: 83 comprobaciones de integración, 37 de transferencias entre dos bancos, 31 de interfaz y recuperación del cajero y 5 del servidor: 156 en total.

Revisión del servidor - 31/10/2026

Referencia: PDF Fase II, secciones 5.1, 5.2, 7 y 8. Alcance: servidor.

Login y dashboard

Login con PBKDF2-SHA256 y PasswordBox. Se añade ojo para mostrar/ocultar; se oculta al enviar o desactivar la ventana y se limpia al terminar cada intento. Están implementados Resumen, Cajeros, Usuarios, Transacciones, Depósitos, Retiros, Cambios de PIN, Alertas, Auditoría, Arqueo, Resumen diario y Configuración. D-11 está en Usuarios/Historial; D-12 en Resumen/Cambios de PIN. Depósitos y Retiros YA tenían total y promedio en el pie, pero ignoraban el filtro de texto. Se corrige para calcular sobre filas visibles, excluyendo rechazos del dinero. Existen tres gráficas y refresco programado a cuatro segundos; verificar duración real bajo carga. Se recarga el puerto guardado al iniciar servicio y se remite al LEEME para restaurar el respaldo sin reinstalar la base. S-12 (correo) y D-17 (tema/sonido/Supervisor completo) son deseables sin completar. No falta una página obligatoria entera. La demostración en tres equipos y video sigue pendiente.

Red y reservas DHCP

Ethernet 2: 192.168.0.34; gateway 192.168.0.1; MAC 9C-7F-64-4D-FE-DE. Wi-Fi: 192.168.2.137; gateway 192.168.2.1; MAC observada F6-23-1D-F5-39-A2. Ambos en DHCP y perfil Público; son redes distintas. Ethernet sin 2 es VirtualBox con 169.254.*, no sirve para los cajeros físicos. El servidor ya escucha en IPAddress.Any. La pantalla ahora identifica adaptadores y actualiza IPv4, omitiendo enlace local, loopback e interfaces sin gateway IPv4. Esto no fija IP.

Para mantener una IP por adaptador: 1. Elegir router de demostración y conectar servidor/clientes a su LAN. 2. Desactivar direcciones de hardware aleatorias del perfil Wi-Fi; reconectar y consultar MAC efectiva. Actualmente está habilitado; la MAC observada puede cambiar. 3. Crear una reserva DHCP por MAC en el router: una IP para Ethernet y otra distinta para Wi-Fi. Mantener Windows en DHCP. 4. Si usan dos routers, crear cada reserva en el correspondiente y su subred. 5. Reconectar y verificar direcciones; configurar los cajeros con la IP alcanzable. Cambiar adaptador puede requerir cambiar destino. Un nombre estable requiere resolución comprobada desde cada cliente.

No asignar la misma IP a ambos adaptadores activos. Para IP manual falta conocer máscara y rango/exclusiones DHCP; no elegir direcciones arbitrarias. No instalar un servidor DHCP en esta PC. No se modificó red ni router: pendiente definir red definitiva con el usuario.

Firewall

No se encontró regla específica CajeroRed TCP 5000. Las reglas de Proyecto Cajero corresponden a otra aplicación. El ejemplo Privado del manual no cubre las redes Públicas actuales. Tras elegir LAN de confianza, permitir TCP del ejecutable Distribucion/Servidor/Servidor.exe en puerto configurado y alcance adecuado. Mantener SQL local. Desde otra PC: Test-NetConnection IP_DEL_SERVIDOR -Port 5000; después registrar tres cajeros simultáneos. Las pruebas locales no acreditan esta conectividad.

Referencia: https://learn.microsoft.com/windows-server/networking/technologies/dhcp/dhcp-scopes

Repositorio

Se excluyen configuración privada, respaldo BAK con usuarios/hashes/movimientos, ejecutables y cachés. Los scripts con datos ficticios permiten crear una base nueva; el respaldo original permanece local.

Ampliación solicitada: NETGEAR y desconexiones

Se añaden Bancos de la red y Transferencias conforme al MD recibido; migración SQL aditiva 08. La bitácora anterior muestra cortes repetidos y algunos simultáneos, pero solo registra un motivo genérico. No prueba de quién fue el fallo. Se separan aceptación/monitoreo TCP y ciclo del cliente del hilo WPF. El cliente observa la tarea que termina (WhenAny por sí solo no propaga su excepción) y programa heartbeat con PeriodicTimer de diez segundos. Nuevos logs JSONL en Logs junto al

ejecutable: motivo, fase, IP, último tipo, milisegundos sin recepción/heartbeat, código Socket/SQL y consultas lentas. No incluyen payload, PIN, tokens de login o contraseñas. Motivos: HEARTBEAT_VENCIDO_30S, REMOTO_CERRO, SOCKET_..., PARADA_SERVIDOR, registro rechazado. La bitácora guarda CONEXION_MOTIVO. Correlacionar UTC de Logs con hora local SQL. La prueba local mantiene tres clientes más de treinta segundos. No demuestra estabilidad de la LAN real. Para diagnóstico de clientes antiguos conviene distribuir después el transporte actualizado del cliente; el alcance de este paquete portable sigue siendo el servidor.

Estado local tras la actualización

Se aplican 08 y 09 sin reinstalar la base. Identidad NETGEAR con BANC02 provisional. Reglas NETGEAR-Servidor-TCP-5000 y NETGEAR-Discovery-UDP-5005 habilitadas para Distribucion/Servidor/Servidor.exe, perfil Any y origen LocalSubnet. No se abrió SQL. Actualizados los portables de servidor y CajeroRed_Cajero; la IP fija permanece pendiente de definir router y reserva DHCP. No se identificó todavía la causa exacta de los cortes físicos; los nuevos logs distinguen el motivo y las pruebas locales verifican el comportamiento esperado.

Correcciones del cajero - NETGEAR 2.1.0

Referencia: cambios.txt entregado el 31/10/2026. Fuente en Codigo/Cajero y Codigo/Aplicacion/ClienteTcp*.cs de la solución del servidor. El paquete CajeroRed_Cajero contiene el portable, no otro fuente diferente.

1. Retiros y depósitos se bloquean en pantalla y en el transporte cuando hay movimiento en curso o pendiente. 2. Cerrar sesión limpia el acceso local inmediatamente; el LOGOUT se notifica sin esperar respuesta. 3. PIN actual/nuevo/confirmación: hasta cuatro dígitos, rechazo de pegados inválidos, cuatro completos, confirmación igual y nuevo distinto del anterior. 4. Saldo muestra saldo, límite, retirado hoy, disponible del límite y mínimo entre saldo/disponible. Los billetes siguen validándose en el servidor. 5. La referencia se escribe atómicamente ANTES del envío y se recupera al autenticar al usuario. Se asocia a dirección/puerto, cajero y usuarioId del servidor. Nunca guarda PIN, token, sesión o tarjeta completa. 6. Tarjeta/PIN/token restringidos a dígitos y longitud; PIN/token enmascarados. Un pegado largo se rechaza completo. 7. Un nuevo acceso limpia el aviso de inactividad. 8. Configuración validada al cargar; JSON roto o puerto inválido permiten corregirla. Los errores de escritura se muestran sin cerrar el cajero; la instalación normal permite respaldo en LocalAppData. 9. Monto y desglose se validan juntos antes de cerrar; un error conserva los valores. Lo mismo para la confirmación de PIN. 10. Guardar comprobante en PDF mediante PDFsharp-WPF 6.2.4 (MIT). Captura solo el resultado confirmado, fecha SQL, referencia, cajero y tarjeta enmascarada; cambio de PIN solo confirma el cambio. Error de guardado no cambia el movimiento. 11. Actividad dentro de formularios cuenta para los 60 segundos. Al expirar se cierran los formularios. 12. Desconexión/deshabilitación cierra formularios e invalida la sesión; una respuesta tardía no vuelve a mostrar información de una sesión terminada.

Servidor compatible

09_CompatibilidadCajero.sql actualiza solo sp_Operacion. VALIDAR_TOKEN añade usuarioId; ESTADO_SOLICITUD devuelve idOriginal/usuarioId y mantiene la autorización SQL por usuario/cajero; saldo y cambio PIN añaden fecha SQL. Ejecutar Actualizar-Interbancario.ps1 aplica 08 y 09 sin reinstalar ni reinicializar la base. Conservar también las validaciones del servidor.

Pendientes y recuperación

Registros en LocalAppData/NETGEAR/Cajero/Pendientes, uno por servidor/puerto/cajero/usuario. El archivo conserva referencia, tipo, monto y fecha técnica local, sin credenciales. Al volver a ingresar con el usuario correspondiente, usar Consultar operación pendiente. Solo se elimina con una respuesta de estado correlacionada del servidor. Código 25 o 96 no significa rechazo: mantener el bloqueo y revisar con administración. Un registro dañado o ilegible bloquea dinero y permite consultas; no se descarta silenciosamente. La dirección de servidor forma parte de la asociación: mantener la misma configuración o un nombre estable al recuperar; si cambió la IP, coordinar la recuperación con administración. Para reemplazar un PIN/token por pegado, vaciar primero el campo; así no se trunca silenciosamente una entrada demasiado larga.

CMD y portable

Iniciar-Cajero.cmd sin argumentos respeta la configuración guardada. Argumentos: código, dirección, puerto opcional. Ejemplo: Iniciar-Cajero.cmd ATM-01 192.168.0.34 5000 Los argumentos prevalecen sobre el JSON. Publicar-Cajero.ps1 actualiza el paquete hermano CajeroRed_Cajero, conserva configuración privada y regenera su manifiesto SHA256. El teclado táctil sigue pendiente como deseable.

Validación

Pruebas/RevisionCajero.cs y --revision-cajero verifican respuesta perdida, bloqueo, recuperación/identidad, entrada numérica, formularios, actividad, deshabilitación, corte, cierre inmediato, respuesta tardía, configuración y cuatro tipos de PDF. Los ensayos son locales con un servidor simulado; la integración SQL/TCP real se verifica aparte en una base temporal.

Resultado final: 28 comprobaciones específicas del cajero, 83 SQL/TCP/UDP y 5 de interfaz del servidor, todas aprobadas. Compilación sin errores ni advertencias. PDF de ejemplo en Pruebas/Evidencias/Comprobantes. Dependencia PDF fijada: https://www.nuget.org/packages/PDFsharp-WPF/6.2.4 (MIT, Windows/WPF).

Índice para localizar y explicar archivos

Si el ingeniero solicita...	Abrir...
La solución de Visual Studio	`CajeroRed.sln`
El servidor TCP y sus conexiones simultáneas	`Codigo/Aplicacion/ServidorTcp.cs`
La reconexión y los heartbeats del cliente	`Codigo/Aplicacion/ClienteTcp.cs`
La estructura de los mensajes	`Codigo/ProtocoloCompartido/Mensaje.cs` y `Documentacion/04_Protocolo/ProtocoloInterno.md`
La creación de tablas y restricciones	`BaseDeDatos/Scripts/01_CrearBase.sql`
Las transacciones, bloqueos y rollback	`BaseDeDatos/Scripts/02_Operaciones.sql`, procedimiento `sp_Operacion`
Un procedimiento específico de retiro o depósito	`03_Contratos.sql`; delega al motor transaccional de 02
La inicialización o cambio administrativo de tarjeta	`04_Administracion.sql`
El trigger de auditoría	`04_Administracion.sql`, `tr_TarjetaBitacora`
Las consultas del dashboard	`05_Vistas.sql` y `Codigo/Servidor/MainWindow.xaml.cs`, método `Refrescar`
Los datos de prueba	`06_DatosPrueba.sql` y `01_Requisitos/DatosDePrueba.md`
Los permisos SQL	`07_Permisos.sql` y `Codigo/Infraestructura/BaseDatos.cs`, método `Abrir`
El hash del PIN	`02_Operaciones.sql`: SHA2_256 de salt + PIN; nunca aparece en respuestas
La contraseña administrativa	`Codigo/Aplicacion/Administracion.cs`: PBKDF2-SHA256 y comparación constante
El formulario breve de usuario	`Codigo/Servidor/MainWindow.xaml.cs`, método `AccionesUsuario`
El estilo original adaptado a WPF	`Codigo/PresentacionCompartida/Tema.xaml`
El recibo del cajero	`Codigo/Cajero/MainWindow.xaml.cs`, método `Comprobante`
El reporte Excel	`Codigo/PresentacionCompartida/Exportador.cs`
El modelo relacional	`03_BaseDeDatos/Modelo.md` y `03_BaseDeDatos/ModeloRelacional.png`
Los pasos para cambiar de router	`05_InstalacionYRed/ManualDeInstalacion.md`
Las pruebas y qué falta demostrar	`07_PruebasYEvidencias/Resultados.md` y `EscenariosFisicos.md`
La conexión con otro grupo	`08_IntegracionEntreGrupos/LEEME_Integracion.md`
El respaldo	`BaseDeDatos/Respaldos/BD_CajeroRed_Fase2.bak` y su LEEME
La entrega al jefe de grupo	`09_Entrega/GuiaDeContinuacion.md`

Las rutas de esta tabla son relativas a la raíz de `CajeroRed_Fase2`, salvo los enlaces entre documentos. La matriz de trazabilidad vincula cada requisito con su archivo y prueba. El PDF reúne arquitectura, modelo, protocolo, manuales, pruebas y scripts íntegros como anexos.

Cambios NETGEAR

- BancoInterbancario.cs: protocolo de bancos; MainWindow.Interbancario.cs: dashboard. - BaseDeDatos/Scripts/08_Interbancario.sql y Actualizar-Interbancario.ps1: actualización aditiva. - Configurar-Firewall.ps1: TCP y UDP para la subred de pruebas. - Documentacion/09_Entrega/RevisionServidor_2026-10-31.md: requisitos, DHCP y diagnóstico.

Alcance y decisiones

Referencia: `PROYECTO_CAJERO_FASE_II_CLIENTE_SERVIDOR.pdf`, 13 páginas entregadas por el usuario. El enunciado es una fuente de requisitos; las decisiones de alcance del usuario prevalecen sobre instrucciones ajenas al proyecto.

La fase II se implementa en C# con WPF para servidor y cajero. Todo dato operativo reside en SQL Server. Se conserva la identidad visual clásica: azul #163A82, azul de interacción #245FD4, fondo #F4F7FC, tablas alternadas y botones de navegación. Se adapta el diseño a ventanas redimensionables, sin depender de PySide6.

El registro administrativo solicita nombre, tarjeta de 16 dígitos, PIN y confirmación, saldo inicial y límite diario (D-10). Identificador, estado y fecha son automáticos. No se exige llenar dirección, propiedad, actividad económica, ingresos o beneficiarios para una operación de cajero. La fase I original se conserva como referencia y no se migran ni alteran sus datos personales reales.

Se implementan las operaciones de usuario enumeradas por la fase II: retiro, depósito, cambio de PIN, consulta de saldo y últimas cinco transacciones. Pagos de servicios, remesas, adelantos, NFC y ESP32 de la aplicación previa no forman parte del flujo obligatorio descrito en el PDF de fase II. No se afirma su migración; si el catedrático exige adicionales del enunciado original de fase I, revisar ese documento antes de ampliar el alcance.

El número de cajeros se configura en base de datos, sin una constante de cinco. La capacidad efectiva depende del equipo y de la red. Se prueba el mínimo de tres conexiones; no se afirma escalabilidad ilimitada.

Actualización 31/10/2026: se implementa NETGEAR con discovery UDP y protocolo IB del MD recibido. Ver ProtocoloExterno.md para atomicidad local, reservas, deduplicación y límites de interoperabilidad. Los ensayos usan pares simulados y no acreditan una prueba física con otro grupo.

Los requisitos deseables quedan separados: S-12 correo; D-17 supervisor, temas, sonido; C-12 impresión y teclado táctil (PDF implementado al 31/10/2026). La tabla Administrador admite el rol Supervisor y el servicio rechaza escrituras para él, pero el acceso completo de solo lectura no se acredita sin su prueba de aceptación.

Datos de demostración

Ocho usuarios ficticios, cada uno con Q5000 iniciales. Usuario 1 tiene límite Q500; los demás Q2000. Tarjetas 4000000000000001 a 4000000000000008, PIN inicial 1234. Administrador admin / GrupoCajero#2026. Son credenciales públicas de una entrega académica con datos ficticios.

Ocho cajeros registrados (ATM-01 a ATM-08). Se pueden agregar otros desde el dashboard sin modificar código. La cantidad debe coincidir como mínimo con los integrantes y nunca ser inferior a tres para la demostración.

Inventario sugerido para inicializar con Q10000: 20 de Q200, 20 de Q100, 20 de Q50, 50 de Q20, 50 de Q10, 100 de Q5 y 1000 de Q1. Total: Q10000. Cada cajero se inicializa una vez por día; la inicialización reemplaza el inventario de apertura y registra el efectivo anterior para trazabilidad. Antes del cambio diario, exportar el arqueo del cierre anterior.

El respaldo entregado conserva los datos ficticios preparados y el historial de acceso generado al verificar pantallas. Los resultados monetarios de las pruebas automáticas están en bases temporales que se eliminan; no contaminan esta base.

No confundir saldo inicial de una cuenta con depósito físico: el saldo al dar de alta un usuario no incrementa el inventario de ningún cajero. Los movimientos de efectivo deben pasar por retiro, depósito o carga administrativa.

Matriz de trazabilidad

La implementación se vincula a los identificadores del PDF. Las pruebas automáticas locales no sustituyen las pruebas físicas ni el video. Las rutas abreviadas de C# se ubican en Codigo/Aplicacion, Codigo/ProtocoloCompartido o los proyectos WPF; los scripts SQL en BaseDeDatos/Scripts.

Servidor

Código	Requisito	Implementación	Verificación
S-01	Escucha configurable e indicadores	Codigo/Aplicacion/ServidorTcp.cs; Codigo/Servidor/MainWindow.xaml.cs	Inicio/detención y puerto
S-02	Registro de identidad, IP, equipo y hora	ServidorTcp.cs; 02_Operaciones.sql / REGISTRAR_CAJERO	Registro TCP; rechazo de identidad duplicada/desconocida
S-03	Concurrencia sin congelar dashboard	ServidorTcp.cs / Aceptar y Procesar	Tres conexiones; retiros simultáneos
S-04	Heartbeat, desconexión, reconexión y bitácora	ServidorTcp.cs / Vigilar; ClienteTcp.cs; sp_Desconectar	Heartbeat caducado y reconexión
S-05	Acceso con tarjeta activa, PIN y token	02_Operaciones.sql / LOGIN y VALIDAR_TOKEN	Login y rechazo de credenciales
S-06	Token 6 dígitos, 2 min, un uso, ventana servidor	sp_GenerarToken; MainWindow.xaml.cs / MostrarToken	Token no expuesto al cliente; vencimiento
S-07	Bloqueo tras tres fallos globales	02_Operaciones.sql / AccesoFallido	Tres cajeros; desbloqueo administrativo
S-08	Operaciones y reglas	02_Operaciones.sql / sp_Operacion	Operaciones, límites, PIN e inventario
S-09	Comandos remotos	ServidorTcp.cs / Comando; Cajero/MainWindow.xaml.cs	Fuera de servicio y habilitar
S-10	Persistencia antes de responder y rollback	02_Operaciones.sql / TRY CATCH; BaseDatos.cs	Trigger de fallo simulado revierte movimiento
S-11	Sesión expira en servidor a 2 min	sp_Operacion; ServidorTcp.cs / Vigilar	Caducar UltimaActividad en base aislada
S-12	Correo de token/alertas	No implementado: deseable	Fuera del alcance obligatorio

Dashboard

Código	Requisito	Implementación	Verificación
D-01	Login admin con hash	Administracion.cs / Login; Administrador	PBKDF2 válido e inválido
D-02	Indicadores diarios	05_Vistas.sql / vw_ResumenDiario; DibujarResumen	Dashboard; comparación SQL
D-03	Tres gráficas	Servidor/MainWindow.xaml.cs / DibujarResumen y Grafica	Revisión visual; denominaciones por cajero
D-04	Monitor de estado, IP, billetes y actividad	vw_EstadoCajeros; página Cajeros	Registro y desconexión; tabla
D-05	Gestión, apertura, cargas, comandos	04_Administracion.sql; AccionesCajero	Una apertura diaria, capacidad y comando
D-06	Transacciones de toda la red	vw_Transacciones; página Transacciones	Movimientos aprobados/rechazados

Código	Requisito	Implementación	Verificación
D-07	Depósitos con filtros, totales y promedio	Refrescar / Depósitos; vw_Transacciones	Filtro fechas/texto y exportación
D-08	Retiros, rechazos y consumo diario	Refrescar / Retiros; vw_RetiradoHoyPorUsuario	Límite global y motivos
D-09	Cambios y restablecimiento de PIN temporal	CambioPin; RESTABLECER_PIN; CambiarPin	Cambio global y PIN temporal
D-10	Gestión de usuarios/tarjetas/límites/bloqueo	04_Administracion.sql; AccionesUsuario	Cambio tarjeta conserva consumo
D-11	Consulta por usuario y último acceso	vw_Usuarios; Historial del usuario	Saldo, máximo y movimientos
D-12	Control de usuarios del día	vw_ResumenDiario; DibujarResumen; Cambios de PIN	Totales, promedio, quién cambió PIN y último acceso
D-13	Alertas y atención	sp_Alertas; sp_Desconectar; ATENDER_ALERTA	Falta billetes, bloqueo, desconexión, atención
D-14	Reportes PDF o Excel	Exportador.cs; Exportar; Arqueo y Resumen diario	Archivo OpenXML verificado
D-15	Bitácora de auditoría con filtros	Bitacora; tr_TarjetaBitacora; Auditoría	Conexión, acciones, rechazos y exportación
D-16	Parámetros en SQL	Parametro; PARAMETROS; Configuración	Validaciones de máximos y puerto
D-17	Supervisor, tema oscuro y sonido	Deseable: no acreditado como entrega completa	No se incluyen en cumplimiento obligatorio

Cajero

Código	Requisito	Implementación	Verificación
C-01	Configuración sin cadena SQL	Configuracion/cajero.ejemplo.json; Cajero/MainWindow.xaml.cs	Archivo y pantalla; --cajero/--servidor/--puerto
C-02	Registro y reconexión cada 5s	ClienteTcp.cs / Reconectar	Reconexión automática
C-03	Heartbeat y comandos	ClienteTcp.cs / Latidos; AtenderComando	Conexión persistente y comando
C-04	Tarjeta, PIN enmascarado y token	Cajero/MainWindow.xaml.cs / Login	Capturas de acceso/token
C-05	Retiro libre, desglose y alternativa	Movimiento; Billetes.Alternativa; sp_Retiro	Q239; alternativa acotada
C-06	Depósito con desglose	Movimiento; sp_Deposito	Q239 e inventario
C-07	Cambio PIN con confirmación	CambiarPin; sp_CambiarPin	PIN anterior deja de funcionar
C-08	Saldo, máximo y disponible global	Saldo; sp_ConsultaUsuario	Límite tras retiro en otro ATM
C-09	Últimos cinco movimientos de la red	Historial; sp_UltimasTransacciones	Prueba de orden/límite y pantalla
C-10	Comprobante en pantalla	Comprobante	Revisión de retiro/depósito/PIN
C-11	Cerrar sesión a 60s	Cajero/MainWindow.xaml.cs / reloj.Tick	Verificación visual local y escenario manual
C-12	Recibo PDF/impresión, teclado táctil	PDF implementado con PDFsharp-WPF; impresión/teclado táctil pendientes	4 PDF y 28 pruebas del cliente

Reglas

Código	Requisito	Implementación	Verificación
RN-01	Denominaciones	Denominacion; Billetes.Denominaciones	Q239 y rechazo de valor desconocido
RN-02	Apertura diaria máx Q10000	INICIALIZAR	Reapertura rechazada
RN-03	Carga y capacidad Q30000	CARGAR; DEPOSITO	Sin inicializar y exceso de capacidad
RN-04	Límite por usuario global	vw_RetiradoHoyPorUsuario; sp_Operacion	Cambio de cajero y de tarjeta
RN-05	Saldo, límite y alternativa	sp_Operacion; Billetes.Alternativa	Retiros y alternativa sin cargo
RN-06	Atomicidad	BEGIN TRAN/COMMIT/ROLLBACK en 02 y 04	Fallo simulado tras actualizar saldo
RN-07	Concurrencia	UPDLOCK/ROWLOCK; CHECK	Dos retiros y saldo final
RN-08	PIN 4, diferente y hash con salt	sp_Operacion / CAMBIO_PIN; Tarjeta	Hash/salt y cambio global
RN-09	Tres fallos y reinicio por éxito	AccesoFallido; VALIDAR_TOKEN	Tres intentos en distintos cajeros
RN-10	Una tarjeta activa e historial	UX_TarjetaActiva; CAMBIAR_TARJETA	Tarjeta anterior inactiva
RN-11	Fecha oficial SQL	SYSDATETIME en procedimientos/defaults	Recibos y filtros de consumo
RN-12	Baja lógica	BAJA_USUARIO; BAJA_CAJERO; Tarjeta.Activa	Historial conservado
RN-13	Admin, ocho usuarios y >=3 ATM	06_DatosPrueba.sql	Semilla: 1 admin, 8 usuarios, 8 ATM

Entregables y validación pendiente

Solución y código: CajeroRed.sln y Codigo/. Scripts: BaseDeDatos/Scripts/. Manuales, diagramas, protocolo y scripts completos: Documentacion/Entrega_Fase_II.pdf. Respaldo: BaseDeDatos/Respaldos/. Pruebas: Pruebas/Program.cs y Documentacion/07_PruebasYEvidencias/. Paquetes portables: Distribucion/.

Pendiente del grupo: nombres/carnés/responsables reales, tres equipos físicos conectados simultáneamente, reglas de firewall en la red elegida, grabación y enlace al video, fecha formal de entrega y acuerdos interbancarios futuros. No se afirma que estas actividades externas ya estén realizadas.

Ampliación NETGEAR (MD recibido el 31/10/2026)

Descubrimiento UDP/IB_PING: BancoInterbancario.cs y listener compartido ServidorTcp.cs. Verificación de cuenta y transferencias: sp_IB_Operacion, BancoInterbancario.cs, Bancos de la red y Transferencias. Deduplicación, cupos concurrentes y rollback: script 08 y pruebas aisladas. No forman parte del PDF original y se incluyen por solicitud del usuario. Ojo del login y totales filtrados: 5 comprobaciones de interfaz. Integración: 83 comprobaciones SQL/TCP/UDP; cajero: 28; interfaz servidor: 5. Total: 116. Pendiente ensayo físico con otros grupos y decisión final de IP/código.

Arquitectura del sistema

El servidor escucha con TcpListener en todas las interfaces IPv4; el puerto se lee de Parametro en SQL Server. Cada conexión tiene una Task asíncrona, un escritor serializado y un lector con tamaño máximo. El registro reserva el código y rechaza otro socket con la misma identidad. La reserva se libera después de registrar la desconexión, evitando que la limpieza de una conexión vieja pise una nueva.

`ProtocoloCompartido`: contrato de mensajes y cálculo acotado de alternativas de billetes. `Infraestructura`: conexiones SQL y parámetros. `Aplicacion`: servidor TCP, cliente TCP, autorización administrativa e interfaz futura entre bancos. `Servidor` y `Cajero`: presentación WPF. `PresentacionCompartida`: tema y utilidades de formularios/exportación. `Pruebas`: verificación reproducible y capturas.

Las reglas financieras están centralizadas en procedimientos SQL ejecutados por el servicio del servidor. Las validaciones de la interfaz ayudan al usuario; SQL vuelve a validar todo. La aprobación se envía solo después de COMMIT. SqlClient usa tiempo máximo de comando de 8 segundos; el cliente espera como máximo 10 segundos. Una respuesta 96 o un corte no se interpreta como autorización para repetir un movimiento.

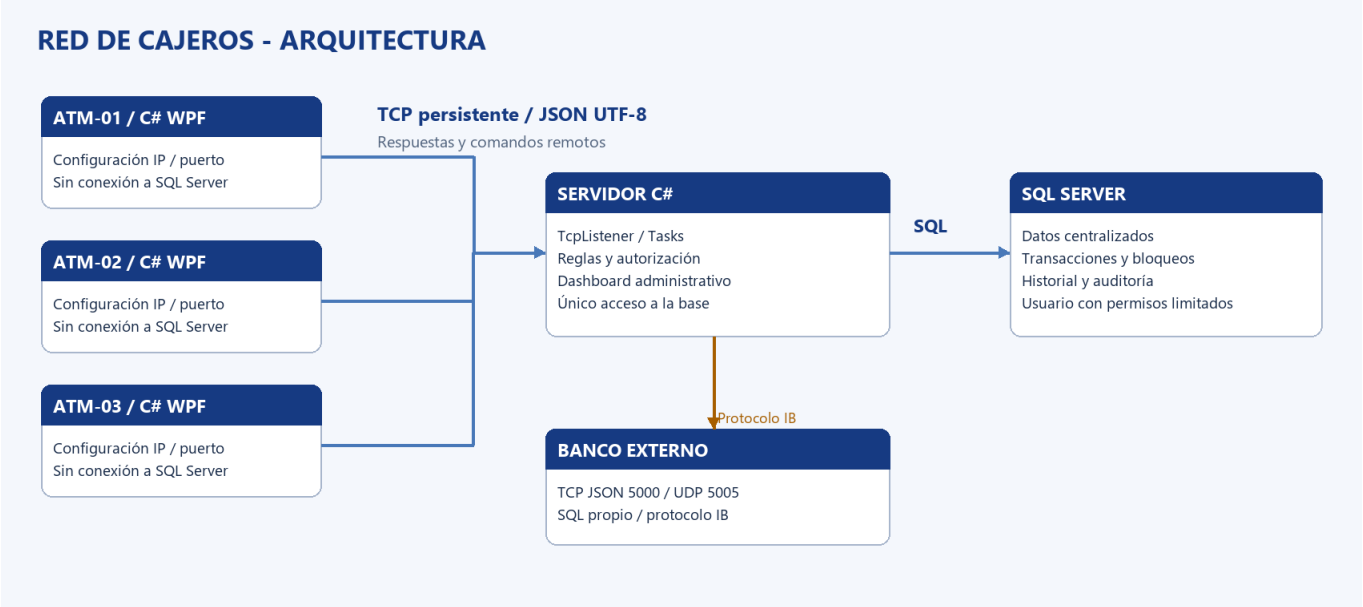
Las conexiones SQL de ejecución adoptan el usuario limitado CajeroServidor. No se usa sa ni se distribuyen credenciales SQL a los cajeros. Pooling se desactiva para que el contexto de seguridad NO REVERT termine al cerrar cada conexión. La cuenta Windows inicial necesita permiso para adoptar ese usuario; ver instalación.

Una operación autorizada bloquea las filas necesarias con UPDLOCK/ROWLOCK y valida el consumo del usuario en todo el día usando la fecha de SQL. Saldo, billetes, encabezado, detalle y resultado idempotente se confirman en una transacción. Las restricciones de base son una segunda defensa contra saldos o cantidades negativas.

Se permite únicamente una instancia central del servicio de escucha por base de datos. Un reinicio del servidor cierra sesiones previas y marca desconectados los cajeros; estos se registran otra vez. La interfaz permanece en su hilo WPF; la E/S es asíncrona.

Un heartbeat no cuenta como actividad de la sesión de usuario. Cliente: cierre a 60 segundos sin teclado/clic; servidor: vigencia de 120 segundos de actividad de negocio. Token: aleatorio, asociado al usuario, tarjeta y cajero, con hash persistido y vencimiento máximo de 120 segundos.

El transporte interno es TCP/JSON sobre la LAN del ejercicio, según el enunciado. SQL usa Encrypt=True; TrustServerCertificate=True se limita al ejemplo local con certificado autofirmado. El protocolo externo y su protección de transporte se acordarán por separado antes de conectar otros bancos.



Modelo relacional y scripts

Las tablas separan usuarios, tarjetas, denominaciones, inventario, encabezados y detalles de efectivo. Existe una tarjeta activa por usuario mediante índice único filtrado; el número de tarjeta es único y de 16 dígitos. HistorialLimite, CambioPin, Bitacora y tarjetas inactivas conservan trazabilidad. El saldo reside en Usuario, siguiendo el modelo mínimo del enunciado; una futura integración de cuentas bancarias debe acordar el identificador de cuenta, sin asumir que coincide con el número de tarjeta.

Script	Responsabilidad
01_CrearBase	Base, tablas, PK/FK, CHECK, UNIQUE, índices y parámetros
02_Operaciones	Motor transaccional, tokens, desconexión y alertas
03_Contratos	Procedimientos públicos del protocolo, que delegan al motor compartido
04_Administracion	Usuarios, tarjetas, límites, efectivo, comandos, trigger y procedimientos administrativos
05_Vistas	Resumen diario, estado de cajeros, consumo global, usuarios, transacciones y arqueo
06_DatosPrueba	Administrador PBKDF2, ocho usuarios y ocho cajeros
07_Permisos	Rol y usuario propios con permisos de ejecución limitados

Cada procedimiento devuelve su resultado después del COMMIT. El motor usa TRY/CATCH con ROLLBACK. SolicitudOperacion tiene PK(Cajero,Solicitud), usuario, huella de la operación y respuesta confirmada; repetir exactamente el mismo movimiento devuelve la misma respuesta sin volver a afectar dinero. Reutilizar el ID con otro cuerpo se rechaza con 94.

No hay borrado físico de usuarios, tarjetas ni cajeros desde la aplicación. Una baja cambia el estado e invalida las sesiones necesarias. La bitácora nunca recibe cuerpos completos de solicitudes, PIN, contraseña ni token. Los números de tarjeta se muestran enmascarados.

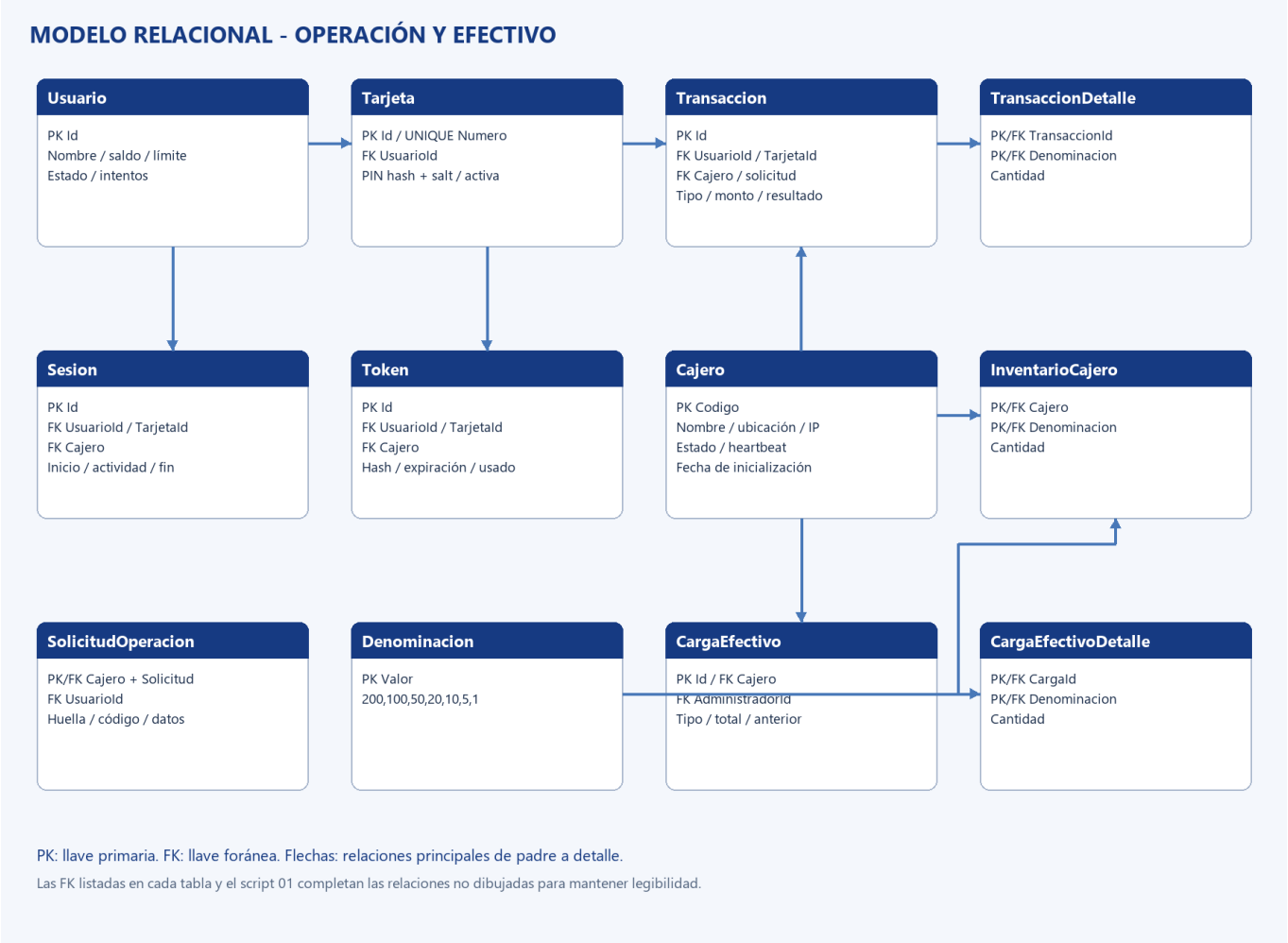
Consultas del dashboard: definiciones completas en 05_Vistas.sql y filtros parametrizados en Servidor/MainWindow.xaml.cs. El cálculo diario filtra por rangos de fecha en lugar de aplicar CAST a cada fila de Transaccion, permitiendo usar su índice. Los cambios de tarjeta no cambian UsuarioId y por ello no reinician el consumo diario.

Extensión interbancaria 08

08_Interbancario.sql es una migración aditiva y repetible sobre una base instalada. Usuario incorpora NumeroCuenta único cuando no es NULL. BancoRed tiene código PK, identidad propia única, IP autorizada y contadores diarios. TransferenciaBancaria referencia Usuario y BancoRed por origen/destino; su clave única es (BancoOrigen,TokenTransferencia). Conserva huella, dirección, monto, estado y respuesta confirmada. vw_TransferenciasUsuario ofrece el historial con cuentas enmascaradas. Los movimientos no alteran TransaccionDetalle ni InventarioCajero: son transferencias contables, no efectivo físico. sp_IB_Operacion verifica administrador para configuración/salidas y valida IP autorizada para entradas. Usa sp_getapplock transaccional, bloqueos de usuario y rollback; nunca espera a otro banco dentro de una transacción SQL. Los diagramas de operación/administración originales corresponden al núcleo ATM; la extensión bancaria se describe aquí y en el script 08 anexo.

09_CompatibilidadCajero.sql actualiza únicamente sp_Operacion: identidad usuarioId al abrir sesión, correlación/usuario en ESTADO_SOLICITUD y fecha SQL en saldo/cambio PIN. No cambia saldos, límites ni inventarios. El instalador nuevo aplica 01-09; una base restaurada se actualiza solo con 08/09.

Diagrama relacional - operación



PK: llave primaria. FK: llave foránea. Flechas: relaciones principales de padre a detalle.

Las FK listadas en cada tabla y el script 01 completan las relaciones no dibujadas para mantener legibilidad.

Diagrama relacional - administración



Protocolo interno v1

Transporte TCP persistente. Un objeto JSON por línea, UTF-8 sin BOM y terminador LF. Puerto por defecto 5000. Solicitud máxima 32768 bytes; respuesta máxima 65536 bytes. El receptor conserva fragmentos de mensajes entre lecturas y admite varias líneas en un mismo paquete TCP. Tamaños excesivos o JSON inválido cierran la conexión. Cada socket tiene un solo escritor serializado para evitar mezclar respuestas y comandos.

Sobre común

```
{"tipo":"RETIRO","idSolicitud":"8e1b52a91e2a4f9d8b1c76929f206abc","cajero":"ATM-01","sesion":"UUID-de-
sesion","datos":{"monto":239,"desglose":{"200":1,"20":1,"10":1,"5":1,"1":4}}}
```

El grupo define los campos específicos dentro de `datos`. `idSolicitud` es texto único de hasta 64 caracteres y correlaciona la respuesta. `cajero` es el código dado de alta, hasta 24 caracteres. `sesion` se recibe al validar el token. Los campos de respuesta no se usan como autoridad en una solicitud.

```
{"tipo":"RESPUESTA","idSolicitud":"8e1b52a91e2a4f9d8b1c76929f206abc","cajero":"ATM-
01","codigo":"00","mensaje":"Operación aprobada","datos":{"idTransaccion":1,"saldo":4761.00,"disponibleHoy":26
1.00,"fecha":"2026-10-28T10:42:15","monto":239,"tipo":"RETIRO","cajero":"ATM-01"}}
```

El cliente espera como máximo 10 segundos. Si pierde una respuesta financiera, muestra la incertidumbre y conserva la referencia en memoria; no reenvía automáticamente el cargo. Para obtener su resultado envía ESTADO_SOLICITUD con la misma identidad de usuario. Un resultado no encontrado requiere revisión administrativa; no significa automáticamente que se pueda repetir.

Mensajes

Tipo	Dirección	datos y efecto
REGISTRAR_CAJERO	Cliente a servidor	equipo, version. Primer mensaje en 15 s. IP tomada del socket. Rechaza desconocidos, inactivos y duplicados. Devuelve habilitado/inicializado.
HEARTBEAT	Cliente a servidor	datos vacío. Cada 10 s; devuelve estado operativo. Tras 30 s sin señal, el monitor cierra la conexión y registra desconexión.
LOGIN	Cliente a servidor	tarjeta, pin (strings). Genera token en servidor; el cajero recibe requiereToken, nunca el código.
VALIDAR_TOKEN	Cliente a servidor	tarjeta, token (string de seis dígitos). Devuelve sesion, nombre, debeCambiarPin.
RETIRO	Cliente a servidor	monto entero positivo, desglose objeto denominación/cantidad. Validación autoritativa SQL.
DEPOSITO	Cliente a servidor	monto y desglose; rechaza superar capacidad del cajero.
CAMBIO_PIN	Cliente a servidor	pinActual, pinNuevo. PIN nuevo de cuatro dígitos distinto al anterior.
CONSULTA_SALDO	Cliente a servidor	vacío. Devuelve saldo, limiteDiario, retiradoHoy, disponibleHoy.
ULTIMAS_TRANSACCIONES	Cliente a servidor	vacío. datos es una lista de máximo cinco movimientos aprobados de cualquier cajero.
LOGOUT	Cliente a servidor	vacío. Cierra la sesión asociada al socket/cajero.
ESTADO_SOLICITUD	Cliente a servidor	idOriginal. Resultado confirmado de un movimiento del mismo usuario y cajero.

Tipo	Dirección	datos y efecto
RESPUESTA	Servidor a cliente	codigo, mensaje, datos; conserva idSolicitud.
COMANDO	Servidor a cliente	comando y texto. FUERA_DE_SERVICIO, HABILITAR, MENSAJE o ACTUALIZAR_INVENTARIO.

Un cajero no puede cambiar su identidad en un socket registrado. El cierre de sesión local ocurre a 60 s de inactividad; SQL rechaza sesiones con más de 120 s. Heartbeats no renuevan la sesión de usuario.

Códigos

Código	Significado
00	Operación aprobada
25	No existe resultado confirmado para la referencia consultada
30	Mensaje, monto o desglose inválido
51	Saldo insuficiente
55	PIN o credenciales incorrectas
56	Token inválido, vencido, usado o de otro cajero
57	Sesión vencida o inválida
58	Debe cambiar el PIN temporal
61	Límite diario global excedido
75	Tarjeta bloqueada o inactiva
91	Cajero fuera de servicio, inactivo o sin inicializar
92	Billetes insuficientes para el desglose; datos.alternativa si existe
93	Se supera la capacidad del cajero
94	Cajero duplicado o ID reutilizado con otro cuerpo
96	No se pudo confirmar; consultar antes de repetir

En un 92 no se aplica retiro. Si el usuario acepta la alternativa, el cliente envía una solicitud NUEVA con otro identificador; SQL vuelve a validar existencias, saldo y límite. El algoritmo prueba primero denominaciones mayores y retrocede cuando el inventario acotado impide una combinación greedy; así encuentra alternativas que un algoritmo voraz simple perdería.

Idempotencia

La clave es (cajero,idSolicitud), con usuario, tipo, huella del JSON y resultado persistido. El cliente mantiene serialización estable al repetir la misma solicitud. Reordenar propiedades del cuerpo cambia la huella: para consultar una operación conocida es preferible ESTADO_SOLICITUD. Nunca reutilizar el ID para aceptar una alternativa con otro desglose.

El estado de una solicitud puede revisarse también en Transacciones/Auditoría del dashboard por su referencia. Si la aplicación cliente se cerró y perdió su referencia en memoria, usar el historial y la auditoría antes de volver a operar.

Listener compartido NETGEAR

Antes de REGISTRAR_CAJERO, los tipos IB_* se encaminan a BancoInterbancario y reciben una respuesta de una sola conexión con idMensaje. Los ATM conservan mensajes e idSolicitud originales. Descubrimiento UDP 5005 y contrato IB: Documentacion/08_IntegracionEntreGrupos/ProtocoloExterno.md.

Compatibilidad Cajero 2.1.0

VALIDAR_TOKEN aprobado añade usuarioId. ESTADO_SOLICITUD conserva su validación por sesión/usuario/cajero y añade idOriginal/usuarioId a los datos confirmados. CONSULTA_SALDO y CAMBIO_PIN aprobado incluyen fecha de SQL para el comprobante. Son campos adicionales: los clientes anteriores pueden seguir interpretando la respuesta. El cliente 2.1 requiere el servidor actualizado para recuperar pendientes con identidad verificada.

Ampliación 2.2.0

Consultar ../09_Entrega/NETGEAR_2.2.0.md para arranque, búsqueda de servidor y transferencias desde el cajero. Mensajes nuevos: NETGEAR_INFO (descubrimiento TCP sin sesión), BANCOS_TRANSFERENCIA, VERIFICAR_DESTINO, TRANSFERIR, ESTADO_TRANSFERENCIA e HISTORIAL_TRANSFERENCIAS. Todos salvo NETGEAR_INFO requieren sesión autenticada. TRANSFERIR requiere bancoDestino, cuentaDestino, nombreDestino, monto y concepto; la identidad origen procede de la sesión. IdSolicitud se conserva para deduplicación. ESTADO_TRANSFERENCIA usa idOriginal y devuelve el resultado correlacionado del mismo usuario/cajero.

Instalación y conexión en red

Equipo servidor

Windows con Visual Studio 2022 y Desarrollo de escritorio .NET para compilar. SDK 9.0.3xx fijado en global.json; destino de las apps .NET 8. SQL Server 2019 o posterior (Express/Developer) y SSMS. En esta PC: Visual Studio 17.14.39, SDK 9.0.317, SQL Server Express 2022 16.0.1210.3, instancia `.\SQLEXPRESS`, SSMS 22.9.2. Estos son resultados locales, no requisitos exactos para todos los equipos.

Abrir PowerShell en la raíz de la solución y ejecutar `./Instalar-BaseDeDatos.ps1 -Servidor '.\SQLEXPRESS'`. Usa autenticación Windows y aplica scripts por orden. Se detiene si ya existe el esquema para evitar reinicializar datos. Para una instancia distinta, editar el ejemplo de conexión o usar la pantalla Configurar SQL Server del servidor. El nombre de la base de esta entrega es BD_CajeroRed_Fase2.

Compilar con `dotnet restore CajeroRed.sln` y `dotnet build CajeroRed.sln -c Release`. Visual Studio puede realizar esos pasos al abrir y compilar la solución. En esta PC se restauraron las bibliotecas disponibles localmente; en una instalación nueva NuGet puede necesitar Internet.

El servidor puede iniciarse por `Iniciar-Servidor.cmd` o desde Visual Studio. Login admin / GrupoCajero#2026 para la demostración. Pulsar Iniciar servicio. Inicializar cada cajero para el día; no se inicializa automáticamente al conectar un cliente.

Usuario de SQL con permisos limitados

El script 07 crea el rol `rol_CajeroServidor` y el usuario de base `CajeroServidor WITHOUT LOGIN`. Cada conexión del proceso C# entra con la identidad configurada y ejecuta `EXECUTE AS USER='CajeroServidor' WITH NO REVERT`. El contexto efectivo puede consultar y ejecutar procedimientos, registrar bitácora y actualizar monitoreo; no puede escribir saldos directamente. Se comprobó que UPDATE Usuario falla con ese contexto. Pooling=False cierra físicamente la conexión al terminar. No usa sa.

Para desplegar con una identidad Windows dedicada existente, un administrador de SQL ejecuta, sustituyendo el nombre real:

```
USE master;
CREATE LOGIN [EQUIPO\CuentaServidor] FROM WINDOWS;
GO
USE BD_CajeroRed_Fase2;
CREATE USER [EQUIPO\CuentaServidor] FOR LOGIN [EQUIPO\CuentaServidor];
ALTER ROLE rol_CajeroServidor ADD MEMBER [EQUIPO\CuentaServidor];
```

Ejecutar el servidor bajo esa cuenta y mantener Integrated Security=True. Si el login ya existe, omitir CREATE LOGIN. No agregarlo a sysadmin ni db_owner. La cuenta con la que se instalan scripts y se hace el respaldo puede tener permisos mayores; debe distinguirse de la cuenta de ejecución.

Si el DBA ya habilitó autenticación SQL, también puede crear un login propio con contraseña y asignarlo al rol. No hace falta cambiar el modo Windows instalado en esta PC. La utilidad de prueba admite `--crear-login` solo cuando el modo mixto está habilitado y se solicita expresamente; guarda una configuración privada sin imprimir la clave. Nunca copiar esa configuración al cajero ni a GitHub.

Configuración y cambio de router o teléfono

Todos los equipos deben estar en la misma LAN o Wi-Fi. El servidor identifica la IP por adaptador activo con gateway IPv4; omite loopback y direcciones 169.254.*. Elegir la interfaz conectada a los demás equipos. `127.0.0.1` solo sirve cuando servidor y cajero están en el mismo equipo.

El cajero tiene Configuración de conexión: dirección, puerto y código único. Guarda `cajero.local.json` junto al ejecutable, fuera del control de versiones. El archivo de ejemplo no contiene cadena SQL. Para varias instancias en una PC, usar argumentos `--cajero ATM-02 --servidor IP --puerto 5000`; cada código debe estar dado de alta. No ejecutar dos instancias con el mismo código.

Al cambiar de router, la dirección IP asignada por DHCP puede cambiar:

1. Conectar el servidor y los clientes a la nueva red. 2. Iniciar el servicio y consultar la IP mostrada en el servidor. 3. Cambiar la dirección en cada cajero y pulsar Probar conexión. 4. Confirmar el registro en el monitor del dashboard.

Un punto de acceso de teléfono funciona únicamente si permite comunicación entre sus clientes. Algunos routers/redes de invitados aíslan dispositivos. Si Probar conexión falla, revisar IP, aislamiento de clientes, puerto y firewall antes de cambiar código. Una reserva DHCP en el router puede mantener una IP estable, si se tiene acceso a administrarlo.

Firewall

Abrir solo el puerto TCP de la aplicación en el equipo servidor. Ejemplo en PowerShell con permisos administrativos, para una LAN marcada como Privada:

```
New-NetFirewallRule -DisplayName 'CajeroRed Fase II TCP 5000' -Direction Inbound -Action Allow -Protocol TCP
-LocalPort 5000 -Profile Private
```

Si se cambia el puerto en Configuración del dashboard, detener/iniciar el servicio, actualizar los cajeros y ajustar la regla. No es necesario exponer SQL Server a los equipos cliente: SQL permanece local al servidor. La creación de esta regla se deja para la red real; no se abre automáticamente el firewall al compilar.

Distribución y respaldo

`Publicar.ps1` genera ejecutables portables Windows x64 con .NET incluido. Cada cajero recibe únicamente `Distribucion/Cajero`; el servidor recibe su carpeta y los scripts/manuales necesarios. Mantener la carpeta completa y permiso de escritura local para configuración. Configurar IP y código en cada PC antes de la demostración.

Restaurar el `.bak` desde SSMS, Tareas/Restaurar base de datos, con una cuenta autorizada y rutas de archivos ajustadas al equipo destino. No sobrescribir una base existente sin un respaldo previo y una decisión explícita. Un respaldo SQL Server 2022 necesita SQL Server 2022 o posterior para restaurarse; para SQL 2019 usar los scripts y datos de prueba.

La cuenta WITHOUT LOGIN y sus permisos viajan dentro del respaldo. Los logins de servidor externos no viajan con el `.bak`; mapearlos en la nueva instalación si se usan. La fecha de inicialización guardada corresponde al día del respaldo; inicializar nuevamente cuando corresponda.

Respaldar.ps1 escribe directamente en BaseDeDatos/Respaldos y concede Modify a la cuenta del servicio SQL únicamente en esa carpeta propia. Para otra instancia, pasar -CuentaServicio con su identidad real. Así el servicio puede crear el archivo y el propietario del proyecto puede leerlo sin copiar desde Program Files.

Actualización NETGEAR del 31/10/2026

La versión del SDK se selecciona desde 9.0.300 con latestFeature; en este equipo se verifica con 9.0.311. Los datos de versión anteriores de este manual son históricos. Después de restaurar el respaldo anterior, aplicar SOLO Actualizar-Interbancario.ps1 con la instancia correcta, sin reinstalar la base. Configurar-Firewall.ps1 permite TCP 5000 y UDP 5005 para el ejecutable y LocalSubnet, incluso si el perfil de la red de pruebas es Público. Leer RevisionServidor_2026-10-31.md para reservas DHCP independientes de Wi-Fi/Ethernet y desactivar MAC aleatoria del perfil Wi-Fi antes de reservarla. Bancos de la red → Mi banco NETGEAR: confirmar código único; BANC02 es provisional. Configurar otros bancos con su IP real, confirmar confianza y realizar pruebas de conectividad antes de transferir.

Manual del servidor y dashboard

1. Ejecutar Servidor.exe o Iniciar-Servidor.cmd. Si SQL está en otra instancia, pulsar Configurar SQL Server antes de iniciar sesión. La cadena se guarda únicamente en el equipo servidor. 2. Ingresar usuario y contraseña administrativa. Los datos de demostración están en LEEME.md. Los intentos fallidos quedan registrados sin contraseña. 3. Pulsar Iniciar servicio. El encabezado muestra estado, número de conexiones, direcciones de red y puerto. Elegir la IP de la LAN correcta si aparecen adaptadores virtuales. Detener cierra las conexiones y permite reiniciar la escucha.

Resumen

Indicadores del día: cajeros en línea/total, efectivo de la red, retirado, depositado, promedio depositado por usuario, cantidad de operaciones, rechazos, cambios de PIN y alertas activas. También muestra cuántos usuarios cambiaron PIN y el último acceso con fecha completa y cajero. Gráficas de retiros/depósitos por hora, montos por cajero y billetes por denominación. La selección del cajero se conserva entre refrescos. Actualización cada cuatro segundos.

Captura: `Capturas/servidor_resumen.png`.

Cajeros

Seleccione una fila antes de usar acciones sobre un cajero. Nuevo cajero solicita código único, nombre y ubicación. Editar conserva el código. Baja lógica conserva su historial y deshabilita operaciones. Inicializar hoy recibe cantidades por denominación y confirma el monto; máximo Q10000, una sola inicialización al día. Agregar efectivo requiere inicialización del día y no puede superar Q30000. Toda carga se registra con administrador y detalle.

Habilitar/Fuera de servicio actualiza el estado autoritativo en SQL y envía el comando al cliente conectado. Un cliente desconectado recibe el estado correcto al registrarse o en el siguiente heartbeat. Enviar mensaje muestra una ventana en el cajero; actualizar inventario provoca una consulta del estado. Los estados están identificados por texto y color, de modo que no dependen solo del color.

Captura: `Capturas/servidor_cajeros.png`.

Usuarios

Crear usuario abre un formulario breve: nombre, tarjeta, PIN, confirmación, saldo y límite. Modificar nombre no afecta su dinero. Cambiar límite registra anterior/nuevo y no reinicia lo retirado hoy. Cambiar tarjeta desactiva la anterior, guarda historial y conserva el límite consumido. El PIN asignado en un reemplazo o restablecimiento es temporal y debe cambiarse al ingresar.

Bloquear/Desbloquear son acciones globales. La baja es lógica. Historial del usuario abre tres pestañas: movimientos, tarjetas enmascaradas y cambios de límite. La tabla principal muestra saldo, límite, disponible, máximo retirable, estado e información del último acceso.

Captura: `Capturas/servidor_usuarios.png`.

Consultas, alertas y reportes

Transacciones reúne movimientos aprobados y rechazados con motivo, desglose y referencia. Depósitos y Retiros permiten elegir fechas, buscar texto por usuario/cajero y ver totales y promedio. Cambios de PIN muestra quién, cuándo, origen y resultado; nunca el PIN. Auditoría permite filtrar por fechas/texto. Alertas incluye efectivo bajo, denominaciones agotadas, desconexiones, bloqueos y retiros sin billetes; seleccionar y Marcar atendida conserva quién la atendió.

Arqueo compara inventario actual con apertura + cargas + depósitos - retiros desde la última inicialización. Exportar el cierre antes de inicializar un día nuevo. Resumen diario agrupa por fecha, cajero, tipo y resultado. Exportar Excel genera un archivo .xlsx con la vista y filtros actuales; no contiene hashes ni PIN. La exportación se registra en auditoría.

Configuración

Puerto, umbral, vigencia del token y máximos se guardan en Parametro. El puerto nuevo se aplica al detener e iniciar el servicio. El token no puede superar 120 s, la apertura Q10000 y la capacidad Q30000; se permiten valores menores que no contradigan inventarios vigentes. El umbral predeterminado es Q1000.

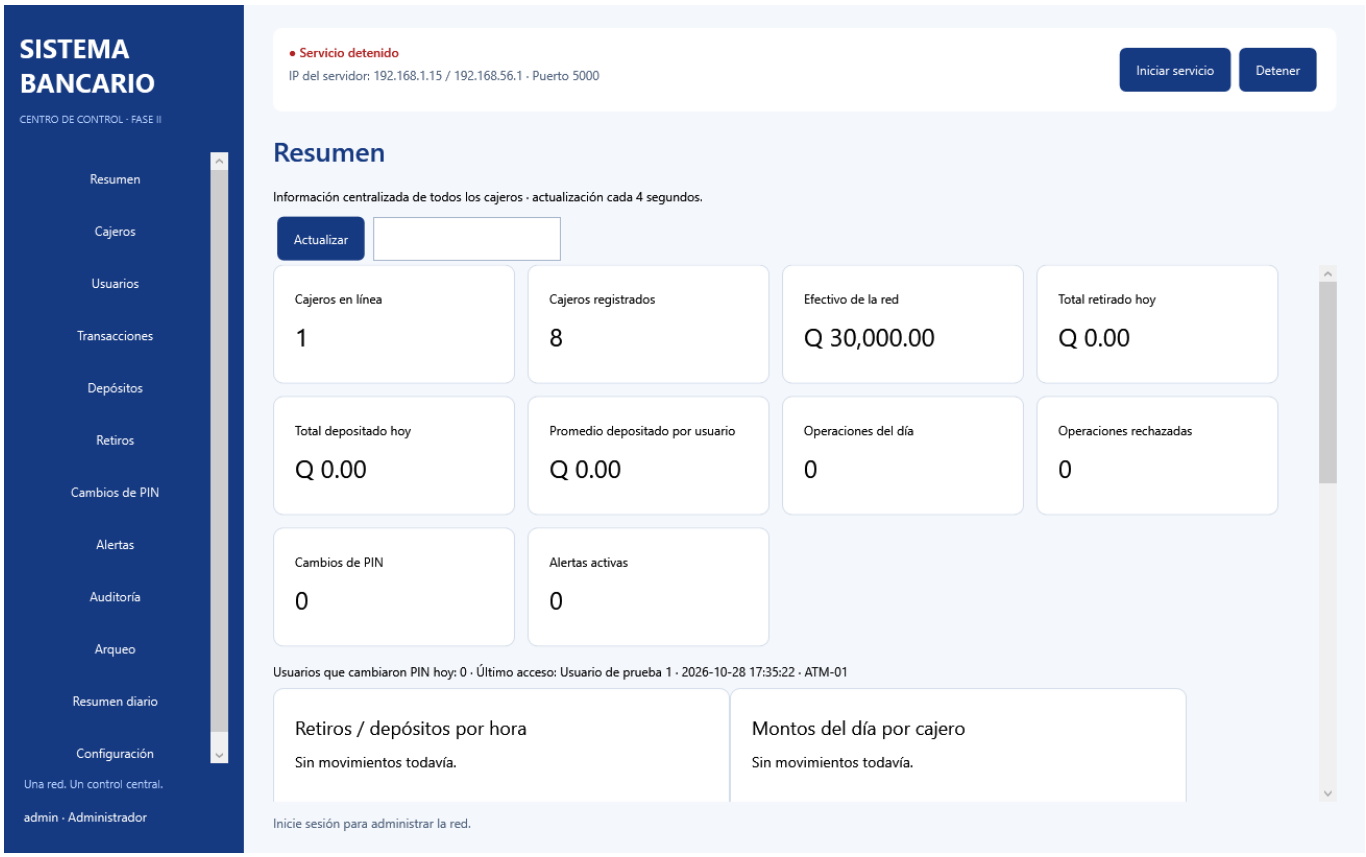
Token del usuario

Al aprobar tarjeta/PIN se abre una ventana no modal en el servidor con el código de seis dígitos y el cajero. Simula el envío al celular requerido por el documento. El usuario lo escribe en el cajero. Vence en SQL según el parámetro y no puede reutilizarse; la ventana se cierra automáticamente a los dos minutos como máximo. No compartir el token con otros usuarios.

NETGEAR: login y bancos de la red

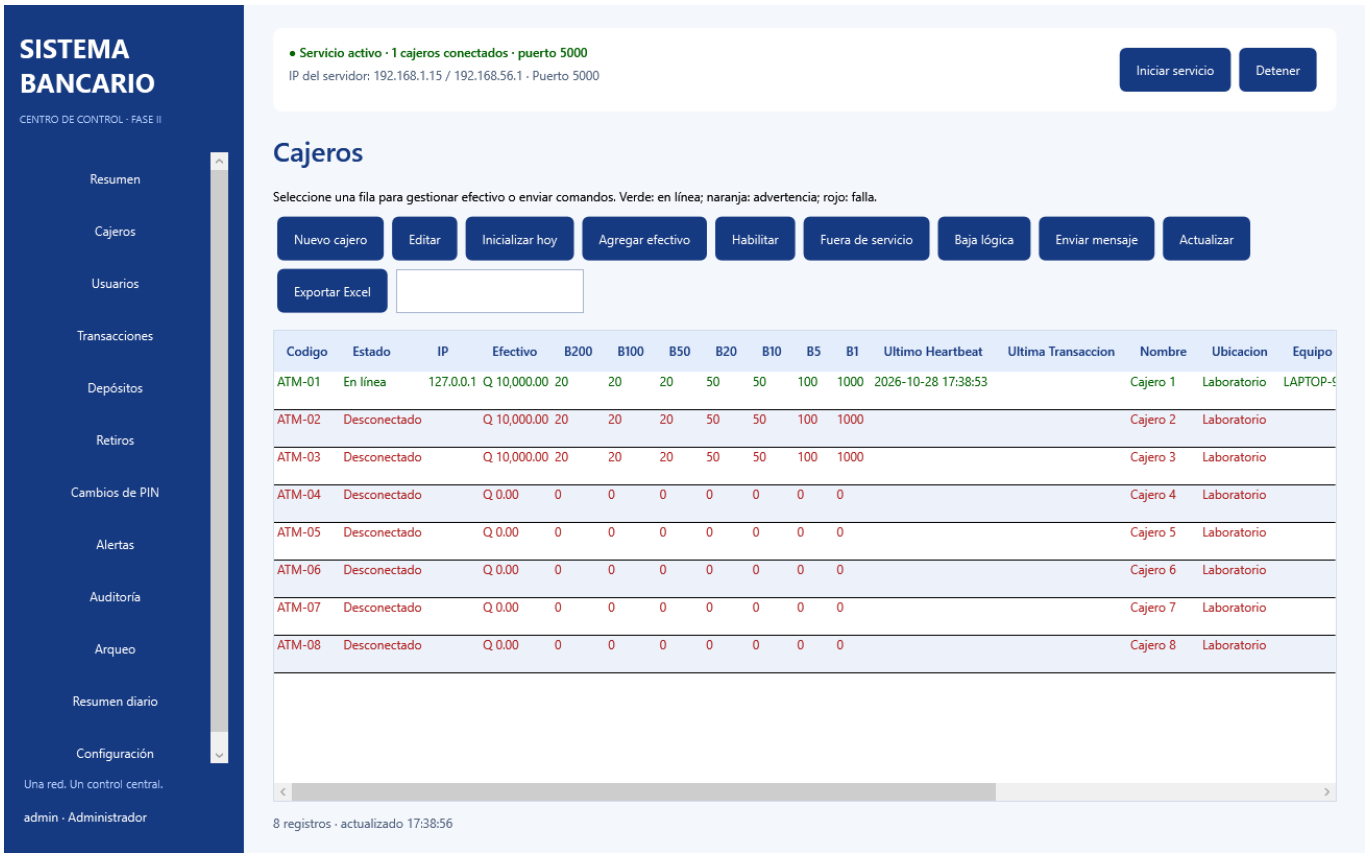
El ojo permite mostrar/ocultar la contraseña; se oculta al cambiar de ventana y se limpia al finalizar el intento. Bancos de la red → Mi banco NETGEAR configura código único y límite. BANC02 es provisional; puede cambiarse mientras no haya historial y no esté ocupado por otro registro. Escanear red local muestra anuncios UDP y prueba por TCP los bancos guardados. Registrar un anuncio no implica confiar en él: confirmar IP/código con el responsable. Registrar / editar banco solicita su IP, puerto, límite y autorización 1/0. Solo bancos autorizados pueden enviar transferencias. Verificar cuenta muestra el titular remoto. Transferir solicita ID de usuario local, cuenta destino, monto y concepto; confirmar con autorización del usuario. Guardar el token del comprobante. Transferencias permite filtrar/exportar y Consultar / conciliar las salidas. INCIERTA o RESERVADA requieren conciliación; no generar otra transferencia ni devolver fondos por un timeout. Usuarios → Asignar cuenta permite un número único de 1-20 dígitos; Historial del usuario añade Transferencias con el mismo token. Logs/conexiones-YYYYMMDD.jsonl junto al ejecutable registra causas técnicas sin PIN ni payload. La fecha del archivo y sus eventos es UTC.

Captura: servidor resumen



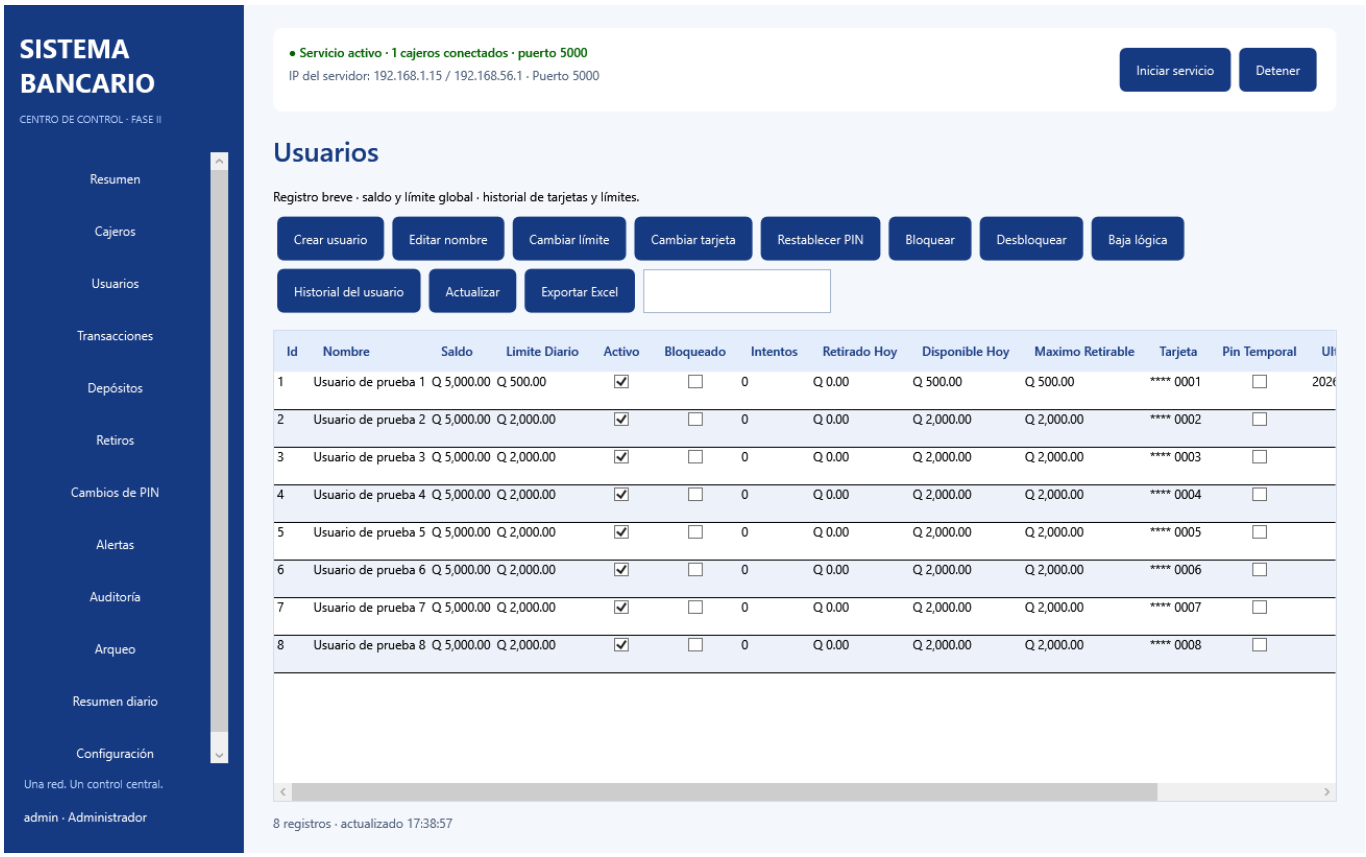
Captura histórica del paquete original en esta PC; no muestra todas las ampliaciones NETGEAR ni sustituye la prueba física.

Captura: servidor cajeros



Captura histórica del paquete original en esta PC; no muestra todas las ampliaciones NETGEAR ni sustituye la prueba física.

Captura: servidor usuarios



Captura histórica del paquete original en esta PC; no muestra todas las ampliaciones NETGEAR ni sustituye la prueba física.

Manual del cajero

Conectar

Ejecutar Cajero.exe. Sin servidor accesible muestra Fuera de servicio y reintenta cada cinco segundos. Abrir Configuración de conexión para indicar IP/nombre, puerto y código único. Probar conexión verifica que el puerto TCP sea accesible; el registro debe aceptarse además en el servidor. Un cajero sin inicializar o deshabilitado no permite acceder.

Al usar argumentos de arranque, estos prevalecen sobre el archivo de configuración. Para que una modificación de la interfaz persista entre arranques, ejecutar el EXE sin argumentos o actualizar también los argumentos del acceso directo.

Ingresar

Tarjeta de dieciséis dígitos y PIN de cuatro, enmascarado. Continuar genera el token y abre Validar token. Obtener los seis dígitos de la ventana Token del usuario en el servidor e ingresarlos antes de que venza. Tres errores consecutivos de PIN/token bloquean la tarjeta en todos los cajeros. Solo administración puede desbloquearla. Si el PIN es temporal, primero debe cambiarse.

Capturas: `Capturas/cajero_acceso.png`, `Capturas/cajero_token.png`, `Capturas/cajero_menu.png`.

Retirar

Ingresar monto en quetzales enteros, por ejemplo 239. Indicar cantidad de billetes de Q200, Q100, Q50, Q20, Q10, Q5 y Q1. La suma debe coincidir con el monto. Confirmar operación. El servidor valida saldo, límite del usuario en todos los cajeros y existencias de este cajero. Si no puede entregar el desglose, propone una alternativa cuando exista. Se puede aceptar o cancelar; aceptar inicia una validación nueva con otro ID.

Un retiro aprobado muestra comprobante con referencia, transacción, monto, saldo, cajero, tarjeta enmascarada y fecha oficial. Continuar vuelve al menú.

Depositar

Ingresar monto y desglose. Confirmar. Aumenta el saldo del usuario y el inventario del cajero de forma conjunta. Si supera Q30000 o el máximo menor configurado, se rechaza sin cambiar datos. El depósito se registra antes de responder.

Otras operaciones

Consultar saldo muestra saldo actual, límite diario, retirado hoy, disponible del límite y máximo según saldo y límite. La cantidad efectivamente retirable también depende del saldo y de los billetes del cajero. Últimas cinco transacciones muestra movimientos aprobados de todos los cajeros, con fecha/hora, tipo y monto. Cambiar PIN requiere actual, nuevo de cuatro dígitos y confirmación; debe ser distinto al anterior. El cambio se aplica a toda la red.

Cerrar sesión termina el acceso. Sesenta segundos sin clics o teclado cierran automáticamente la sesión local; SQL rechaza sesiones que superen dos minutos sin operación. Los heartbeats no mantienen abierta la sesión del usuario.

Corte durante una operación

Si pasan diez segundos sin respuesta o se pierde la conexión, no asumir que el movimiento fue rechazado. La referencia puede existir como operación confirmada en SQL. Volver a ingresar con el mismo usuario y pulsar Consultar operación pendiente. Si aparece un resultado confirmado, la interfaz permite volver a operar. Si no hay resultado, pedir revisión al administrador antes de repetir. La app conserva la referencia en LocalAppData y la recupera al ingresar con el usuario correcto; si la IP cambió, revisar con administración. No borre un pendiente por un timeout.

El cliente no continúa operaciones fuera de línea y nunca modifica saldo localmente. No guarda una copia de cuentas ni una cadena de conexión SQL.

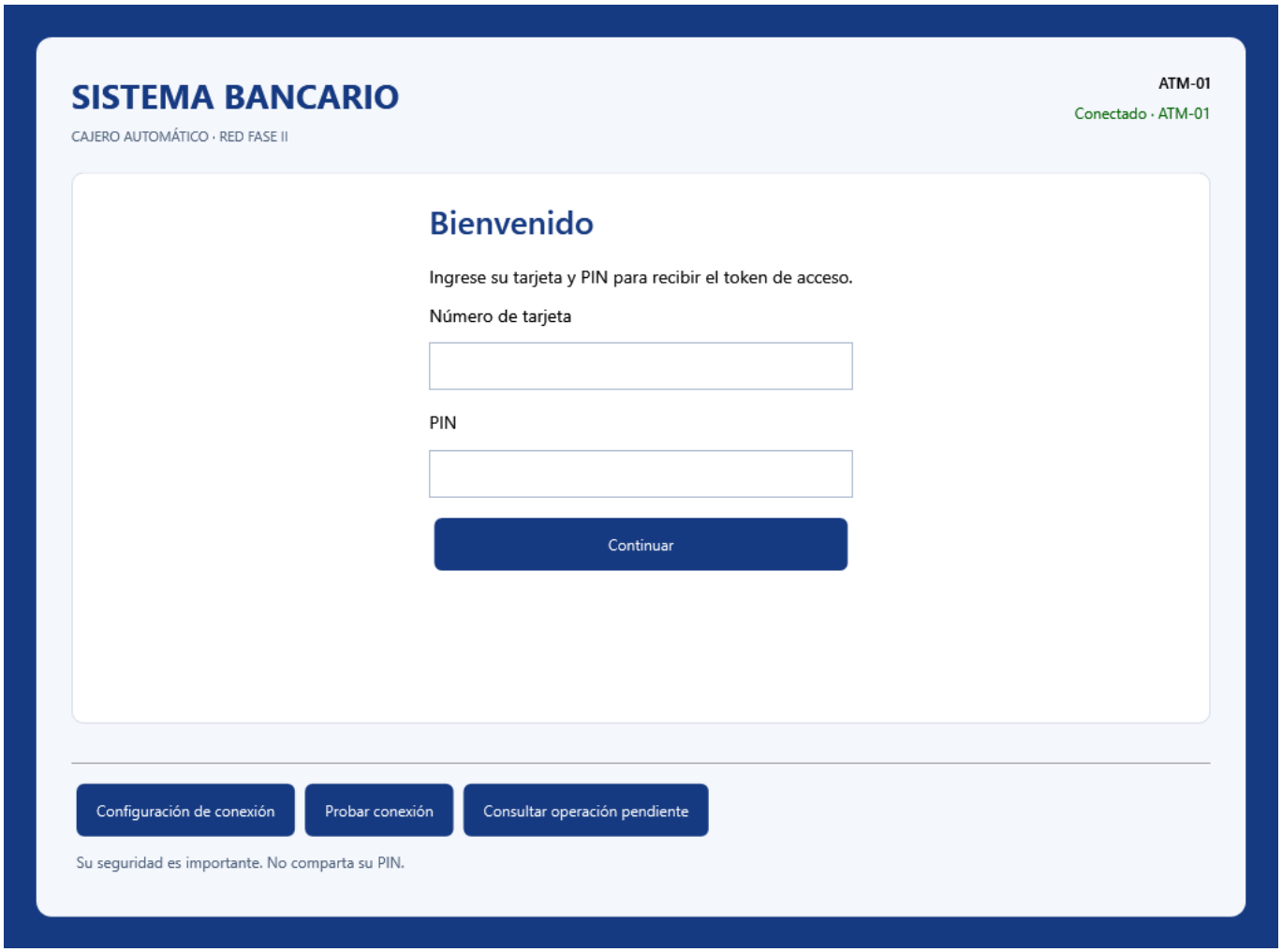
Actualización NETGEAR 2.1.0

El CMD acepta código, servidor y puerto opcional; sin argumentos usa la configuración guardada. Los formularios validan antes de cerrar y conservan datos al corregir un error. PIN/token solo admiten dígitos; para reemplazarlos por pegado, vaciar primero el campo. El cierre local es inmediato aunque el servidor no responda. Los formularios cuentan actividad y se cierran al vencer la sesión, perder la conexión o deshabilitar el cajero. El comprobante confirmado permite Guardar comprobante en PDF. No incluye PIN/token y usa la fecha del servidor. Un fallo de guardado no revierte la operación. Los retiros y depósitos quedan bloqueados hasta resolver un pendiente con su usuario; consultas y cierre siguen disponibles. Ver Documentacion/09_Entrega/CorreccionesCajero_2026-10-31.md para detalle de los doce puntos y compatibilidad del servidor.

Ampliación 2.2.0

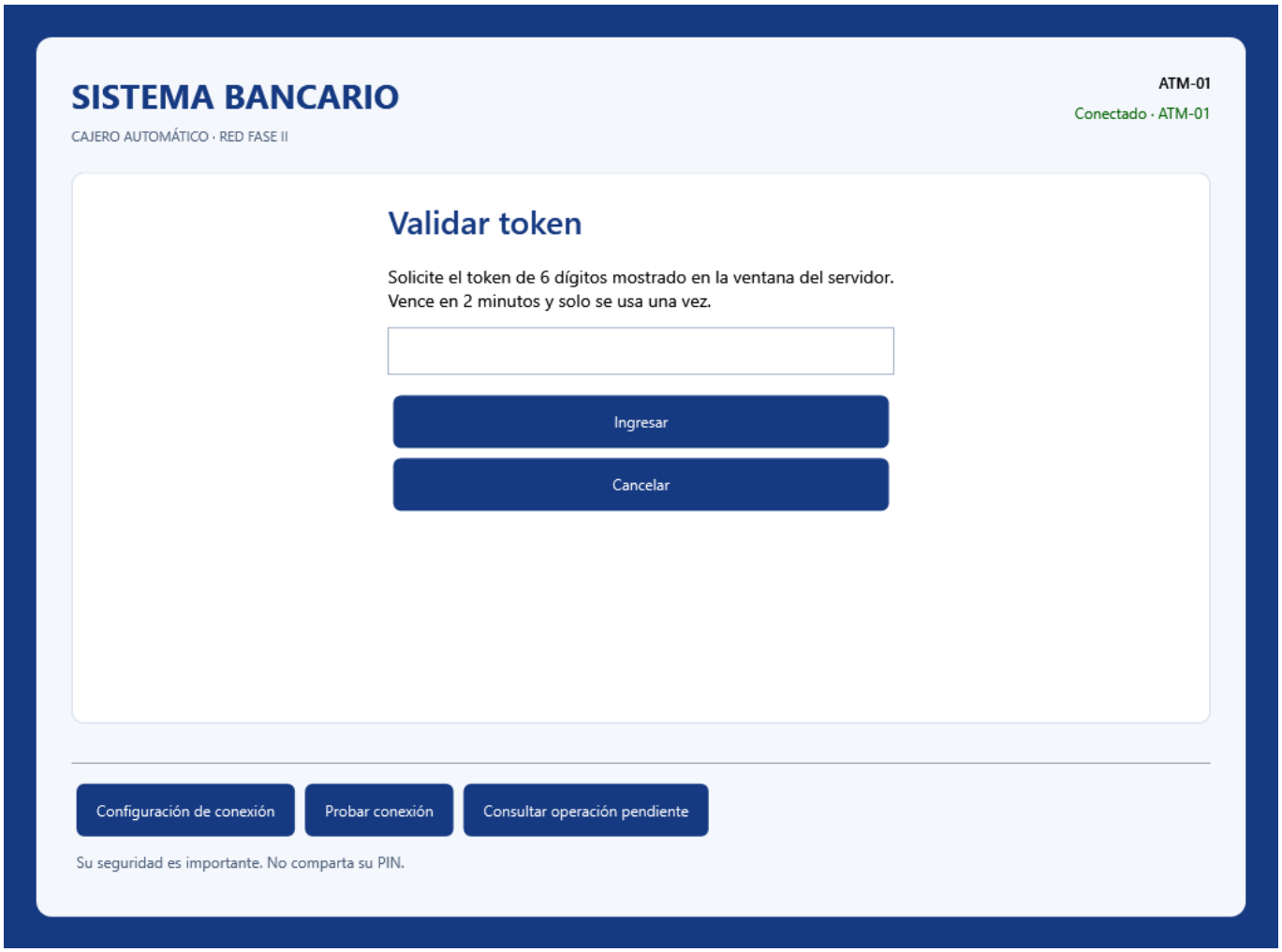
Consultar ../09_Entrega/NETGEAR_2.2.0.md para arranque, búsqueda de servidor y transferencias desde el cajero. Mensajes nuevos: NETGEAR_INFO (descubrimiento TCP sin sesión), BANCOS_TRANSFERENCIA, VERIFICAR_DESTINO, TRANSFERIR, ESTADO_TRANSFERENCIA e HISTORIAL_TRANSFERENCIAS. Todos salvo NETGEAR_INFO requieren sesión autenticada. TRANSFERIR requiere bancoDestino, cuentaDestino, nombreDestino, monto y concepto; la identidad origen procede de la sesión. IdSolicitud se conserva para deduplicación. ESTADO_TRANSFERENCIA usa idOriginal y devuelve el resultado correlacionado del mismo usuario/cajero.

Captura: cajero acceso



Captura histórica del paquete original en esta PC; no muestra todas las ampliaciones NETGEAR ni sustituye la prueba física.

Captura: cajero token



Captura histórica del paquete original en esta PC; no muestra todas las ampliaciones NETGEAR ni sustituye la prueba física.

Captura: cajero menu



Captura histórica del paquete original en esta PC; no muestra todas las ampliaciones NETGEAR ni sustituye la prueba física.

Verificación de la entrega

Verificación local del 28 de octubre de 2026: compilación sin errores ni advertencias; 60 comprobaciones automáticas básicas/SQL/TCP y dos comprobaciones de interfaz: carga al navegar a Usuarios y cierre por inactividad. También se compila con MSBuild de la instalación real de Visual Studio 2022. El reloj de actividad se adelanta en el ensayo de interfaz para activar el mismo temporizador sin esperar un minuto. Los logs conservan la salida exacta.

Las pruebas se ejecutan mediante `Pruebas/Program.cs --integracion`, usando una base temporal con prefijo BD_CajeroRed_Pruebas y sufijo aleatorio. El nombre se valida antes de eliminarla. No se elimina ni reinicializa BD_CajeroRed_Fase2 ni la base de fase I.

Cobertura: mensajes JSON, desglose Q239, alternativa con existencias acotadas, exportación OpenXML sin fórmulas procedentes de datos, administrador PBKDF2, usuario SQL limitado, apertura/cargas, tres sockets persistentes, token no expuesto al cliente, límite global, retiros simultáneos, saldo consistente, idempotencia, consulta de referencia, alternativa sin cargo, depósito, cambio PIN, bloqueo global, token vencido, reemplazo tarjeta conservando consumo, comandos remotos, rollback por fallo simulado, expiración de sesión, heartbeat vencido, reconexión y auditoría.

La evidencia textual exacta se guarda en `Pruebas/Evidencias/ultima_ejecucion.txt` durante la verificación final. Una captura de pantalla demuestra la interfaz en este equipo; no demuestra por sí misma la red física.

La comprobación de rollback instala un trigger que provoca un fallo únicamente en la base temporal, después de iniciar un movimiento. La prueba exige saldo sin cambios y respuesta 96; después elimina el trigger. El respaldo de entrega no contiene ese trigger.

La verificación visual navega por servidor y cajero y guarda capturas en Documentacion/06_Manuales/Capturas. Se revisan acceso, token, menú, resumen y tablas. La aplicación conserva la paleta original y usa controles WPF redimensionables.

Pendientes que requieren a los integrantes: tres equipos físicos, red/firewall reales, los doce escenarios grabados, reparto definitivo de responsabilidades y enlace al video. Las pruebas en una sola PC no se anuncian como cumplimiento de esa demostración.

Verificación vigente NETGEAR del 31/10/2026

Compilación Release: cero errores y cero advertencias, SDK 9.0.311, destino .NET 8. 116 comprobaciones aprobadas: 83 SQL/TCP/UDP, 5 de interfaz del servidor y 28 del cajero. Evidencia exacta en Pruebas/Evidencias/integracion_20261031.txt, revision_servidor_20261031.txt y revision_cajero_20261031.txt. Integración: tres clientes estables más de treinta segundos, autenticación, dinero, concurrencia, rollback, autorización por usuario de la consulta pendiente e identidad/fechas compatibles con el nuevo cliente. Interbancario: UDP, IB_PING, alias de cuenta, banco no autorizado, idempotencia, contenido distinto, cupo concurrente, validación, rollback saldo/cupo, consulta de confirmación, salida confirmada y respuesta perdida. Cajero: referencia persistida antes del envío, bloqueo, reinicio/identidad, entrada numérica, formulario conservado ante error, actividad, deshabilitación/corte, cierre inmediato, respuesta tardía, configuración dañada/fuera de rango/sin escritura y cuatro tipos de PDF. Los PDF se revisaron con pypdf y dos vistas rasterizadas: una página, datos legibles y tarjeta enmascarada. La integración financiera se ejecuta en una base temporal eliminada al terminar. Los ensayos del cliente usan un par TCP simulado. No se mueven fondos operativos para esas pruebas. Faltan la validación física con otros grupos, decisión final del código y reservas DHCP en el router de la demostración. Los ensayos de loopback no acreditan estabilidad de esa LAN.

Demostración en equipos físicos y guion del video

Preparación: un servidor, tres o más PCs cliente, misma red, puerto autorizado y códigos distintos dados de alta. La excepción de varias instancias solo aplica si el grupo tiene menos integrantes, según el enunciado. Completar integrantes, carnés y responsables en GuiaDeContinuacion.md. Mostrar el servidor y el cajero a la vez cuando corresponda.

Nº	Acción	Evidencia esperada
1	Conectar tres PCs con ATM-01, ATM-02 y ATM-03	Dashboard: tres conexiones, IP real e inventario individual
2	Desconectar un cliente, esperar y volver a conectar	Desconectado en hasta 30 s, alerta, reconexión y bitácora
3	Usuario límite Q500: retirar Q300 en ATM-01, luego Q300 en ATM-02	Segundo retiro código 61; consumo global Q300
4	Dos retiros simultáneos sobre saldo suficiente para solo uno	Un aprobado; otro rechazado; saldo nunca negativo
5	Solicitar Q300 en tres billetes de Q100 cuando solo existen dos	Alternativa con otros billetes; aceptar o cancelar sin cargo previo
6	Depositar en ATM-02	Saldo e inventario de ATM-02 aumentan; dashboard <=5 s
7	Cambiar PIN en ATM-03	Registro de cambio; PIN anterior inválido en otro cajero
8	Tres PIN incorrectos en distintos cajeros	Bloqueo global; desbloqueo desde dashboard
9	Cambiar tarjeta después de un retiro del día	Anterior inactiva; nueva conserva consumo del límite
10	Cargar cajero sin inicializar o superar Q30000	Rechazo sin cambio de inventario
11	Generar token y esperar más de dos minutos antes de usarlo	Código 56; no se abre sesión
12	Poner cajero fuera de servicio desde el servidor	Pantalla bloqueada; no aprueba operaciones hasta habilitar

Adicionales recomendados: cerrar sesión a los 60 s, consulta de últimas cinco transacciones, restablecimiento de PIN temporal y cambio obligatorio, reportes Excel, arqueo sin diferencias, cambio de router/IP, código duplicado e ID financiero repetido sin doble cargo.

Guion

1. Portada: proyecto, fecha, nombres y carnés de todos; explicar responsabilidades. 2. Mostrar físicamente servidor, PCs cliente y conexión de red. 3. Mostrar arquitectura: clientes sin cadena SQL y servidor como único acceso a base. 4. Ejecutar los doce escenarios en orden. Preparar datos aislados para que el saldo y límites de un caso no invaliden otro. 5. Cada integrante explica su parte identificable: archivo C#, procedimiento SQL o pantalla, usando el índice de documentación. 6. Mostrar exportación, estructura documental y respaldo. Grabar imagen y audio legibles. 7. Completar y verificar el enlace al video antes de entregar.

No presentar conexiones locales automatizadas como prueba de varias PCs físicas. Registrar fecha, red utilizada, códigos, IP del servidor y resultado de cada escenario en una copia de esta tabla.

Integración NETGEAR

Implementación del MD recibido: descubrimiento UDP 5005 y solicitudes IB por TCP 5000. Leer ProtocoloExterno.md y PROTOCOLO_CONEXION_BANCOS_LLM.md. Nombre confirmado: NETGEAR. Código provisional: BANC02, confirmar con los grupos. En Bancos de la red configurar el propio, registrar al otro banco y autorizar su IP después de confirmarla con el responsable. Comprobar primero Ping y Verificar cuenta; después una transferencia pequeña y el mismo token en ambos historiales. No se modificó la interfaz del cajero: el envío de prueba se realiza en el dashboard con autorización administrativa y del usuario origen. Las extensiones de deduplicación/consulta de estado deben probarse en ambos servidores. El MD no garantiza atomicidad entre instituciones. La configuración de bancos reside en SQL Server. ConfiguracionEjemplo.json solo documenta valores de red; no contiene secretos. No intercambiar PIN, hashes, contraseñas SQL ni respaldos operativos. No se afirma haber probado aún con servidores físicos de otros grupos.

Ficha NETGEAR

Campo	Valor
Nombre	NETGEAR
Código	BANC02 provisional, confirmar unicidad
TCP	5000 por defecto, IPv4 Any, JSON UTF-8 por línea
UDP	5005, DISCOVER_BANCO_ATM / HERE_BANCO_ATM
IP	La interfaz conectada a la LAN de los otros grupos; pendiente red definitiva
Cupo	Q25,000.00 diario por banco
Cuenta	NumeroCuenta de 1-20 dígitos o tarjeta activa de 16 dígitos
Operaciones	IB_PING, IB_VERIFICAR_CUENTA, IB_TRANSFERIR
Extensión	IB_ESTADO_TRANSFERENCIA (acordar soporte)
Restricción de acceso	Código e IP autorizados; sin firma/TLS definidos por el MD
Responsable/fecha del ensayo	Completar entre grupos

Wi-Fi y Ethernet pueden tener IP distintas. La red uwu y las IP de BANC01 del MD son referencias del salón, no direcciones de este equipo.

Protocolo implementado - NETGEAR

Referencia: [MD recibido](PROTOCOLO_CONEXION_BANCOS_LLM.md), entregado el 31/10/2026. Se conserva intacto; esta página identifica implementación y extensiones.

Transporte

TCP IPv4 en IPAddress.Any, mismo listener que los cajeros, puerto configurable 5000 por defecto. Antes de registrar un ATM se inspecciona tipo sin distinguir mayúsculas. Una petición IB recibe una respuesta RESP_OK correlacionada por idMensaje y se cierra. Los cajeros conservan su conexión persistente. UTF-8, JSON por línea con LF, máximo 32 KiB; espera del cliente IB de diez segundos. UDP 5005 recibe DISCOVER_BANCO_ATM y responde HERE_BANCO_ATM al remitente, con importes invariantes e IP local de la ruta hacia ese remitente. Escanear usa broadcast limitado y por adaptador, y ping TCP a bancos registrados. No barre una /8 de Radmin. Si no hay broadcast, registrar IP y hacer ping. UDP no descubre otra subred automáticamente.

Operaciones

IB_PING devuelve identidad y cupo propio. IB_VERIFICAR_CUENTA acepta numeroCuenta o cuentaDestino; busca NumeroCuenta (único, 1-20 dígitos) o tarjeta activa de usuario activo y no bloqueado. IB_TRANSFERIR valida token TRF-YYYYMMDD-XXXXXX, monto positivo de hasta dos decimales, cuenta y banco destino. Solo confirma después del COMMIT SQL. BANC02 es provisional para NETGEAR; confirmar código único. BANC01 pertenece a NEXUSBANK según el MD.

Autorización

Los anuncios no conceden permiso. Confirmar IP y código con el otro grupo y registrar Confiable=1. IB_TRANSFERIR e IB_ESTADO_TRANSFERENCIA exigen bancoOrigen registrado, autorizado, activo y con IP coincidente con el TCP remitente. El MD no define firmas, secretos ni TLS. El control por IP mantiene compatibilidad con los campos acordados; no equivale a identidad criptográfica. Usar la red de pruebas con datos ficticios. No compartir PIN, hashes ni acceso SQL.

Datos, cupos e historial

08_Interbancario.sql añade BancoRed, TransferenciaBancaria, NumeroCuenta en Usuario y vw_TransferenciasUsuario. No reinicializa usuarios o inventarios. Un bloqueo de aplicación transaccional ordena operaciones; saldo, registro y ambos contadores bancarios locales cambian en una transacción. Fecha y reinicio diario proceden de SQL. Cada institución es autoridad de su saldo y cupo propio. El registro local del otro banco refleja operaciones conocidas y no puede conocer sus transferencias con terceros. El destino vuelve a comprobar su cupo al acreditar. La coordinación global de toda la red requiere un acuerdo adicional. Las transferencias tienen historial separado de retiros y depósitos físicos; no alteran billetes ni arqueo. El token une el movimiento del usuario y su registro bancario. Consultar Usuarios → Historial → Transferencias o la página Transferencias, exportable a Excel. El límite de retiro del usuario no se consume: el MD define cupos de banco para transferencias. Cualquier regla adicional debe acordarse. Se recomienda cuenta de longitud distinta a tarjeta para evitar ambigüedades futuras. La asignación rechaza cuentas que ya sean tarjetas de otro usuario.

Duplicados y respuesta perdida

Clave única: (bancoOrigen, tokenTransferencia). Repetir una entrada aprobada con iguales datos devuelve su confirmación sin duplicar saldo; otro contenido devuelve 94. El origen reserva saldo y cupos en SQL antes del envío. No mantiene una transacción SQL abierta esperando a la red. Confirmación con token y monto correctos: APROBADA. Rechazos definitivos 14/65/57/30: libera reserva. Timeout, corte, error 96 o confirmación no correlacionada: INCIERTA. Un reinicio puede dejar RESERVADA; también requiere conciliación. No devolver dinero ni generar otro token solo por un timeout. El MD no define transacción distribuida, reversos ni consulta de estado.

Extensión opcional IB_ESTADO_TRANSFERENCIA: idMensaje, bancoOrigen, bancoDestino y tokenTransferencia. Devuelve confirmación de entrada persistida o código 25 si no la encuentra. Consultar / conciliar confirma una salida solo si el par devuelve mismo token y monto. No encontrarla, no soportar la extensión o no responder no autoriza devolución. Conciliar con

el otro grupo. No se reenvía automáticamente a un par cuya idempotencia no esté comprobada. Entregar el token del comprobante al usuario y conservarlo en ambos bancos.

Códigos

00 confirmada; 14 cuenta/destino inexistente; 25 estado no encontrado (extensión); 30 formato/operación inválida; 51 saldo/usuario origen no disponible; 57 banco/IP no autorizado; 65 cupo insuficiente; 94 token en conflicto; 96 sin confirmación. RESP_OK también envuelve rechazos: leer codigo.

Pruebas

Base temporal y par TCP simulado: formato, alias de cuenta, banco no autorizado, tokens repetidos, contenido distinto, cupo concurrente, UDP, salida confirmada y respuesta perdida. Falta probar físicamente con NEXUSBANK y los demás grupos; no se acredita compatibilidad externa hasta hacerlo.

Lista para probar la integración futura

Registrar fecha, instituciones, versiones y responsables. Usar datos ficticios y guardar referencias en ambos servidores.

1. Comprobar IP/DNS y puerto sin realizar un cargo. 2. Verificar autenticación institucional correcta e incorrecta. 3. Consultar una cuenta válida e inexistente según el contrato acordado. 4. Solicitar una transferencia autorizada y comprobar los dos asientos y referencias. 5. Solicitar con saldo insuficiente y demostrar que no se acredita el destino. 6. Repetir exactamente el mismo ID y comprobar que no duplica el débito ni el crédito. 7. Reutilizar ID con otro monto y exigir rechazo. 8. Cortar la red después de enviar y antes de recibir respuesta. Consultar estado sin repetir el cargo. 9. Reiniciar un servidor con una operación pendiente y recuperar su estado persistido. 10. Conciliar las operaciones de ambos bancos por ID y referencia remota. 11. Cambiar de router/IP y demostrar que basta actualizar configuración. 12. Ejecutar dos operaciones concurrentes sobre el mismo saldo y validar límites.

Hay pruebas automáticas locales del protocolo y del par simulado (Resultados.md). Los escenarios de esta lista con instituciones reales siguen pendientes; no se anuncian como demostrados.

Entrega al jefe de grupo

Contenido y primer recorrido

1. Abrir LEEME.md y CajeroRed.sln en Visual Studio 2022. 2. Localizar la matriz y el índice de archivos para la defensa. 3. En otra PC servidor: instalar o restaurar la base, configurar la identidad SQL y la cadena local. 4. Iniciar Servidor, inicializar cajeros para el día y conectar al menos tres PCs con códigos diferentes. 5. Ejecutar/verificar los doce escenarios y completar el video grupal.

La fase I permanece intacta en Proyecto_Cajero. Esta solución nueva usa C#/WPF y SQL Server. Se simplificó el registro a los datos de D-10, manteniendo el estilo azul de las pantallas anteriores. Los módulos de remesas, pagos, adelantos, NFC y ESP32 de Python no se presentan como migrados en esta entrega; consultar el enunciado original de fase I si se solicitan extras.

Subir a GitHub

No se ha creado repositorio remoto ni se ha publicado nada. Subir la carpeta de esta solución como proyecto independiente, sin mezclar la fase I ni su Fase_II ignorada. La solución no depende de rutas al proyecto Python.

Incluir código, solución, scripts SQL, ejemplos de configuración, manuales, diagramas, PDF y pruebas. El .gitignore excluye bin/obj, cachés, configuraciones privadas y ejecutables publicados. Adjuntar el .bak y paquetes portables por el mecanismo de entrega o publicaciones del repositorio; no versionar carpetas de dependencias ni contraseñas.

Antes de publicar, revisar los archivos que se van a agregar. Los usuarios y credenciales del seed son ficticios de demostración. No copiar datos, secretos de hardware ni configuraciones de la fase I. El enlace al video y los datos reales de integrantes se completan por el jefe.

Integración futura con otro grupo

Compartir solo Documentacion/08_IntegracionEntreGrupos. Completar la ficha, acordar campos y versiones y ensayar con datos ficticios. La implementación vigente es Codigo/Aplicacion/BancoInterbancario.cs; Bancos de la red permite verificar y transferir con bancos autorizados. Leer el contrato vigente antes del ensayo físico.

Las transferencias futuras deben ejecutarse servidor a servidor, con autenticación de instituciones, identificación de cuentas, idempotencia persistida, consulta de estado y conciliación. No agregar una cadena SQL remota a los cajeros. No interpretar un timeout como rechazo ni revertir dinero sin confirmar el estado remoto.

Completar antes de entregar

- Fecha de entrega: por confirmar con el catedrático. - Integrantes, carnés y responsables: completar con datos reales. - PC servidor, red utilizada y códigos de cada integrante: completar. - Resultados y evidencia de los doce escenarios físicos: completar. - Enlace del video: pendiente de grabación. - Banco/identificador del otro grupo y operaciones acordadas: pendiente de coordinación posterior.

Distribuir responsabilidades reales y explicables. Esta guía no inventa quién desarrolló cada módulo ni sustituye la explicación individual exigida.

Anexo - scripts SQL completos

Aplicar en orden mediante Instalar-BaseDeDatos.ps1. Los scripts no eliminan bases existentes; la instalación se detiene si el esquema ya está creado. Los procedimientos pueden actualizarse por separado después de revisar su versión.

01_CrearBase.sql

```
-- Ejecutar con una cuenta autorizada para crear bases. Nunca elimina una base existente.

USE master;

GO

IF DB_ID(N'BD_CajeroRed_Fase2') IS NULL CREATE DATABASE BD_CajeroRed_Fase2;

GO

USE BD_CajeroRed_Fase2;

GO

SET QUOTED_IDENTIFIER ON;

SET XACT_ABORT ON;

GO

IF OBJECT_ID('dbo.Usuario') IS NOT NULL THROW 50001, 'El esquema ya existe. No vuelva a aplicar los scripts
sobre datos existentes.', 1;

GO

BEGIN TRAN;

CREATE TABLE dbo.Usuario(

 Id int IDENTITY PRIMARY KEY, Nombre nvarchar(120) NOT NULL,

 Saldo decimal(18,2) NOT NULL CHECK(Saldo&gt;=0), LimiteDiario decimal(18,2) NOT NULL CHECK(LimiteDiario&gt;=0),

 Activo bit NOT NULL DEFAULT 1, Bloqueado bit NOT NULL DEFAULT 0,

 Intentos int NOT NULL DEFAULT 0 CHECK(Intentos BETWEEN 0 AND 3), Creado datetime2 NOT NULL DEFAULT
SYSDATETIME());

CREATE TABLE dbo.Tarjeta(

 Id int IDENTITY PRIMARY KEY, UsuarioId int NOT NULL REFERENCES dbo.Usuario(Id),

 Numero varchar(16) NOT NULL UNIQUE CHECK(LEN(Numero)=16 AND Numero NOT LIKE '%[^0-9]%'),

 PinHash varbinary(32) NOT NULL, Salt varchar(64) NOT NULL,

 Activa bit NOT NULL DEFAULT 1, PinTemporal bit NOT NULL DEFAULT 0,

 Asignada datetime2 NOT NULL DEFAULT SYSDATETIME(), Baja datetime2 NULL);

CREATE UNIQUE INDEX UX_TarjetaActiva ON dbo.Tarjeta(UsuarioId) WHERE Activa=1;

CREATE TABLE dbo.Administrador(

 Id int IDENTITY PRIMARY KEY, Usuario nvarchar(60) NOT NULL UNIQUE,

 Hash varbinary(32) NOT NULL, Salt varbinary(32) NOT NULL, Iteraciones int NOT NULL DEFAULT 210000,

 Rol varchar(20) NOT NULL DEFAULT 'Administrador' CHECK(Rol IN('Administrador','Supervisor')), Activo bit NOT
NULL DEFAULT 1);

CREATE TABLE dbo.Cajero(

 Codigo varchar(24) PRIMARY KEY, Nombre nvarchar(100) NOT NULL, Ubicacion nvarchar(120) NOT NULL DEFAULT '',

 IP varchar(45) NULL, Equipo nvarchar(100) NULL, Version nvarchar(40) NULL,

 Activo bit NOT NULL DEFAULT 1, Conectado bit NOT NULL DEFAULT 0, Habilitado bit NOT NULL DEFAULT 1,

 FechaInicializacion date NULL, UltimoHeartbeat datetime2 NULL, UltimaConexion datetime2 NULL);

CREATE TABLE dbo.Denominacion(Valor int PRIMARY KEY CHECK(Valor IN(200,100,50,20,10,5,1)));

INSERT dbo.Denominacion VALUES(200),(100),(50),(20),(10),(5),(1);

CREATE TABLE dbo.InventarioCajero(

 Cajero varchar(24) NOT NULL REFERENCES dbo.Cajero(Codigo), Denominacion int NOT NULL REFERENCES
dbo.Denominacion(Valor),
```

```
 Cantidad int NOT NULL DEFAULT 0 CHECK(Cantidad BETWEEN 0 AND 30000), PRIMARY KEY(Cajero,Denominacion));

CREATE TABLE dbo.CargaEfectivo(

 Id bigint IDENTITY PRIMARY KEY, Cajero varchar(24) NOT NULL REFERENCES dbo.Cajero(Codigo),

 Tipo varchar(20) NOT NULL CHECK(Tipo IN('INICIALIZACION','CARGA')), Total decimal(18,2) NOT NULL
CHECK(Total&gt;0),

 EfectivoAnterior decimal(18,2) NOT NULL, AdministradorId int NOT NULL REFERENCES dbo.Administrador(Id), Fecha
 datetime2 NOT NULL DEFAULT SYSDATETIME());

CREATE TABLE dbo.CargaEfectivoDetalle(

 CargaId bigint NOT NULL REFERENCES dbo.CargaEfectivo(Id), Denominacion int NOT NULL REFERENCES
dbo.Denominacion(Valor),

 Cantidad int NOT NULL CHECK(Cantidad&gt;=0), PRIMARY KEY(CargaId,Denominacion));

CREATE TABLE dbo.Sesion(

 Id varchar(36) PRIMARY KEY DEFAULT CONVERT(varchar(36),NEWID()), UsuarioId int NULL REFERENCES
dbo.Usuario(Id),

 TarjetaId int NULL REFERENCES dbo.Tarjeta(Id), Cajero varchar(24) NOT NULL REFERENCES dbo.Cajero(Codigo),

 Inicio datetime2 NOT NULL DEFAULT SYSDATETIME(), UltimaActividad datetime2 NOT NULL DEFAULT SYSDATETIME(),

 Fin datetime2 NULL, Resultado varchar(32) NOT NULL);

CREATE TABLE dbo.Token(

 Id bigint IDENTITY PRIMARY KEY, UsuarioId int NOT NULL REFERENCES dbo.Usuario(Id), TarjetaId int NOT NULL
REFERENCES dbo.Tarjeta(Id),

 Cajero varchar(24) NOT NULL REFERENCES dbo.Cajero(Codigo), CodigoHash varbinary(32) NOT NULL,

 Expira datetime2 NOT NULL, Usado bit NOT NULL DEFAULT 0, Creado datetime2 NOT NULL DEFAULT SYSDATETIME());

CREATE TABLE dbo.Transaccion(

 Id bigint IDENTITY PRIMARY KEY, UsuarioId int NULL REFERENCES dbo.Usuario(Id), TarjetaId int NULL REFERENCES
dbo.Tarjeta(Id),

 Cajero varchar(24) NOT NULL REFERENCES dbo.Cajero(Codigo), Solicitud varchar(64) NOT NULL,

 Tipo varchar(20) NOT NULL CHECK(Tipo IN('RETIRO','DEPOSITO')), Monto decimal(18,2) NOT NULL CHECK(Monto&gt;=0),

 CodigoRespuesta varchar(2) NOT NULL, Motivo nvarchar(240) NOT NULL, Fecha datetime2 NOT NULL DEFAULT
SYSDATETIME(),

 UNIQUE(Cajero,Solicitud));

CREATE INDEX IX_TransaccionUsuarioFecha ON dbo.Transaccion(UsuarioId,Fecha)
INCLUDE(Tipo,Monto,CodigoRespuesta);

CREATE TABLE dbo.TransaccionDetalle(

 TransaccionId bigint NOT NULL REFERENCES dbo.Transaccion(Id), Denominacion int NOT NULL REFERENCES
dbo.Denominacion(Valor),

 Cantidad int NOT NULL CHECK(Cantidad&gt;=0), PRIMARY KEY(TransaccionId,Denominacion));

CREATE TABLE dbo.CambioPin(

 Id bigint IDENTITY PRIMARY KEY, UsuarioId int NOT NULL REFERENCES dbo.Usuario(Id), Cajero varchar(24) NULL
REFERENCES dbo.Cajero(Codigo),

 Fecha datetime2 NOT NULL DEFAULT SYSDATETIME(), Origen varchar(20) NOT NULL, Resultado nvarchar(120) NOT
NULL);

CREATE TABLE dbo.HistorialLimite(

 Id bigint IDENTITY PRIMARY KEY, UsuarioId int NOT NULL REFERENCES dbo.Usuario(Id), Anterior decimal(18,2) NOT
 NULL,

 Nuevo decimal(18,2) NOT NULL, AdministradorId int NOT NULL REFERENCES dbo.Administrador(Id), Fecha datetime2
NOT NULL DEFAULT SYSDATETIME());

CREATE TABLE dbo.Alerta(

 Id bigint IDENTITY PRIMARY KEY, Tipo varchar(40) NOT NULL, Cajero varchar(24) NULL REFERENCES
dbo.Cajero(Codigo),

 Descripcion nvarchar(250) NOT NULL, Fecha datetime2 NOT NULL DEFAULT SYSDATETIME(), Atendida bit NOT NULL
DEFAULT 0,

 AdministradorId int NULL REFERENCES dbo.Administrador(Id), FechaAtencion datetime2 NULL);

CREATE TABLE dbo.Bitacora(

 Id bigint IDENTITY PRIMARY KEY, Origen nvarchar(60) NOT NULL, Accion nvarchar(80) NOT NULL,
```

```
 Detalle nvarchar(500) NOT NULL, Fecha datetime2 NOT NULL DEFAULT SYSDATETIME());

CREATE TABLE dbo.Parametro(Clave varchar(40) PRIMARY KEY, Valor decimal(18,2) NOT NULL CHECK(Valor&gt;0));

INSERT dbo.Parametro VALUES('Puerto',5000),('UmbralEfectivo',1000),('VigenciaToken',120),('MaxInicializacion',
10000),('MaxEfectivo',30000);

CREATE TABLE dbo.SolicitudOperacion(

 Cajero varchar(24) NOT NULL REFERENCES dbo.Cajero(Codigo), Solicitud varchar(64) NOT NULL, UsuarioId int NOT
NULL REFERENCES dbo.Usuario(Id),

 Tipo varchar(40) NOT NULL, Huella varbinary(32) NOT NULL, Codigo varchar(2) NOT NULL,

 Mensaje nvarchar(240) NOT NULL, Datos nvarchar(max) NOT NULL CHECK(ISJSON(Datos)=1), Fecha datetime2 NOT NULL
 DEFAULT SYSDATETIME(),

 PRIMARY KEY(Cajero,Solicitud));

COMMIT;

GO
```

02_Operaciones.sql

```
USE BD_CajeroRed_Fase2;

GO

CREATE OR ALTER PROCEDURE dbo.sp_Alertas @Cajero varchar(24)

AS

BEGIN

 SET NOCOUNT ON;

 DECLARE @total decimal(18,2)=(SELECT ISNULL(SUM(Denominacion*Cantidad),0) FROM dbo.InventarioCajero WHERE
Cajero=@Cajero);

 IF @total&lt;(SELECT Valor FROM dbo.Parametro WHERE Clave='UmbralEfectivo')

 AND NOT EXISTS(SELECT 1 FROM dbo.Alerta WHERE Cajero=@Cajero AND Tipo='EFECTIVO_BAJO' AND Atendida=0)

 INSERT dbo.Alerta(Tipo,Cajero,Descripcion) VALUES('EFECTIVO_BAJO',@Cajero,N'Efectivo por debajo del
umbral.');

 IF EXISTS(SELECT 1 FROM dbo.InventarioCajero WHERE Cajero=@Cajero AND Cantidad=0)

 AND NOT EXISTS(SELECT 1 FROM dbo.Alerta WHERE Cajero=@Cajero AND Tipo='DENOMINACION_AGOTADA' AND Atendida=0)

 INSERT dbo.Alerta(Tipo,Cajero,Descripcion) VALUES('DENOMINACION_AGOTADA',@Cajero,N'Hay denominaciones sin
billetes.');

END;

GO

CREATE OR ALTER PROCEDURE dbo.sp_Desconectar @Cajero varchar(24)

AS

BEGIN

 SET NOCOUNT ON; SET XACT_ABORT ON;

 BEGIN TRAN;

 UPDATE dbo.Cajero SET Conectado=0 WHERE Codigo=@Cajero AND Conectado=1;

 IF @@ROWCOUNT&gt;0

 BEGIN

  INSERT dbo.Bitacora(Origen,Accion,Detalle) VALUES(@Cajero,'DESCONEXION',N'Conexión finalizada o heartbeat
vencido.');

  INSERT dbo.Alerta(Tipo,Cajero,Descripcion) VALUES('DESCONECTADO',@Cajero,N'Cajero desconectado.');

  UPDATE dbo.Sesion SET Fin=SYSDATETIME() WHERE Cajero=@Cajero AND Fin IS NULL;

 END;

 COMMIT;

END;

GO

CREATE OR ALTER PROCEDURE dbo.sp_GenerarToken

 @Usuario int, @Tarjeta int, @Cajero varchar(24), @Token varchar(6) OUTPUT

AS

BEGIN

 SET NOCOUNT ON;

 DECLARE @v int=(SELECT Valor FROM dbo.Parametro WHERE Clave='VigenciaToken');

 -- CRYPT_GEN_RANDOM evita Random() predecible; el token nunca se persiste en claro.

 SET @Token=RIGHT('000000'+CONVERT(varchar(6),CONVERT(bigint,CRYPT_GEN_RANDOM(4))%1000000),6);

 UPDATE dbo.Token SET Usado=1 WHERE UsuarioId=@Usuario AND Cajero=@Cajero AND Usado=0;

 INSERT dbo.Token(UsuarioId,TarjetaId,Cajero,CodigoHash,Expira)

 VALUES(@Usuario,@Tarjeta,@Cajero,HASHBYTES('SHA2_256',@Token),DATEADD(second,@v,SYSDATETIME()));

END;

GO
```

```
-- Motor transaccional compartido. Los procedimientos públicos abajo conservan contratos explícitos.

CREATE OR ALTER PROCEDURE dbo.sp_Operacion

 @Tipo varchar(40), @Json nvarchar(max), @Cajero varchar(24), @Sesion varchar(36)=NULL, @Solicitud varchar(64)

AS

BEGIN

 SET NOCOUNT ON; SET XACT_ABORT ON;

 DECLARE @codigo varchar(2)='00',@mensaje nvarchar(240)=N'Operación aprobada',@datos nvarchar(max)='{}',

 @u int,@t int,@activo bit,@bloq bit,@intentos int,@saldo decimal(18,2),@limite decimal(18,2),@retirado
decimal(18,2),

 @hash varbinary(32),@salt varchar(64),@temporal bit,@hoy date=CAST(SYSDATETIME() AS date),@fecha
datetime2=SYSDATETIME(),

 @habilitado bit,@inicializado date,@cactivo bit,@monto decimal(18,2),@efectivo decimal(18,2),@id
bigint,@token varchar(6),

 @ses varchar(36),@tokenId bigint,@huella varbinary(32)=HASHBYTES('SHA2_256',@Tipo+':'+@Json),@replay bit=0;

 DECLARE @billetes TABLE(Denominacion int PRIMARY KEY,Cantidad int NOT NULL);

 BEGIN TRY

  IF ISJSON(@Json)&lt;&gt;1 THROW 50002,'JSON inválido.',1;

  BEGIN TRAN;

  SELECT @cactivo=Activo,@habilitado=Habilitado,@inicializado=FechaInicializacion

  FROM dbo.Cajero WITH(UPDLOCK,ROWLOCK) WHERE Codigo=@Cajero;

  IF @cactivo IS NULL OR @cactivo=0 BEGIN SET @codigo='91';SET @mensaje=N'Cajero no dado de alta o
inactivo.';GOTO Terminar;END;

  IF @Tipo='REGISTRAR_CAJERO'

  BEGIN

   UPDATE dbo.Cajero SET
IP=JSON_VALUE(@Json,'$.ip'),Equipo=JSON_VALUE(@Json,'$.equipo'),Version=JSON_VALUE(@Json,'$.version'),

   Conectado=1,UltimoHeartbeat=@fecha,UltimaConexion=@fecha WHERE Codigo=@Cajero;

   INSERT dbo.Bitacora(Origen,Accion,Detalle) VALUES(@Cajero,'CONEXION',N'Cajero registrado o reconectado.');

   SET @datos=(SELECT @habilitado habilitado,CASE WHEN @inicializado=@hoy THEN CAST(1 AS bit) ELSE CAST(0 AS
bit) END inicializado FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

   GOTO Terminar;

  END;

  IF @Tipo='HEARTBEAT'

  BEGIN

   UPDATE dbo.Cajero SET UltimoHeartbeat=@fecha WHERE Codigo=@Cajero;

   SET @datos=(SELECT @habilitado habilitado,CASE WHEN @inicializado=@hoy THEN CAST(1 AS bit) ELSE CAST(0 AS
bit) END inicializado FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

   GOTO Terminar;

  END;

  IF @Tipo='LOGOUT'

  BEGIN UPDATE dbo.Sesion SET Fin=@fecha WHERE Id=@Sesion AND Cajero=@Cajero AND Fin IS NULL;GOTO
Terminar;END;

  IF @habilitado=0 OR @inicializado IS NULL OR @inicializado&lt;&gt;@hoy

  BEGIN SET @codigo='91';SET @mensaje=N'Cajero fuera de servicio o sin inicializar hoy.';GOTO Terminar;END;

  IF @Tipo IN('LOGIN','VALIDAR_TOKEN')

  BEGIN

   SELECT @t=Id,@u=UsuarioId,@hash=PinHash,@salt=Salt,@temporal=PinTemporal FROM dbo.Tarjeta
WITH(UPDLOCK,ROWLOCK)

   WHERE Numero=JSON_VALUE(@Json,'$.tarjeta');

  END

  ELSE
```

```
  BEGIN

   SELECT @u=UsuarioId,@t=TarjetaId FROM dbo.Sesion WITH(UPDLOCK,ROWLOCK)

   WHERE Id=@Sesion AND Cajero=@Cajero AND Fin IS NULL AND UltimaActividad&gt;=DATEADD(second,-120,@fecha) AND
Resultado='00';

   IF @u IS NOT NULL

   SELECT @temporal=PinTemporal,@hash=PinHash,@salt=Salt FROM dbo.Tarjeta WHERE Id=@t AND Activa=1;

   ELSE BEGIN SET @codigo='57';SET @mensaje=N'Sesión vencida. Inicie sesión nuevamente.';GOTO Terminar;END;

  END;

  SELECT @activo=Activo,@bloq=Bloqueado,@intentos=Intentos,@saldo=Saldo,@limite=LimiteDiario FROM dbo.Usuario
WITH(UPDLOCK,ROWLOCK) WHERE Id=@u;

  IF @u IS NULL BEGIN SET @codigo='55';SET @mensaje=N'Tarjeta o credenciales incorrectas.';GOTO
AccesoFallido;END;

  IF @activo=0 OR @bloq=1 OR NOT EXISTS(SELECT 1 FROM dbo.Tarjeta WHERE Id=@t AND Activa=1)

  BEGIN SET @codigo='75';SET @mensaje=N'Tarjeta bloqueada o inactiva.';GOTO Terminar;END;

  IF @Tipo='LOGIN'

  BEGIN

   IF LEN(ISNULL(JSON_VALUE(@Json,'$.pin'),''))&lt;&gt;4 OR JSON_VALUE(@Json,'$.pin') LIKE '%[^0-9]%'

    OR HASHBYTES('SHA2_256',CONVERT(varchar(200),@salt+JSON_VALUE(@Json,'$.pin')))&lt;&gt;@hash

   BEGIN SET @codigo='55';SET @mensaje=N'PIN incorrecto.';GOTO AccesoFallido;END;

   EXEC dbo.sp_GenerarToken @u,@t,@Cajero,@token OUTPUT;

   SET @datos=(SELECT @token tokenServidor FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

   GOTO Terminar;

  END;

  IF @Tipo='VALIDAR_TOKEN'

  BEGIN

   SELECT TOP(1) @tokenId=Id FROM dbo.Token WITH(UPDLOCK,ROWLOCK) WHERE UsuarioId=@u AND TarjetaId=@t AND
Cajero=@Cajero

   AND Usado=0 AND Expira&gt;@fecha AND
CodigoHash=HASHBYTES('SHA2_256',CONVERT(varchar(6),JSON_VALUE(@Json,'$.token'))) ORDER BY Id DESC;

   IF @tokenId IS NULL OR LEN(ISNULL(JSON_VALUE(@Json,'$.token'),''))&lt;&gt;6

   BEGIN SET @codigo='56';SET @mensaje=N'Token inválido, usado o expirado.';GOTO AccesoFallido;END;

   UPDATE dbo.Token SET Usado=1 WHERE Id=@tokenId;

   UPDATE dbo.Usuario SET Intentos=0 WHERE Id=@u;

   UPDATE dbo.Sesion SET Fin=@fecha WHERE UsuarioId=@u AND Cajero=@Cajero AND Fin IS NULL;

   SET @ses=CONVERT(varchar(36),NEWID());

   INSERT dbo.Sesion(Id,UsuarioId,TarjetaId,Cajero,Resultado) VALUES(@ses,@u,@t,@Cajero,'00');

   INSERT dbo.Bitacora(Origen,Accion,Detalle) VALUES(@Cajero,'ACCESO',CONCAT('Usuario ',@u));

   SET @datos=(SELECT @ses sesion,@u usuarioId,@temporal debeCambiarPin,(SELECT Nombre FROM dbo.Usuario WHERE
Id=@u) nombre FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

   GOTO Terminar;

  END;

  UPDATE dbo.Sesion SET UltimaActividad=@fecha WHERE Id=@Sesion;

  IF @Tipo='ESTADO_SOLICITUD'

  BEGIN

   SELECT @datos=(SELECT Solicitud idOriginal,UsuarioId usuarioId,Codigo codigo,Mensaje
mensaje,JSON_QUERY(Datos) datos FROM dbo.SolicitudOperacion

    WHERE Cajero=@Cajero AND UsuarioId=@u AND Solicitud=JSON_VALUE(@Json,'$.idOriginal') FOR JSON
PATH,WITHOUT_ARRAY_WRAPPER);

   IF NULLIF(@datos,'') IS NULL BEGIN SET @codigo='25';SET @mensaje=N'No existe resultado confirmado. Consulte
 al administrador antes de repetir.';SET @datos='{}';END;

   GOTO Terminar;
```

```
  END;

  IF @Tipo IN('RETIRO','DEPOSITO','CAMBIO_PIN') AND EXISTS(SELECT 1 FROM dbo.SolicitudOperacion WHERE
Cajero=@Cajero AND Solicitud=@Solicitud)

  BEGIN

   IF NOT EXISTS(SELECT 1 FROM dbo.SolicitudOperacion WHERE Cajero=@Cajero AND Solicitud=@Solicitud AND
UsuarioId=@u AND Tipo=@Tipo AND Huella=@huella)

   BEGIN SET @codigo='94';SET @mensaje=N'Identificador reutilizado con datos distintos.';GOTO Terminar;END;

   SELECT @codigo=Codigo,@mensaje=Mensaje,@datos=Datos FROM dbo.SolicitudOperacion WHERE Cajero=@Cajero AND
Solicitud=@Solicitud;

   SET @replay=1;GOTO Terminar;

  END;

  IF @temporal=1 AND @Tipo NOT IN('CAMBIO_PIN')

  BEGIN SET @codigo='58';SET @mensaje=N'Debe cambiar su PIN temporal antes de operar.';GOTO Terminar;END;

  SELECT @retirado=ISNULL(SUM(Monto),0) FROM dbo.Transaccion WHERE UsuarioId=@u AND Tipo='RETIRO' AND
CodigoRespuesta='00' AND Fecha&gt;=@hoy AND Fecha&lt;DATEADD(day,1,@hoy);

  IF @Tipo='CONSULTA_SALDO'

  BEGIN SET @datos=(SELECT @fecha fecha,@saldo saldo,@limite limiteDiario,CASE WHEN @limite&gt;@retirado THEN
@limite-@retirado ELSE 0 END disponibleHoy,@retirado retiradoHoy FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);GOTO
Terminar;END;

  IF @Tipo='ULTIMAS_TRANSACCIONES'

  BEGIN SET @datos=(SELECT TOP(5) Fecha fecha,Tipo tipo,Monto monto,Cajero cajero FROM dbo.Transaccion WHERE
UsuarioId=@u AND CodigoRespuesta='00' ORDER BY Fecha DESC,Id DESC FOR JSON PATH);GOTO Terminar;END;

  IF @Tipo='CAMBIO_PIN'

  BEGIN

   DECLARE @actual varchar(50)=JSON_VALUE(@Json,'$.pinActual'),@nuevo
varchar(50)=JSON_VALUE(@Json,'$.pinNuevo');

   IF LEN(ISNULL(@actual,''))&lt;&gt;4 OR HASHBYTES('SHA2_256',CONVERT(varchar(200),@salt+@actual))&lt;&gt;@hash

   BEGIN SET @codigo='55';SET @mensaje=N'PIN actual incorrecto.';

    INSERT dbo.CambioPin(UsuarioId,Cajero,Origen,Resultado) VALUES(@u,@Cajero,'USUARIO',@mensaje);GOTO
AccesoFallido;END;

   IF LEN(ISNULL(@nuevo,''))&lt;&gt;4 OR @nuevo LIKE '%[^0-9]%' OR @actual=@nuevo

   BEGIN SET @codigo='30';SET @mensaje=N'El nuevo PIN debe tener 4 dígitos y ser distinto al actual.';GOTO
Terminar;END;

   SET @salt=CONVERT(varchar(64),CRYPT_GEN_RANDOM(32),2);

   UPDATE dbo.Tarjeta SET
Salt=@salt,PinHash=HASHBYTES('SHA2_256',CONVERT(varchar(200),@salt+@nuevo)),PinTemporal=0 WHERE Id=@t;

   INSERT dbo.CambioPin(UsuarioId,Cajero,Origen,Resultado) VALUES(@u,@Cajero,'USUARIO','00');

   UPDATE dbo.Token SET Usado=1 WHERE UsuarioId=@u;

   UPDATE dbo.Sesion SET Fin=@fecha WHERE UsuarioId=@u AND Id&lt;&gt;@Sesion AND Fin IS NULL;

   SET @datos=(SELECT @fecha fecha FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

   SET @mensaje=N'PIN actualizado en toda la red.';GOTO GuardarResultado;

  END;

  IF @Tipo NOT IN('RETIRO','DEPOSITO') BEGIN SET @codigo='30';SET @mensaje=N'Operación desconocida.';GOTO
Terminar;END;

  SET @monto=TRY_CONVERT(decimal(18,2),JSON_VALUE(@Json,'$.monto'));

  IF @monto IS NULL OR @monto&lt;=0 OR @monto&gt;30000 OR @monto&lt;&gt;FLOOR(@monto)

  BEGIN SET @codigo='30';SET @mensaje=N'Monto inválido; use quetzales enteros entre 1 y 30000.';GOTO
Terminar;END;

  IF JSON_QUERY(@Json,'$.desglose') IS NULL BEGIN SET @codigo='30';SET @mensaje=N'Falta desglose.';GOTO
Terminar;END;

  IF EXISTS(SELECT 1 FROM OPENJSON(@Json,'$.desglose') WHERE TRY_CONVERT(int,[key]) NOT
IN(200,100,50,20,10,5,1)

   OR TRY_CONVERT(int,[key]) IS NULL OR TRY_CONVERT(int,value) IS NULL OR TRY_CONVERT(int,value)&lt;0 OR
TRY_CONVERT(int,value)&gt;30000 OR type&lt;&gt;2)
```

```
   OR EXISTS(SELECT [key] FROM OPENJSON(@Json,'$.desglose') GROUP BY [key] HAVING COUNT(*)&gt;1)

  BEGIN SET @codigo='30';SET @mensaje=N'Denominaciones o cantidades inválidas.';GOTO Terminar;END;

  INSERT @billetes SELECT CONVERT(int,[key]),CONVERT(int,value) FROM OPENJSON(@Json,'$.desglose');

  IF ISNULL((SELECT SUM(CONVERT(bigint,Denominacion)*Cantidad) FROM @billetes),0)&lt;&gt;@monto

  BEGIN SET @codigo='30';SET @mensaje=N'El desglose no coincide con el monto.';GOTO Terminar;END;

  SELECT @efectivo=SUM(CONVERT(bigint,Denominacion)*Cantidad) FROM dbo.InventarioCajero WITH(UPDLOCK,ROWLOCK)
WHERE Cajero=@Cajero;

  IF @Tipo='RETIRO'

  BEGIN

   IF @saldo&lt;@monto BEGIN SET @codigo='51';SET @mensaje=N'Saldo insuficiente.';GOTO RegistrarMovimiento;END;

   IF @retirado+@monto&gt;@limite BEGIN SET @codigo='61';SET @mensaje=N'Excede el límite diario global
disponible.';GOTO RegistrarMovimiento;END;

   IF EXISTS(SELECT 1 FROM @billetes b LEFT JOIN dbo.InventarioCajero i ON i.Cajero=@Cajero AND
i.Denominacion=b.Denominacion WHERE b.Cantidad&gt;ISNULL(i.Cantidad,0))

   BEGIN

    SET @codigo='92';SET @mensaje=N'No hay billetes para ese desglose.';

    SET @datos=(SELECT JSON_QUERY((SELECT Denominacion denominacion,Cantidad cantidad FROM
dbo.InventarioCajero WHERE Cajero=@Cajero FOR JSON PATH)) inventario FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

    INSERT dbo.Alerta(Tipo,Cajero,Descripcion) VALUES('RETIRO_SIN_BILLETES',@Cajero,N'Retiro rechazado por
falta de billetes.');

    GOTO RegistrarMovimiento;

   END;

  END

  ELSE IF @efectivo+@monto&gt;(SELECT Valor FROM dbo.Parametro WHERE Clave='MaxEfectivo')

   BEGIN SET @codigo='93';SET @mensaje=N'El depósito supera la capacidad del cajero.';GOTO
RegistrarMovimiento;END;

  UPDATE dbo.Usuario SET Saldo=Saldo+CASE WHEN @Tipo='RETIRO' THEN -@monto ELSE @monto END WHERE Id=@u;

  UPDATE i SET Cantidad=i.Cantidad+CASE WHEN @Tipo='RETIRO' THEN -b.Cantidad ELSE b.Cantidad END

  FROM dbo.InventarioCajero i JOIN @billetes b ON i.Denominacion=b.Denominacion WHERE i.Cajero=@Cajero;

  SET @saldo=@saldo+CASE WHEN @Tipo='RETIRO' THEN -@monto ELSE @monto END;

  SET @retirado=@retirado+CASE WHEN @Tipo='RETIRO' THEN @monto ELSE 0 END;

  RegistrarMovimiento:

  INSERT dbo.Transaccion(UsuarioId,TarjetaId,Cajero,Solicitud,Tipo,Monto,CodigoRespuesta,Motivo)
VALUES(@u,@t,@Cajero,@Solicitud,@Tipo,@monto,@codigo,@mensaje);

  SET @id=SCOPE_IDENTITY();

  INSERT dbo.TransaccionDetalle SELECT @id,Denominacion,Cantidad FROM @billetes;

  IF @codigo='00'

  SET @datos=(SELECT @id idTransaccion,@saldo saldo,CASE WHEN @limite&gt;@retirado THEN @limite-@retirado ELSE 0
END disponibleHoy,@fecha fecha,@monto monto,@Tipo tipo,@Cajero cajero FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

  EXEC dbo.sp_Alertas @Cajero;

  GOTO GuardarResultado;

  AccesoFallido:

  INSERT dbo.Sesion(UsuarioId,TarjetaId,Cajero,Fin,Resultado) VALUES(@u,@t,@Cajero,@fecha,@codigo);

  IF @u IS NOT NULL

  BEGIN

   UPDATE dbo.Usuario SET Intentos=CASE WHEN Intentos&lt;3 THEN Intentos+1 ELSE 3 END,Bloqueado=CASE WHEN
Intentos&gt;=2 THEN 1 ELSE Bloqueado END WHERE Id=@u;

   IF @intentos&gt;=2

   BEGIN

    SET @codigo='75';SET @mensaje=N'Tarjeta bloqueada por tres intentos fallidos.';

    INSERT dbo.Alerta(Tipo,Cajero,Descripcion) VALUES('TARJETA_BLOQUEADA',@Cajero,CONCAT('Usuario ',@u,'
```

```
bloqueado.'));

    UPDATE dbo.Sesion SET Fin=@fecha WHERE UsuarioId=@u AND Fin IS NULL;

   END;

  END;

  GOTO Terminar;

  GuardarResultado:

  INSERT dbo.SolicitudOperacion(Cajero,Solicitud,UsuarioId,Tipo,Huella,Codigo,Mensaje,Datos)
VALUES(@Cajero,@Solicitud,@u,@Tipo,@huella,@codigo,@mensaje,@datos);

  Terminar:

  IF @codigo='00' AND @datos='{}' SET @datos=(SELECT @fecha fecha FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

  IF @Tipo NOT IN('HEARTBEAT') AND @replay=0

   INSERT dbo.Bitacora(Origen,Accion,Detalle) VALUES(@Cajero,@Tipo,CONCAT('Usuario ',@u,'; resultado
',@codigo,'; solicitud ',@Solicitud));

  COMMIT;

  SELECT @codigo Codigo,@mensaje Mensaje,ISNULL(@datos,'{}') Datos;

 END TRY

 BEGIN CATCH

  IF XACT_STATE()&lt;&gt;0 ROLLBACK;

  -- El servidor no debe anunciar éxito si SQL no confirmó. Sin datos sensibles en errores.

  SELECT '96' Codigo,N'Error de base de datos; consulte el estado de la solicitud antes de repetir.'
Mensaje,'{}' Datos;

 END CATCH;

END;

GO
```

03_Contratos.sql

```
USE BD_CajeroRed_Fase2;

GO

CREATE OR ALTER PROCEDURE dbo.sp_RegistrarCajero @Json nvarchar(max),@Cajero varchar(24),@Sesion
varchar(36)=NULL,@Solicitud varchar(64)

AS BEGIN SET NOCOUNT ON; EXEC dbo.sp_Operacion 'REGISTRAR_CAJERO',@Json,@Cajero,@Sesion,@Solicitud; END;

GO

CREATE OR ALTER PROCEDURE dbo.sp_Heartbeat @Json nvarchar(max),@Cajero varchar(24),@Sesion
varchar(36)=NULL,@Solicitud varchar(64)

AS BEGIN SET NOCOUNT ON; EXEC dbo.sp_Operacion 'HEARTBEAT',@Json,@Cajero,@Sesion,@Solicitud; END;

GO

CREATE OR ALTER PROCEDURE dbo.sp_ValidarCredenciales @Json nvarchar(max),@Cajero varchar(24),@Sesion
varchar(36)=NULL,@Solicitud varchar(64)

AS BEGIN SET NOCOUNT ON; EXEC dbo.sp_Operacion 'LOGIN',@Json,@Cajero,@Sesion,@Solicitud; END;

GO

CREATE OR ALTER PROCEDURE dbo.sp_ValidarToken @Json nvarchar(max),@Cajero varchar(24),@Sesion
varchar(36)=NULL,@Solicitud varchar(64)

AS BEGIN SET NOCOUNT ON; EXEC dbo.sp_Operacion 'VALIDAR_TOKEN',@Json,@Cajero,@Sesion,@Solicitud; END;

GO

CREATE OR ALTER PROCEDURE dbo.sp_Retiro @Json nvarchar(max),@Cajero varchar(24),@Sesion
varchar(36)=NULL,@Solicitud varchar(64)

AS BEGIN SET NOCOUNT ON; EXEC dbo.sp_Operacion 'RETIRO',@Json,@Cajero,@Sesion,@Solicitud; END;

GO

CREATE OR ALTER PROCEDURE dbo.sp_Deposito @Json nvarchar(max),@Cajero varchar(24),@Sesion
varchar(36)=NULL,@Solicitud varchar(64)

AS BEGIN SET NOCOUNT ON; EXEC dbo.sp_Operacion 'DEPOSITO',@Json,@Cajero,@Sesion,@Solicitud; END;

GO

CREATE OR ALTER PROCEDURE dbo.sp_CambiarPin @Json nvarchar(max),@Cajero varchar(24),@Sesion
varchar(36)=NULL,@Solicitud varchar(64)

AS BEGIN SET NOCOUNT ON; EXEC dbo.sp_Operacion 'CAMBIO_PIN',@Json,@Cajero,@Sesion,@Solicitud; END;

GO

CREATE OR ALTER PROCEDURE dbo.sp_ConsultaUsuario @Json nvarchar(max),@Cajero varchar(24),@Sesion
varchar(36)=NULL,@Solicitud varchar(64)

AS BEGIN SET NOCOUNT ON; EXEC dbo.sp_Operacion 'CONSULTA_SALDO',@Json,@Cajero,@Sesion,@Solicitud; END;

GO

CREATE OR ALTER PROCEDURE dbo.sp_UltimasTransacciones @Json nvarchar(max),@Cajero varchar(24),@Sesion
varchar(36)=NULL,@Solicitud varchar(64)

AS BEGIN SET NOCOUNT ON; EXEC dbo.sp_Operacion 'ULTIMAS_TRANSACCIONES',@Json,@Cajero,@Sesion,@Solicitud; END;

GO

CREATE OR ALTER PROCEDURE dbo.sp_CerrarSesion @Json nvarchar(max),@Cajero varchar(24),@Sesion
varchar(36)=NULL,@Solicitud varchar(64)

AS BEGIN SET NOCOUNT ON; EXEC dbo.sp_Operacion 'LOGOUT',@Json,@Cajero,@Sesion,@Solicitud; END;

GO

CREATE OR ALTER PROCEDURE dbo.sp_EstadoSolicitud @Json nvarchar(max),@Cajero varchar(24),@Sesion
varchar(36)=NULL,@Solicitud varchar(64)

AS BEGIN SET NOCOUNT ON; EXEC dbo.sp_Operacion 'ESTADO_SOLICITUD',@Json,@Cajero,@Sesion,@Solicitud; END;

GO
```

04_Administracion.sql

```
USE BD_CajeroRed_Fase2;

GO

CREATE OR ALTER TRIGGER dbo.tr_TarjetaBitacora ON dbo.Tarjeta AFTER INSERT,UPDATE

AS

BEGIN

 SET NOCOUNT ON;

 INSERT dbo.Bitacora(Origen,Accion,Detalle)

 SELECT 'SQL','CAMBIO_TARJETA',CONCAT('Usuario ',i.UsuarioId,'; tarjeta **** ',RIGHT(i.Numero,4),';
activa=',i.Activa)

 FROM inserted i LEFT JOIN deleted d ON d.Id=i.Id WHERE d.Id IS NULL OR d.Activa&lt;&gt;i.Activa OR
d.Numero&lt;&gt;i.Numero;

END;

GO

CREATE OR ALTER PROCEDURE dbo.sp_AdminOperacion @Administrador int,@Tipo varchar(40),@Json nvarchar(max)

AS

BEGIN

 SET NOCOUNT ON;SET XACT_ABORT ON;

 IF NOT EXISTS(SELECT 1 FROM dbo.Administrador WHERE Id=@Administrador AND Activo=1 AND Rol='Administrador')
THROW 50003,'Administrador no autorizado.',1;

 DECLARE @u int=TRY_CONVERT(int,JSON_VALUE(@Json,'$.usuarioId')),@c varchar(24)=JSON_VALUE(@Json,'$.cajero'),

 @nombre nvarchar(120)=JSON_VALUE(@Json,'$.nombre'),@pin varchar(50)=JSON_VALUE(@Json,'$.pin'),@tarjeta
varchar(50)=JSON_VALUE(@Json,'$.tarjeta'),

 @saldo decimal(18,2)=TRY_CONVERT(decimal(18,2),JSON_VALUE(@Json,'$.saldo')),@limite
decimal(18,2)=TRY_CONVERT(decimal(18,2),JSON_VALUE(@Json,'$.limite')),

 @salt varchar(64)=CONVERT(varchar(64),CRYPT_GEN_RANDOM(32),2),@total decimal(18,2),@anterior
decimal(18,2),@id bigint,@fecha date=CAST(SYSDATETIME() AS date);

 BEGIN TRY

 BEGIN TRAN;

 IF @Tipo='CREAR_USUARIO'

 BEGIN

  IF NULLIF(TRIM(@nombre),'') IS NULL OR @saldo IS NULL OR @saldo&lt;0 OR @limite IS NULL OR @limite&lt;0 THROW
50004,'Nombre, saldo o límite inválidos.',1;

  IF LEN(ISNULL(@pin,''))&lt;&gt;4 OR @pin LIKE '%[^0-9]%' OR LEN(ISNULL(@tarjeta,''))&lt;&gt;16 OR @tarjeta LIKE
'%[^0-9]%' THROW 50005,'Tarjeta o PIN inválidos.',1;

  INSERT dbo.Usuario(Nombre,Saldo,LimiteDiario) VALUES(@nombre,@saldo,@limite);SET @u=SCOPE_IDENTITY();

  INSERT dbo.Tarjeta(UsuarioId,Numero,PinHash,Salt)
VALUES(@u,@tarjeta,HASHBYTES('SHA2_256',CONVERT(varchar(200),@salt+@pin)),@salt);

 END

 ELSE IF @Tipo IN('EDITAR_USUARIO','CAMBIAR_LIMITE','CAMBIAR_TARJETA','BLOQUEAR','DESBLOQUEAR','BAJA_USUARIO',
'RESTABLECER_PIN')

 BEGIN

  SELECT @anterior=LimiteDiario FROM dbo.Usuario WITH(UPDLOCK,ROWLOCK) WHERE Id=@u;

  IF @anterior IS NULL THROW 50006,'Usuario inexistente.',1;

  IF @Tipo='EDITAR_USUARIO'

  BEGIN

   IF NULLIF(TRIM(@nombre),'') IS NULL THROW 50004,'Nombre requerido.',1;

   UPDATE dbo.Usuario SET Nombre=@nombre WHERE Id=@u;

  END;

  IF @Tipo='CAMBIAR_LIMITE'

  BEGIN
```

```
   IF @limite IS NULL OR @limite&lt;0 THROW 50004,'Límite inválido.',1;

   INSERT dbo.HistorialLimite(UsuarioId,Anterior,Nuevo,AdministradorId)
VALUES(@u,@anterior,@limite,@Administrador);

   UPDATE dbo.Usuario SET LimiteDiario=@limite WHERE Id=@u;

  END;

  IF @Tipo IN('CAMBIAR_TARJETA','RESTABLECER_PIN')

  BEGIN

   IF LEN(ISNULL(@pin,''))&lt;&gt;4 OR @pin LIKE '%[^0-9]%' THROW 50005,'PIN temporal inválido.',1;

   IF @Tipo='CAMBIAR_TARJETA'

   BEGIN

    IF LEN(ISNULL(@tarjeta,''))&lt;&gt;16 OR @tarjeta LIKE '%[^0-9]%' THROW 50005,'Tarjeta inválida.',1;

    UPDATE dbo.Tarjeta SET Activa=0,Baja=SYSDATETIME() WHERE UsuarioId=@u AND Activa=1;

    INSERT dbo.Tarjeta(UsuarioId,Numero,PinHash,Salt,PinTemporal)
VALUES(@u,@tarjeta,HASHBYTES('SHA2_256',CONVERT(varchar(200),@salt+@pin)),@salt,1);

   END

   ELSE UPDATE dbo.Tarjeta SET
Salt=@salt,PinHash=HASHBYTES('SHA2_256',CONVERT(varchar(200),@salt+@pin)),PinTemporal=1 WHERE UsuarioId=@u AND
 Activa=1;

   INSERT dbo.CambioPin(UsuarioId,Origen,Resultado) VALUES(@u,'ADMINISTRADOR','PIN temporal asignado');

  END;

  IF @Tipo='BLOQUEAR' UPDATE dbo.Usuario SET Bloqueado=1 WHERE Id=@u;

  IF @Tipo='DESBLOQUEAR' UPDATE dbo.Usuario SET Bloqueado=0,Intentos=0 WHERE Id=@u;

  IF @Tipo='BAJA_USUARIO'

  BEGIN UPDATE dbo.Usuario SET Activo=0 WHERE Id=@u;UPDATE dbo.Tarjeta SET Activa=0,Baja=SYSDATETIME() WHERE
UsuarioId=@u AND Activa=1;END;

  IF @Tipo NOT IN('EDITAR_USUARIO','CAMBIAR_LIMITE')

  BEGIN UPDATE dbo.Sesion SET Fin=SYSDATETIME() WHERE UsuarioId=@u AND Fin IS NULL;UPDATE dbo.Token SET
Usado=1 WHERE UsuarioId=@u;END;

 END

 ELSE IF @Tipo='ALTA_CAJERO'

 BEGIN

  IF NULLIF(TRIM(@c),'') IS NULL OR LEN(@c)&gt;24 OR NULLIF(TRIM(@nombre),'') IS NULL THROW 50007,'Código y
nombre de cajero requeridos.',1;

  INSERT dbo.Cajero(Codigo,Nombre,Ubicacion) VALUES(@c,@nombre,ISNULL(JSON_VALUE(@Json,'$.ubicacion'),''));

  INSERT dbo.InventarioCajero SELECT @c,Valor,0 FROM dbo.Denominacion;

 END

 ELSE IF @Tipo IN('EDITAR_CAJERO','BAJA_CAJERO','HABILITAR','FUERA_DE_SERVICIO','INICIALIZAR','CARGAR')

 BEGIN

  IF NOT EXISTS(SELECT 1 FROM dbo.Cajero WITH(UPDLOCK,ROWLOCK) WHERE Codigo=@c AND Activo=1) THROW
50008,'Cajero inexistente o inactivo.',1;

  IF @Tipo='EDITAR_CAJERO'

  BEGIN

   IF NULLIF(TRIM(@nombre),'') IS NULL THROW 50007,'Nombre requerido.',1;

   UPDATE dbo.Cajero SET Nombre=@nombre,Ubicacion=ISNULL(JSON_VALUE(@Json,'$.ubicacion'),'') WHERE Codigo=@c;

  END;

  IF @Tipo='BAJA_CAJERO' UPDATE dbo.Cajero SET Activo=0,Habilitado=0 WHERE Codigo=@c;

  IF @Tipo='HABILITAR' UPDATE dbo.Cajero SET Habilitado=1 WHERE Codigo=@c;

  IF @Tipo IN('FUERA_DE_SERVICIO','BAJA_CAJERO')

  BEGIN UPDATE dbo.Cajero SET Habilitado=0 WHERE Codigo=@c;UPDATE dbo.Sesion SET Fin=SYSDATETIME() WHERE
Cajero=@c AND Fin IS NULL;END;

  IF @Tipo IN('INICIALIZAR','CARGAR')
```

```
  BEGIN

   IF @Tipo='INICIALIZAR' AND EXISTS(SELECT 1 FROM dbo.Cajero WHERE Codigo=@c AND FechaInicializacion=@fecha)
THROW 50009,'El cajero ya fue inicializado hoy.',1;

   IF @Tipo='CARGAR' AND NOT EXISTS(SELECT 1 FROM dbo.Cajero WHERE Codigo=@c AND FechaInicializacion=@fecha)
THROW 50010,'Inicialice el cajero hoy antes de agregar efectivo.',1;

   IF JSON_QUERY(@Json,'$.desglose') IS NULL THROW 50011,'Falta desglose.',1;

   IF EXISTS(SELECT 1 FROM OPENJSON(@Json,'$.desglose') WHERE TRY_CONVERT(int,[key]) IS NULL OR
TRY_CONVERT(int,[key]) NOT IN(200,100,50,20,10,5,1)

    OR TRY_CONVERT(int,value) IS NULL OR TRY_CONVERT(int,value)&lt;0 OR TRY_CONVERT(int,value)&gt;30000 OR type&lt;&gt;2)

    OR EXISTS(SELECT [key] FROM OPENJSON(@Json,'$.desglose') GROUP BY [key] HAVING COUNT(*)&gt;1) THROW
50011,'Desglose inválido.',1;

   DECLARE @b TABLE(d int PRIMARY KEY,n int);

   INSERT @b SELECT CONVERT(int,[key]),CONVERT(int,value) FROM OPENJSON(@Json,'$.desglose');

   SELECT @total=SUM(CONVERT(bigint,d)*n) FROM @b;

   SELECT @anterior=SUM(CONVERT(bigint,Denominacion)*Cantidad) FROM dbo.InventarioCajero WHERE Cajero=@c;

   IF @total IS NULL OR @total&lt;=0 THROW 50011,'Carga vacía.',1;

   IF @Tipo='INICIALIZAR' AND @total&gt;(SELECT Valor FROM dbo.Parametro WHERE Clave='MaxInicializacion') THROW
50012,'Inicialización supera el máximo permitido.',1;

   IF @Tipo='CARGAR' AND @anterior+@total&gt;(SELECT Valor FROM dbo.Parametro WHERE Clave='MaxEfectivo') THROW
50013,'Se supera la capacidad de efectivo.',1;

   INSERT dbo.CargaEfectivo(Cajero,Tipo,Total,EfectivoAnterior,AdministradorId) VALUES(@c,CASE WHEN
@Tipo='INICIALIZAR' THEN 'INICIALIZACION' ELSE 'CARGA' END,@total,@anterior,@Administrador);

   SET @id=SCOPE_IDENTITY(); INSERT dbo.CargaEfectivoDetalle SELECT @id,d,n FROM @b;

   UPDATE i SET Cantidad=CASE WHEN @Tipo='INICIALIZAR' THEN ISNULL(b.n,0) ELSE i.Cantidad+ISNULL(b.n,0) END
FROM dbo.InventarioCajero i LEFT JOIN @b b ON b.d=i.Denominacion WHERE i.Cajero=@c;

   IF @Tipo='INICIALIZAR' UPDATE dbo.Cajero SET FechaInicializacion=@fecha WHERE Codigo=@c;

   EXEC dbo.sp_Alertas @c;

  END;

 END

 ELSE IF @Tipo='ATENDER_ALERTA'

  UPDATE dbo.Alerta SET Atendida=1,AdministradorId=@Administrador,FechaAtencion=SYSDATETIME() WHERE
Id=TRY_CONVERT(bigint,JSON_VALUE(@Json,'$.alertaId')) AND Atendida=0;

 ELSE IF @Tipo='PARAMETROS'

 BEGIN

  DECLARE @clave varchar(40)=JSON_VALUE(@Json,'$.clave'),@valor
decimal(18,2)=TRY_CONVERT(decimal(18,2),JSON_VALUE(@Json,'$.valor'));

  IF @valor IS NULL OR @valor&lt;=0 OR @clave NOT
IN('Puerto','UmbralEfectivo','VigenciaToken','MaxInicializacion','MaxEfectivo') THROW 50014,'Parámetro
inválido.',1;

  IF @clave='Puerto' AND (@valor&gt;65535 OR @valor&lt;&gt;FLOOR(@valor)) THROW 50014,'Puerto inválido.',1;

  IF @clave='VigenciaToken' AND (@valor&gt;120 OR @valor&lt;&gt;FLOOR(@valor)) THROW 50014,'Token: entre 1 y 120
segundos.',1;

  IF @clave='MaxInicializacion' AND @valor&gt;10000 THROW 50014,'El máximo de inicialización no puede superar Q
10000.',1;

  IF @clave='MaxEfectivo' AND @valor&gt;30000 THROW 50014,'La capacidad no puede superar Q 30000.',1;

  IF @clave='MaxInicializacion' AND @valor&gt;(SELECT Valor FROM dbo.Parametro WHERE Clave='MaxEfectivo') THROW
50014,'Inicialización supera capacidad.',1;

  IF @clave='MaxEfectivo' AND (@valor&lt;(SELECT Valor FROM dbo.Parametro WHERE Clave='MaxInicializacion') OR
EXISTS(SELECT Cajero FROM dbo.InventarioCajero GROUP BY Cajero HAVING SUM(Denominacion*Cantidad)&gt;@valor))
THROW 50014,'La capacidad es inferior a un inventario o a la inicialización.',1;

  UPDATE dbo.Parametro SET Valor=@valor WHERE Clave=@clave;

 END

 ELSE IF @Tipo='MENSAJE' BEGIN IF NULLIF(JSON_VALUE(@Json,'$.texto'),'') IS NULL THROW 50015,'Mensaje
requerido.',1;END

 ELSE THROW 50016,'Acción administrativa desconocida.',1;
```

```
 INSERT dbo.Bitacora(Origen,Accion,Detalle) VALUES(CONCAT('ADMIN-',@Administrador),@Tipo,CONCAT('Usuario
',@u,'; cajero ',@c));

 COMMIT;

 SELECT @u UsuarioId,N'Operación administrativa completada.' Mensaje;

 END TRY

 BEGIN CATCH

  IF XACT_STATE()&lt;&gt;0 ROLLBACK;

  THROW;

 END CATCH;

END;

GO

CREATE OR ALTER PROCEDURE dbo.sp_InicializarCajero @Administrador int,@Json nvarchar(max)

AS EXEC dbo.sp_AdminOperacion @Administrador,'INICIALIZAR',@Json;

GO

CREATE OR ALTER PROCEDURE dbo.sp_AgregarEfectivo @Administrador int,@Json nvarchar(max)

AS EXEC dbo.sp_AdminOperacion @Administrador,'CARGAR',@Json;

GO

CREATE OR ALTER PROCEDURE dbo.sp_CambiarTarjeta @Administrador int,@Json nvarchar(max)

AS EXEC dbo.sp_AdminOperacion @Administrador,'CAMBIAR_TARJETA',@Json;

GO

CREATE OR ALTER PROCEDURE dbo.sp_CambiarLimite @Administrador int,@Json nvarchar(max)

AS EXEC dbo.sp_AdminOperacion @Administrador,'CAMBIAR_LIMITE',@Json;

GO
```

05_Vistas.sql

```
USE BD_CajeroRed_Fase2;

GO

CREATE OR ALTER VIEW dbo.vw_RetiradoHoyPorUsuario AS

SELECT UsuarioId, SUM(Monto) RetiradoHoy FROM dbo.Transaccion

WHERE Tipo='RETIRO' AND CodigoRespuesta='00' AND Fecha&gt;=CAST(SYSDATETIME() AS date) AND
Fecha&lt;DATEADD(day,1,CAST(SYSDATETIME() AS date)) GROUP BY UsuarioId;

GO

CREATE OR ALTER VIEW dbo.vw_EstadoCajeros AS

SELECT c.Codigo,c.Nombre,c.Ubicacion,c.IP,c.Equipo,c.Version,c.Activo,c.Habilitado,c.Conectado,c.FechaIniciali
zacion,c.UltimoHeartbeat,

 ISNULL(i.Efectivo,0) Efectivo,c.UltimaConexion,t.UltimaTransaccion,

 CASE WHEN c.Activo=0 THEN N'Inactivo' WHEN c.Conectado=0 THEN N'Desconectado' WHEN c.Habilitado=0 THEN
N'Fuera de servicio'

 WHEN c.FechaInicializacion IS NULL OR c.FechaInicializacion&lt;&gt;CAST(SYSDATETIME() AS date) THEN N'Sin
inicializar'

 WHEN ISNULL(i.Efectivo,0)&lt;(SELECT Valor FROM dbo.Parametro WHERE Clave='UmbralEfectivo') THEN N'Efectivo
bajo' ELSE N'En línea' END Estado,

 ISNULL(i.B200,0) B200,ISNULL(i.B100,0) B100,ISNULL(i.B50,0) B50,ISNULL(i.B20,0) B20,ISNULL(i.B10,0)
B10,ISNULL(i.B5,0) B5,ISNULL(i.B1,0) B1

FROM dbo.Cajero c

OUTER APPLY(SELECT SUM(Denominacion*Cantidad) Efectivo,SUM(CASE WHEN Denominacion=200 THEN Cantidad ELSE 0
END) B200,

SUM(CASE WHEN Denominacion=100 THEN Cantidad ELSE 0 END) B100,SUM(CASE WHEN Denominacion=50 THEN Cantidad ELSE
 0 END) B50,

SUM(CASE WHEN Denominacion=20 THEN Cantidad ELSE 0 END) B20,SUM(CASE WHEN Denominacion=10 THEN Cantidad ELSE 0
 END) B10,

SUM(CASE WHEN Denominacion=5 THEN Cantidad ELSE 0 END) B5,SUM(CASE WHEN Denominacion=1 THEN Cantidad ELSE 0
END) B1 FROM dbo.InventarioCajero WHERE Cajero=c.Codigo)i

OUTER APPLY(SELECT MAX(Fecha) UltimaTransaccion FROM dbo.Transaccion WHERE Cajero=c.Codigo)t;

GO

CREATE OR ALTER VIEW dbo.vw_ResumenDiario AS

SELECT (SELECT COUNT(*) FROM dbo.Cajero WHERE Conectado=1 AND Activo=1) CajerosEnLinea,

 (SELECT COUNT(*) FROM dbo.Cajero WHERE Activo=1) TotalCajeros,

 (SELECT ISNULL(SUM(CONVERT(bigint,Denominacion)*Cantidad),0) FROM dbo.InventarioCajero) EfectivoRed,

 ISNULL(SUM(CASE WHEN Tipo='RETIRO' AND CodigoRespuesta='00' THEN Monto ELSE 0 END),0) TotalRetirado,

 ISNULL(SUM(CASE WHEN Tipo='DEPOSITO' AND CodigoRespuesta='00' THEN Monto ELSE 0 END),0) TotalDepositado,

 ISNULL(SUM(CASE WHEN Tipo='DEPOSITO' AND CodigoRespuesta='00' THEN Monto ELSE 0 END)/NULLIF(COUNT(DISTINCT
CASE WHEN Tipo='DEPOSITO' AND CodigoRespuesta='00' THEN UsuarioId END),0),0) PromedioDepositadoPorUsuario,

 COUNT(*) Transacciones,SUM(CASE WHEN CodigoRespuesta&lt;&gt;'00' THEN 1 ELSE 0 END) Rechazadas,

 (SELECT COUNT(*) FROM dbo.CambioPin WHERE Fecha&gt;=CAST(SYSDATETIME() AS date)) CambiosPin,

 (SELECT COUNT(*) FROM dbo.Alerta WHERE Atendida=0) AlertasActivas

FROM dbo.Transaccion WHERE Fecha&gt;=CAST(SYSDATETIME() AS date) AND Fecha&lt;DATEADD(day,1,CAST(SYSDATETIME() AS
date));

GO

CREATE OR ALTER VIEW dbo.vw_CambiosPinHoy AS

SELECT p.Id,u.Nombre,p.Cajero,p.Fecha,p.Origen,p.Resultado FROM dbo.CambioPin p JOIN dbo.Usuario u ON
u.Id=p.UsuarioId WHERE p.Fecha&gt;=CAST(SYSDATETIME() AS date);

GO

CREATE OR ALTER VIEW dbo.vw_Usuarios AS

SELECT u.Id,u.Nombre,u.Saldo,u.LimiteDiario,u.Activo,u.Bloqueado,u.Intentos,

 ISNULL(r.RetiradoHoy,0) RetiradoHoy,CASE WHEN u.LimiteDiario&gt;ISNULL(r.RetiradoHoy,0) THEN u.LimiteDiario-
ISNULL(r.RetiradoHoy,0) ELSE 0 END DisponibleHoy,
```

```
 CASE WHEN u.Saldo&lt;CASE WHEN u.LimiteDiario&gt;ISNULL(r.RetiradoHoy,0) THEN u.LimiteDiario-
ISNULL(r.RetiradoHoy,0) ELSE 0 END THEN u.Saldo

 ELSE CASE WHEN u.LimiteDiario&gt;ISNULL(r.RetiradoHoy,0) THEN u.LimiteDiario-ISNULL(r.RetiradoHoy,0) ELSE 0 END
END MaximoRetirable,

 '**** '+RIGHT(t.Numero,4) Tarjeta,t.PinTemporal,a.UltimoAcceso,a.UltimoCajero

FROM dbo.Usuario u LEFT JOIN dbo.Tarjeta t ON t.UsuarioId=u.Id AND t.Activa=1 LEFT JOIN
dbo.vw_RetiradoHoyPorUsuario r ON r.UsuarioId=u.Id

OUTER APPLY(SELECT TOP(1) Inicio UltimoAcceso,Cajero UltimoCajero FROM dbo.Sesion WHERE UsuarioId=u.Id AND
Resultado='00' ORDER BY Inicio DESC)a;

GO

CREATE OR ALTER VIEW dbo.vw_Transacciones AS

SELECT t.Id,t.Fecha,t.Cajero,u.Nombre Usuario,t.UsuarioId,'**** '+RIGHT(c.Numero,4)
Tarjeta,t.Tipo,t.Monto,t.CodigoRespuesta Resultado,t.Motivo,

 ISNULL(d.Desglose,'') Desglose,t.Solicitud FROM dbo.Transaccion t LEFT JOIN dbo.Usuario u ON u.Id=t.UsuarioId
 LEFT JOIN dbo.Tarjeta c ON c.Id=t.TarjetaId

OUTER APPLY(SELECT STRING_AGG(CONCAT(Denominacion,' x ',Cantidad),'; ') Desglose FROM dbo.TransaccionDetalle
WHERE TransaccionId=t.Id)d;

GO

CREATE OR ALTER VIEW dbo.vw_Arqueo AS

SELECT c.Codigo,c.Nombre,ISNULL(a.Total,0) Inicializacion,ISNULL(g.Total,0)
CargasAdicionales,ISNULL(m.Depositos,0) Depositos,

 ISNULL(m.Retiros,0) Retiros,ISNULL(a.Total,0)+ISNULL(g.Total,0)+ISNULL(m.Depositos,0)-ISNULL(m.Retiros,0)
Esperado,

 ISNULL(i.Efectivo,0) InventarioActual,ISNULL(i.Efectivo,0)-
(ISNULL(a.Total,0)+ISNULL(g.Total,0)+ISNULL(m.Depositos,0)-ISNULL(m.Retiros,0)) Diferencia

FROM dbo.Cajero c

OUTER APPLY(SELECT TOP(1) Total,Fecha FROM dbo.CargaEfectivo WHERE Cajero=c.Codigo AND Tipo='INICIALIZACION'
ORDER BY Fecha DESC,Id DESC)a

OUTER APPLY(SELECT SUM(Total) Total FROM dbo.CargaEfectivo WHERE Cajero=c.Codigo AND Tipo='CARGA' AND
Fecha&gt;=a.Fecha)g

OUTER APPLY(SELECT SUM(CASE WHEN Tipo='DEPOSITO' THEN Monto ELSE 0 END) Depositos,SUM(CASE WHEN Tipo='RETIRO'
THEN Monto ELSE 0 END) Retiros FROM dbo.Transaccion WHERE Cajero=c.Codigo AND CodigoRespuesta='00' AND
Fecha&gt;=a.Fecha)m

OUTER APPLY(SELECT SUM(Denominacion*Cantidad) Efectivo FROM dbo.InventarioCajero WHERE Cajero=c.Codigo)i;

GO
```

06_DatosPrueba.sql

```
USE BD_CajeroRed_Fase2;

GO

-- Solo datos ficticios. Clave administrativa de demostración: GrupoCajero#2026.

-- PBKDF2-SHA256, 210000 iteraciones, salt aleatorio de 32 bytes.

IF EXISTS(SELECT 1 FROM dbo.Usuario) THROW 50020,'Este script requiere tablas de usuarios vacías.',1;

INSERT dbo.Administrador(Usuario,Hash,Salt) VALUES(N'admin',0x0909f886ed78fb6dc48980f5e9706b8233d5f1d5a8e35f54
5ba2337167ff3830,0xe165d0c008403ac5c90a6553ca763b05cfa3535320c9122a9fa53900fc19f69a);

DECLARE @i int=1,@c varchar(24),@salt varchar(64);

WHILE @i&lt;=8

BEGIN

 INSERT dbo.Usuario(Nombre,Saldo,LimiteDiario) VALUES(CONCAT(N'Usuario de prueba ',@i),5000,CASE WHEN @i=1
THEN 500 ELSE 2000 END);

 SET @salt=CONVERT(varchar(64),CRYPT_GEN_RANDOM(32),2);

 INSERT dbo.Tarjeta(UsuarioId,Numero,PinHash,Salt) VALUES(SCOPE_IDENTITY(),CONCAT('400000000000',RIGHT('0000'+
CONVERT(varchar(4),@i),4)),HASHBYTES('SHA2_256',CONVERT(varchar(200),@salt+'1234')),@salt);

 SET @i+=1;

END;

SET @i=1;

WHILE @i&lt;=8

BEGIN

 SET @c=CONCAT('ATM-',RIGHT('00'+CONVERT(varchar(3),@i),2));

 INSERT dbo.Cajero(Codigo,Nombre,Ubicacion) VALUES(@c,CONCAT(N'Cajero ',@i),N'Laboratorio');

 INSERT dbo.InventarioCajero SELECT @c,Valor,0 FROM dbo.Denominacion;

 SET @i+=1;

END;

GO

-- Por diseño se inicializan desde el dashboard cada día, no al arrancar el cliente.
```

07_Permisos.sql

```
USE BD_CajeroRed_Fase2;
GO
IF DATABASE_PRINCIPAL_ID('rol_CajeroServidor') IS NULL CREATE ROLE rol_CajeroServidor;
-- Solo lectura de tablas del esquema, procedimientos y las escrituras de monitoreo.
GRANT SELECT ON SCHEMA::dbo TO rol_CajeroServidor;
GRANT EXECUTE ON SCHEMA::dbo TO rol_CajeroServidor;
GRANT INSERT ON dbo.Bitacora TO rol_CajeroServidor;
GRANT UPDATE ON dbo.Cajero TO rol_CajeroServidor;
GRANT UPDATE ON dbo.Sesion TO rol_CajeroServidor;
IF DATABASE_PRINCIPAL_ID('CajeroServidor') IS NULL CREATE USER CajeroServidor WITHOUT LOGIN;
ALTER ROLE rol_CajeroServidor ADD MEMBER CajeroServidor;
GRANT IMPERSONATE ON USER::CajeroServidor TO rol_CajeroServidor;
-- En autenticación Windows el servidor abre la conexión y adopta este usuario
-- de base de datos con EXECUTE AS USER; cada conexión se cierra sin pooling.
-- No cambia el modo de autenticación de SQL ni requiere reiniciar el servicio.
-- El instalador configura un login propio. Nunca se incluye ni se usa sa.
-- Ejemplo para un login YA CREADO por el DBA:
-- CREATE USER CajeroServidor FOR LOGIN CajeroServidor;
-- ALTER ROLE rol_CajeroServidor ADD MEMBER CajeroServidor;
GO
```

08_Interbancario.sql

```
USE BD_CajeroRed_Fase2;

GO

-- Migración aditiva y repetible: no reinicializa usuarios ni inventarios.

IF COL_LENGTH('dbo.Usuario','NumeroCuenta') IS NULL ALTER TABLE dbo.Usuario ADD NumeroCuenta varchar(20) NULL;

IF NOT EXISTS(SELECT 1 FROM sys.indexes WHERE name='UX_UsuarioNumeroCuenta' AND
object_id=OBJECT_ID('dbo.Usuario'))

 EXEC('CREATE UNIQUE INDEX UX_UsuarioNumeroCuenta ON dbo.Usuario(NumeroCuenta) WHERE NumeroCuenta IS NOT
NULL');

IF OBJECT_ID('dbo.BancoRed') IS NULL

CREATE TABLE dbo.BancoRed(

 CodigoBanco varchar(15) PRIMARY KEY,NombreBanco nvarchar(100) NOT NULL,IP varchar(50) NOT NULL,

 Puerto int NOT NULL DEFAULT 5000 CHECK(Puerto BETWEEN 1 AND 65535),EsPropio bit NOT NULL DEFAULT 0,

 Confiable bit NOT NULL DEFAULT 0,Activo bit NOT NULL DEFAULT 1,

 LimiteDiario decimal(18,2) NOT NULL DEFAULT 25000 CHECK(LimiteDiario&gt;0),

 TransferidoHoy decimal(18,2) NOT NULL DEFAULT 0 CHECK(TransferidoHoy&gt;=0),

 FechaUltimoReinicio date NOT NULL DEFAULT CAST(SYSDATETIME() AS date));

IF NOT EXISTS(SELECT 1 FROM sys.indexes WHERE name='UX_BancoPropio' AND object_id=OBJECT_ID('dbo.BancoRed'))

 CREATE UNIQUE INDEX UX_BancoPropio ON dbo.BancoRed(EsPropio) WHERE EsPropio=1;

IF OBJECT_ID('dbo.TransferenciaBancaria') IS NULL

CREATE TABLE dbo.TransferenciaBancaria(

 IdTransferencia bigint IDENTITY PRIMARY KEY,TokenTransferencia varchar(40) NOT NULL,

 Direccion varchar(10) NOT NULL CHECK(Direccion IN('ENTRANTE','SALIENTE')),

 BancoOrigen varchar(15) NOT NULL REFERENCES dbo.BancoRed(CodigoBanco),

 BancoDestino varchar(15) NOT NULL REFERENCES dbo.BancoRed(CodigoBanco),

 UsuarioId int NOT NULL REFERENCES dbo.Usuario(Id),

 CuentaOrigen varchar(20) NOT NULL,CuentaDestino varchar(20) NOT NULL,

 NombreOrigen nvarchar(120) NOT NULL,NombreDestino nvarchar(120) NOT NULL,

 Monto decimal(18,2) NOT NULL CHECK(Monto&gt;0),Concepto nvarchar(200) NOT NULL,

 Estado varchar(20) NOT NULL CHECK(Estado IN('RESERVADA','APROBADA','RECHAZADA','INCIERTA')),

 Huella varbinary(32) NOT NULL,RespuestaDatos nvarchar(max) NOT NULL DEFAULT '{}',

 FechaCreacion datetime2 NOT NULL DEFAULT SYSDATETIME(),FechaActualizacion datetime2 NOT NULL DEFAULT
SYSDATETIME(),

 UNIQUE(BancoOrigen,TokenTransferencia));

GO

CREATE OR ALTER VIEW dbo.vw_TransferenciasUsuario AS

SELECT t.IdTransferencia,t.UsuarioId,t.FechaCreacion
Fecha,t.TokenTransferencia,t.Direccion,t.BancoOrigen,t.BancoDestino,

 '**** '+RIGHT(t.CuentaOrigen,4) CuentaOrigen,'**** '+RIGHT(t.CuentaDestino,4) CuentaDestino,

 t.NombreOrigen,t.NombreDestino,t.Monto,t.Estado,t.Concepto

FROM dbo.TransferenciaBancaria t;

GO

CREATE OR ALTER PROCEDURE dbo.sp_IB_Operacion

 @Tipo varchar(40),@Json nvarchar(max),@IP varchar(50)='',@Administrador int=NULL

AS

BEGIN

 SET NOCOUNT ON;SET XACT_ABORT ON;

 DECLARE @codigo varchar(2)='00',@mensaje nvarchar(240)=N'Operación aprobada.',@datos nvarchar(max)='{}',
```

```
 @propio varchar(15),@nombre nvarchar(100),@bo varchar(15)=JSON_VALUE(@Json,'$.bancoOrigen'),

 @bd varchar(15)=JSON_VALUE(@Json,'$.bancoDestino'),@token
varchar(40)=JSON_VALUE(@Json,'$.tokenTransferencia'),

 @co varchar(20)=JSON_VALUE(@Json,'$.cuentaOrigen'),@cd
varchar(20)=COALESCE(JSON_VALUE(@Json,'$.cuentaDestino'),JSON_VALUE(@Json,'$.numeroCuenta')),

 @no nvarchar(120)=ISNULL(JSON_VALUE(@Json,'$.nombreOrigen'),''),@nd nvarchar(120),

 @concepto nvarchar(200)=ISNULL(JSON_VALUE(@Json,'$.concepto'),''),

 @m decimal(18,2)=TRY_CONVERT(decimal(18,2),JSON_VALUE(@Json,'$.monto')),

 @u int=TRY_CONVERT(int,JSON_VALUE(@Json,'$.usuarioId')),@hoy date=CAST(SYSDATETIME() AS date),

 @huella varbinary(32),@anterior varbinary(32),@estado varchar(20),@id bigint,@lock int;

 BEGIN TRY

 BEGIN TRAN;

 EXEC @lock=sys.sp_getapplock
@Resource='CajeroRed_Interbancario',@LockMode='Exclusive',@LockOwner='Transaction',@LockTimeout=2000;

 IF @lock&lt;0 THROW 50200,'Integración ocupada; reintente.',1;

 IF @Tipo IN('CONFIGURAR','ASIGNAR_CUENTA','RESERVAR','FINALIZAR')

 AND NOT EXISTS(SELECT 1 FROM dbo.Administrador WHERE Id=@Administrador AND Activo=1 AND Rol='Administrador')

 THROW 50201,'Administrador no autorizado.',1;

 IF @Tipo='CONFIGURAR'

 BEGIN

  DECLARE @c varchar(15)=JSON_VALUE(@Json,'$.codigoBanco'),@n nvarchar(100)=JSON_VALUE(@Json,'$.nombreBanco'),

  @ipb varchar(50)=JSON_VALUE(@Json,'$.ip'),@p int=TRY_CONVERT(int,JSON_VALUE(@Json,'$.puerto')),

  @es bit=TRY_CONVERT(bit,JSON_VALUE(@Json,'$.esPropio')),@conf
bit=TRY_CONVERT(bit,JSON_VALUE(@Json,'$.confiable')),

  @lim decimal(18,2)=TRY_CONVERT(decimal(18,2),JSON_VALUE(@Json,'$.limiteDiario'));

  IF NULLIF(@c,'') IS NULL OR @c LIKE '%[^A-Z0-9_-]%' OR NULLIF(TRIM(@n),'') IS NULL OR NULLIF(@ipb,'') IS
NULL

   OR @p IS NULL OR @p NOT BETWEEN 1 AND 65535 OR @es IS NULL OR @conf IS NULL OR @lim IS NULL OR @lim&lt;=0

   THROW 50202,'Configuración bancaria inválida.',1;

  IF @es=1 AND EXISTS(SELECT 1 FROM dbo.BancoRed WHERE EsPropio=1 AND CodigoBanco&lt;&gt;@c)

  BEGIN

   IF EXISTS(SELECT 1 FROM dbo.TransferenciaBancaria) OR EXISTS(SELECT 1 FROM dbo.BancoRed WHERE
CodigoBanco=@c)

    THROW 50203,'No se puede cambiar un código con historial ni usar uno ya registrado.',1;

   UPDATE dbo.BancoRed SET CodigoBanco=@c WHERE EsPropio=1;

  END;

  IF EXISTS(SELECT 1 FROM dbo.BancoRed WHERE CodigoBanco=@c AND EsPropio&lt;&gt;@es)

   THROW 50204,'No se puede cambiar la propiedad de un banco existente.',1;

  IF EXISTS(SELECT 1 FROM dbo.BancoRed WHERE CodigoBanco=@c AND FechaUltimoReinicio=@hoy AND
TransferidoHoy&gt;@lim)

   THROW 50205,'El límite no puede ser menor al cupo ya consumido.',1;

  IF EXISTS(SELECT 1 FROM dbo.BancoRed WHERE CodigoBanco=@c)

   UPDATE dbo.BancoRed SET NombreBanco=@n,IP=@ipb,Puerto=@p,Confiable=@conf,LimiteDiario=@lim,Activo=1 WHERE
CodigoBanco=@c;

  ELSE INSERT dbo.BancoRed(CodigoBanco,NombreBanco,IP,Puerto,EsPropio,Confiable,LimiteDiario)
VALUES(@c,@n,@ipb,@p,@es,@conf,@lim);

  INSERT dbo.Bitacora(Origen,Accion,Detalle)
VALUES(CONCAT('ADMIN-',@Administrador),'IB_CONFIGURAR',CONCAT('Banco ',@c));

  GOTO Fin;

 END;

 IF @Tipo='ASIGNAR_CUENTA'
```

```
 BEGIN

  SET @cd=JSON_VALUE(@Json,'$.numeroCuenta');

  IF @cd IS NULL OR LEN(@cd) NOT BETWEEN 1 AND 20 OR @cd LIKE '%[^0-9]%' THROW 50206,'Cuenta inválida: 1 a 20
dígitos.',1;

  -- Evita que una cuenta de otro usuario se confunda con una tarjeta.

  IF EXISTS(SELECT 1 FROM dbo.Tarjeta WHERE Numero=@cd AND UsuarioId&lt;&gt;@u) THROW 50207,'La cuenta coincide con
la tarjeta de otro usuario.',1;

  UPDATE dbo.Usuario SET NumeroCuenta=@cd WHERE Id=@u AND Activo=1;

  IF @@ROWCOUNT=0 THROW 50208,'Usuario no disponible.',1;

  INSERT dbo.Bitacora(Origen,Accion,Detalle)
VALUES(CONCAT('ADMIN-',@Administrador),'IB_ASIGNAR_CUENTA',CONCAT('Usuario ',@u));

  GOTO Fin;

 END;

 SELECT @propio=CodigoBanco,@nombre=NombreBanco FROM dbo.BancoRed WHERE EsPropio=1 AND Activo=1;

 IF @propio IS NULL BEGIN SET @codigo='91';SET @mensaje=N'Configure primero el banco propio.';GOTO Fin;END;

 UPDATE dbo.BancoRed SET TransferidoHoy=0,FechaUltimoReinicio=@hoy WHERE FechaUltimoReinicio&lt;&gt;@hoy;

 IF @Tipo='IB_PING'

 BEGIN

  SET @datos=(SELECT CodigoBanco codigoBanco,NombreBanco nombreBanco,LimiteDiario-TransferidoHoy
cupoDisponibleHoy,'EN_LINEA' estado FROM dbo.BancoRed WHERE CodigoBanco=@propio FOR JSON
PATH,WITHOUT_ARRAY_WRAPPER);

  SET @mensaje=N'Banco '+@propio+N' en línea';GOTO Fin;

 END;

 IF @Tipo IN('IB_VERIFICAR_CUENTA','IB_TRANSFERIR','IB_ESTADO_TRANSFERENCIA') AND ISNULL(@bd,@propio)&lt;&gt;@propio

 BEGIN SET @codigo='14';SET @mensaje=N'Banco destino incorrecto.';GOTO Fin;END;

 IF @Tipo='IB_VERIFICAR_CUENTA'

 BEGIN

  SELECT TOP(1) @u=u.Id,@nd=u.Nombre FROM dbo.Usuario u

  WHERE u.Activo=1 AND u.Bloqueado=0 AND (u.NumeroCuenta=@cd OR EXISTS(SELECT 1 FROM dbo.Tarjeta t WHERE
t.UsuarioId=u.Id AND t.Numero=@cd AND t.Activa=1));

  SET @datos=(SELECT CAST(CASE WHEN @u IS NULL THEN 0 ELSE 1 END AS bit) existe,@propio codigoBanco,@nombre
nombreBanco,

   @cd numeroCuenta,ISNULL(@nd,'') nombreTitular,LimiteDiario-TransferidoHoy cupoDisponibleBanco FROM
dbo.BancoRed WHERE CodigoBanco=@propio FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

  IF @u IS NULL BEGIN SET @codigo='14';SET @mensaje=N'La cuenta no existe en este banco.';END;

  GOTO Fin;

 END;

 IF @Tipo IN('IB_TRANSFERIR','IB_ESTADO_TRANSFERENCIA')

 AND NOT EXISTS(SELECT 1 FROM dbo.BancoRed WHERE CodigoBanco=@bo AND EsPropio=0 AND Activo=1 AND Confiable=1
AND IP=@IP)

 BEGIN SET @codigo='57';SET @mensaje=N'Banco origen o IP no autorizado.';GOTO Fin;END;

 IF @Tipo='IB_ESTADO_TRANSFERENCIA'

 BEGIN

  SELECT @estado=Estado,@datos=RespuestaDatos FROM dbo.TransferenciaBancaria WHERE BancoOrigen=@bo AND
TokenTransferencia=@token AND Direccion='ENTRANTE';

  IF @estado IS NULL BEGIN SET @codigo='25';SET @mensaje=N'Transferencia no registrada; no confirma rechazo ni
 autoriza devolución.';END;

  GOTO Fin;

 END;

 IF @Tipo='IB_TRANSFERIR'

 BEGIN

  IF @token IS NULL OR LEN(@token)&lt;&gt;19 OR @token NOT LIKE
```

```
'TRF-[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]-[0-9][0-9][0-9][0-9][0-9][0-9]'

   OR @m IS NULL OR @m&lt;=0 OR @co IS NULL OR @cd IS NULL OR @bo=@propio

  BEGIN SET @codigo='30';SET @mensaje=N'Token, cuentas o monto inválidos.';GOTO Fin;END;

  SET @huella=HASHBYTES('SHA2_256',CONCAT(@bo,'|',@bd,'|',@co,'|',@cd,'|',@m,'|',@no,'|',@concepto));

  SELECT @anterior=Huella,@datos=RespuestaDatos FROM dbo.TransferenciaBancaria WHERE BancoOrigen=@bo AND
TokenTransferencia=@token;

  IF @anterior IS NOT NULL

  BEGIN

   IF @anterior&lt;&gt;@huella BEGIN SET @codigo='94';SET @mensaje=N'Token reutilizado con contenido distinto.';SET
@datos='{}';END;

   GOTO Fin;

  END;

  SET @u=NULL;

  SELECT TOP(1) @u=u.Id,@nd=u.Nombre FROM dbo.Usuario u WITH(UPDLOCK,ROWLOCK)

  WHERE u.Activo=1 AND u.Bloqueado=0 AND (u.NumeroCuenta=@cd OR EXISTS(SELECT 1 FROM dbo.Tarjeta t WHERE
t.UsuarioId=u.Id AND t.Numero=@cd AND t.Activa=1));

  IF @u IS NULL BEGIN SET @codigo='14';SET @mensaje=N'La cuenta no existe en este banco.';GOTO Fin;END;

  IF EXISTS(SELECT 1 FROM dbo.BancoRed WHERE CodigoBanco IN(@propio,@bo) AND LimiteDiario-TransferidoHoy&lt;@m)

  BEGIN SET @codigo='65';SET @mensaje=N'Cupo diario bancario insuficiente.';GOTO Fin;END;

  UPDATE dbo.Usuario SET Saldo=Saldo+@m WHERE Id=@u;

  UPDATE dbo.BancoRed SET TransferidoHoy=TransferidoHoy+@m WHERE CodigoBanco IN(@propio,@bo);

  SET @datos=(SELECT @token tokenTransferencia,@propio bancoDestino,@cd cuentaDestino,@nd nombreDestino,@m
monto,LimiteDiario-TransferidoHoy cupoRestanteBancoDestino FROM dbo.BancoRed WHERE CodigoBanco=@propio FOR
JSON PATH,WITHOUT_ARRAY_WRAPPER);

  INSERT dbo.TransferenciaBancaria(TokenTransferencia,Direccion,BancoOrigen,BancoDestino,UsuarioId,CuentaOrige
n,CuentaDestino,NombreOrigen,NombreDestino,Monto,Concepto,Estado,Huella,RespuestaDatos)

  VALUES(@token,'ENTRANTE',@bo,@propio,@u,@co,@cd,@no,@nd,@m,@concepto,'APROBADA',@huella,@datos);

  INSERT dbo.Bitacora(Origen,Accion,Detalle) VALUES(@bo,'IB_RECIBIDA',@token);

  SET @mensaje=N'Transferencia interbancaria acreditada y token registrado.';GOTO Fin;

 END;

 IF @Tipo='RESERVAR'

 BEGIN

  IF @m IS NULL OR @m&lt;=0 OR @bd=@propio OR NOT EXISTS(SELECT 1 FROM dbo.BancoRed WHERE CodigoBanco=@bd AND
Activo=1 AND EsPropio=0 AND Confiable=1)

   THROW 50209,'Monto o banco destino no autorizado.',1;

  SELECT @co=COALESCE(u.NumeroCuenta,t.Numero),@no=u.Nombre FROM dbo.Usuario u WITH(UPDLOCK,ROWLOCK)

  LEFT JOIN dbo.Tarjeta t ON t.UsuarioId=u.Id AND t.Activa=1 WHERE u.Id=@u AND u.Activo=1 AND u.Bloqueado=0
AND u.Saldo&gt;=@m;

  IF @co IS NULL BEGIN SET @codigo='51';SET @mensaje=N'Usuario sin cuenta activa o saldo insuficiente.';GOTO
Fin;END;

  IF EXISTS(SELECT 1 FROM dbo.BancoRed WHERE CodigoBanco IN(@propio,@bd) AND LimiteDiario-TransferidoHoy&lt;@m)

  BEGIN SET @codigo='65';SET @mensaje=N'Cupo diario bancario insuficiente.';GOTO Fin;END;

  IF EXISTS(SELECT 1 FROM dbo.TransferenciaBancaria WHERE BancoOrigen=@propio AND TokenTransferencia=@token)

  BEGIN SET @codigo='94';SET @mensaje=N'Token ya registrado.';GOTO Fin;END;

  SET @nd=ISNULL(JSON_VALUE(@Json,'$.nombreDestino'),'');

  UPDATE dbo.Usuario SET Saldo=Saldo-@m WHERE Id=@u;

  UPDATE dbo.BancoRed SET TransferidoHoy=TransferidoHoy+@m WHERE CodigoBanco IN(@propio,@bd);

  SET @huella=HASHBYTES('SHA2_256',CONCAT(@propio,'|',@bd,'|',@co,'|',@cd,'|',@m,'|',@no,'|',@concepto));

  INSERT dbo.TransferenciaBancaria(TokenTransferencia,Direccion,BancoOrigen,BancoDestino,UsuarioId,CuentaOrige
n,CuentaDestino,NombreOrigen,NombreDestino,Monto,Concepto,Estado,Huella)

  VALUES(@token,'SALIENTE',@propio,@bd,@u,@co,@cd,@no,@nd,@m,@concepto,'RESERVADA',@huella);
```

```
  SET @datos=(SELECT 'IB_TRANSFERIR' tipo,'SERVER' cajero,REPLACE(CONVERT(varchar(36),NEWID()),'-','')
idMensaje,

   CONVERT(varchar(19),SYSDATETIME(),120) timestamp,@token tokenTransferencia,@propio bancoOrigen,@co
cuentaOrigen,@no nombreOrigen,

   @bd bancoDestino,@cd cuentaDestino,@nd nombreDestino,@m monto,@concepto concepto FOR JSON
PATH,WITHOUT_ARRAY_WRAPPER);

  INSERT dbo.Bitacora(Origen,Accion,Detalle) VALUES(CONCAT('ADMIN-',@Administrador),'IB_RESERVADA',@token);

  GOTO Fin;

 END;

 IF @Tipo='FINALIZAR'

 BEGIN

  SELECT @id=IdTransferencia,@estado=Estado,@u=UsuarioId,@m=Monto,@bo=BancoOrigen,@bd=BancoDestino FROM
dbo.TransferenciaBancaria

   WHERE TokenTransferencia=@token AND BancoOrigen=@propio AND Direccion='SALIENTE';

  IF @id IS NULL THROW 50210,'Transferencia saliente no encontrada.',1;

  IF @estado IN('APROBADA','RECHAZADA') GOTO Fin;

  DECLARE @resultado varchar(2)=JSON_VALUE(@Json,'$.codigo');

  IF @resultado='00' SET @estado='APROBADA';

  ELSE IF @resultado IN('14','65','57','30')

  BEGIN

   SET @estado='RECHAZADA';

   UPDATE dbo.Usuario SET Saldo=Saldo+@m WHERE Id=@u;

   UPDATE dbo.BancoRed SET TransferidoHoy=TransferidoHoy-@m

   WHERE CodigoBanco IN(@bo,@bd) AND FechaUltimoReinicio=CAST((SELECT FechaCreacion FROM
dbo.TransferenciaBancaria WHERE IdTransferencia=@id) AS date);

  END

  ELSE SET @estado='INCIERTA';

  UPDATE dbo.TransferenciaBancaria SET Estado=@estado,FechaActualizacion=SYSDATETIME() WHERE
IdTransferencia=@id;

  INSERT dbo.Bitacora(Origen,Accion,Detalle) VALUES(CONCAT('ADMIN-',@Administrador),'IB_'+@estado,@token);

  SET @datos=(SELECT @token tokenTransferencia,@estado estado FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

  GOTO Fin;

 END;

 SET @codigo='30';SET @mensaje=N'Operación interbancaria no soportada.';

 Fin:

 COMMIT;

 SELECT @codigo Codigo,@mensaje Mensaje,@datos Datos,CONVERT(varchar(19),SYSDATETIME(),120) Fecha;

 END TRY

 BEGIN CATCH

 IF @@TRANCOUNT&gt;0 ROLLBACK;

 THROW;

 END CATCH;

END;

GO
```

09_CompatibilidadCajero.sql

```
USE BD_CajeroRed_Fase2;

GO

-- Actualiza solo el motor. Conserva datos, inventarios e historial.

CREATE OR ALTER PROCEDURE dbo.sp_Operacion

 @Tipo varchar(40), @Json nvarchar(max), @Cajero varchar(24), @Sesion varchar(36)=NULL, @Solicitud varchar(64)

AS

BEGIN

 SET NOCOUNT ON; SET XACT_ABORT ON;

 DECLARE @codigo varchar(2)='00',@mensaje nvarchar(240)=N'Operación aprobada',@datos nvarchar(max)='{}',

 @u int,@t int,@activo bit,@bloq bit,@intentos int,@saldo decimal(18,2),@limite decimal(18,2),@retirado
decimal(18,2),

 @hash varbinary(32),@salt varchar(64),@temporal bit,@hoy date=CAST(SYSDATETIME() AS date),@fecha
datetime2=SYSDATETIME(),

 @habilitado bit,@inicializado date,@cactivo bit,@monto decimal(18,2),@efectivo decimal(18,2),@id
bigint,@token varchar(6),

 @ses varchar(36),@tokenId bigint,@huella varbinary(32)=HASHBYTES('SHA2_256',@Tipo+':'+@Json),@replay bit=0;

 DECLARE @billetes TABLE(Denominacion int PRIMARY KEY,Cantidad int NOT NULL);

 BEGIN TRY

  IF ISJSON(@Json)&lt;&gt;1 THROW 50002,'JSON inválido.',1;

  BEGIN TRAN;

  SELECT @cactivo=Activo,@habilitado=Habilitado,@inicializado=FechaInicializacion

  FROM dbo.Cajero WITH(UPDLOCK,ROWLOCK) WHERE Codigo=@Cajero;

  IF @cactivo IS NULL OR @cactivo=0 BEGIN SET @codigo='91';SET @mensaje=N'Cajero no dado de alta o
inactivo.';GOTO Terminar;END;

  IF @Tipo='REGISTRAR_CAJERO'

  BEGIN

   UPDATE dbo.Cajero SET
IP=JSON_VALUE(@Json,'$.ip'),Equipo=JSON_VALUE(@Json,'$.equipo'),Version=JSON_VALUE(@Json,'$.version'),

   Conectado=1,UltimoHeartbeat=@fecha,UltimaConexion=@fecha WHERE Codigo=@Cajero;

   INSERT dbo.Bitacora(Origen,Accion,Detalle) VALUES(@Cajero,'CONEXION',N'Cajero registrado o reconectado.');

   SET @datos=(SELECT @habilitado habilitado,CASE WHEN @inicializado=@hoy THEN CAST(1 AS bit) ELSE CAST(0 AS
bit) END inicializado FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

   GOTO Terminar;

  END;

  IF @Tipo='HEARTBEAT'

  BEGIN

   UPDATE dbo.Cajero SET UltimoHeartbeat=@fecha WHERE Codigo=@Cajero;

   SET @datos=(SELECT @habilitado habilitado,CASE WHEN @inicializado=@hoy THEN CAST(1 AS bit) ELSE CAST(0 AS
bit) END inicializado FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

   GOTO Terminar;

  END;

  IF @Tipo='LOGOUT'

  BEGIN UPDATE dbo.Sesion SET Fin=@fecha WHERE Id=@Sesion AND Cajero=@Cajero AND Fin IS NULL;GOTO
Terminar;END;

  IF @habilitado=0 OR @inicializado IS NULL OR @inicializado&lt;&gt;@hoy

  BEGIN SET @codigo='91';SET @mensaje=N'Cajero fuera de servicio o sin inicializar hoy.';GOTO Terminar;END;

  IF @Tipo IN('LOGIN','VALIDAR_TOKEN')

  BEGIN

   SELECT @t=Id,@u=UsuarioId,@hash=PinHash,@salt=Salt,@temporal=PinTemporal FROM dbo.Tarjeta
```

```
WITH(UPDLOCK,ROWLOCK)

   WHERE Numero=JSON_VALUE(@Json,'$.tarjeta');

  END

  ELSE

  BEGIN

   SELECT @u=UsuarioId,@t=TarjetaId FROM dbo.Sesion WITH(UPDLOCK,ROWLOCK)

   WHERE Id=@Sesion AND Cajero=@Cajero AND Fin IS NULL AND UltimaActividad&gt;=DATEADD(second,-120,@fecha) AND
Resultado='00';

   IF @u IS NOT NULL

   SELECT @temporal=PinTemporal,@hash=PinHash,@salt=Salt FROM dbo.Tarjeta WHERE Id=@t AND Activa=1;

   ELSE BEGIN SET @codigo='57';SET @mensaje=N'Sesión vencida. Inicie sesión nuevamente.';GOTO Terminar;END;

  END;

  SELECT @activo=Activo,@bloq=Bloqueado,@intentos=Intentos,@saldo=Saldo,@limite=LimiteDiario FROM dbo.Usuario
WITH(UPDLOCK,ROWLOCK) WHERE Id=@u;

  IF @u IS NULL BEGIN SET @codigo='55';SET @mensaje=N'Tarjeta o credenciales incorrectas.';GOTO
AccesoFallido;END;

  IF @activo=0 OR @bloq=1 OR NOT EXISTS(SELECT 1 FROM dbo.Tarjeta WHERE Id=@t AND Activa=1)

  BEGIN SET @codigo='75';SET @mensaje=N'Tarjeta bloqueada o inactiva.';GOTO Terminar;END;

  IF @Tipo='LOGIN'

  BEGIN

   IF LEN(ISNULL(JSON_VALUE(@Json,'$.pin'),''))&lt;&gt;4 OR JSON_VALUE(@Json,'$.pin') LIKE '%[^0-9]%'

    OR HASHBYTES('SHA2_256',CONVERT(varchar(200),@salt+JSON_VALUE(@Json,'$.pin')))&lt;&gt;@hash

   BEGIN SET @codigo='55';SET @mensaje=N'PIN incorrecto.';GOTO AccesoFallido;END;

   EXEC dbo.sp_GenerarToken @u,@t,@Cajero,@token OUTPUT;

   SET @datos=(SELECT @token tokenServidor FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

   GOTO Terminar;

  END;

  IF @Tipo='VALIDAR_TOKEN'

  BEGIN

   SELECT TOP(1) @tokenId=Id FROM dbo.Token WITH(UPDLOCK,ROWLOCK) WHERE UsuarioId=@u AND TarjetaId=@t AND
Cajero=@Cajero

   AND Usado=0 AND Expira&gt;@fecha AND
CodigoHash=HASHBYTES('SHA2_256',CONVERT(varchar(6),JSON_VALUE(@Json,'$.token'))) ORDER BY Id DESC;

   IF @tokenId IS NULL OR LEN(ISNULL(JSON_VALUE(@Json,'$.token'),''))&lt;&gt;6

   BEGIN SET @codigo='56';SET @mensaje=N'Token inválido, usado o expirado.';GOTO AccesoFallido;END;

   UPDATE dbo.Token SET Usado=1 WHERE Id=@tokenId;

   UPDATE dbo.Usuario SET Intentos=0 WHERE Id=@u;

   UPDATE dbo.Sesion SET Fin=@fecha WHERE UsuarioId=@u AND Cajero=@Cajero AND Fin IS NULL;

   SET @ses=CONVERT(varchar(36),NEWID());

   INSERT dbo.Sesion(Id,UsuarioId,TarjetaId,Cajero,Resultado) VALUES(@ses,@u,@t,@Cajero,'00');

   INSERT dbo.Bitacora(Origen,Accion,Detalle) VALUES(@Cajero,'ACCESO',CONCAT('Usuario ',@u));

   SET @datos=(SELECT @ses sesion,@u usuarioId,@temporal debeCambiarPin,(SELECT Nombre FROM dbo.Usuario WHERE
Id=@u) nombre FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

   GOTO Terminar;

  END;

  UPDATE dbo.Sesion SET UltimaActividad=@fecha WHERE Id=@Sesion;

  IF @Tipo='ESTADO_SOLICITUD'

  BEGIN

   SELECT @datos=(SELECT Solicitud idOriginal,UsuarioId usuarioId,Codigo codigo,Mensaje
mensaje,JSON_QUERY(Datos) datos FROM dbo.SolicitudOperacion
```

```
    WHERE Cajero=@Cajero AND UsuarioId=@u AND Solicitud=JSON_VALUE(@Json,'$.idOriginal') FOR JSON
PATH,WITHOUT_ARRAY_WRAPPER);

   IF NULLIF(@datos,'') IS NULL BEGIN SET @codigo='25';SET @mensaje=N'No existe resultado confirmado. Consulte
 al administrador antes de repetir.';SET @datos='{}';END;

   GOTO Terminar;

  END;

  IF @Tipo IN('RETIRO','DEPOSITO','CAMBIO_PIN') AND EXISTS(SELECT 1 FROM dbo.SolicitudOperacion WHERE
Cajero=@Cajero AND Solicitud=@Solicitud)

  BEGIN

   IF NOT EXISTS(SELECT 1 FROM dbo.SolicitudOperacion WHERE Cajero=@Cajero AND Solicitud=@Solicitud AND
UsuarioId=@u AND Tipo=@Tipo AND Huella=@huella)

   BEGIN SET @codigo='94';SET @mensaje=N'Identificador reutilizado con datos distintos.';GOTO Terminar;END;

   SELECT @codigo=Codigo,@mensaje=Mensaje,@datos=Datos FROM dbo.SolicitudOperacion WHERE Cajero=@Cajero AND
Solicitud=@Solicitud;

   SET @replay=1;GOTO Terminar;

  END;

  IF @temporal=1 AND @Tipo NOT IN('CAMBIO_PIN')

  BEGIN SET @codigo='58';SET @mensaje=N'Debe cambiar su PIN temporal antes de operar.';GOTO Terminar;END;

  SELECT @retirado=ISNULL(SUM(Monto),0) FROM dbo.Transaccion WHERE UsuarioId=@u AND Tipo='RETIRO' AND
CodigoRespuesta='00' AND Fecha&gt;=@hoy AND Fecha&lt;DATEADD(day,1,@hoy);

  IF @Tipo='CONSULTA_SALDO'

  BEGIN SET @datos=(SELECT @fecha fecha,@saldo saldo,@limite limiteDiario,CASE WHEN @limite&gt;@retirado THEN
@limite-@retirado ELSE 0 END disponibleHoy,@retirado retiradoHoy FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);GOTO
Terminar;END;

  IF @Tipo='ULTIMAS_TRANSACCIONES'

  BEGIN SET @datos=(SELECT TOP(5) Fecha fecha,Tipo tipo,Monto monto,Cajero cajero FROM dbo.Transaccion WHERE
UsuarioId=@u AND CodigoRespuesta='00' ORDER BY Fecha DESC,Id DESC FOR JSON PATH);GOTO Terminar;END;

  IF @Tipo='CAMBIO_PIN'

  BEGIN

   DECLARE @actual varchar(50)=JSON_VALUE(@Json,'$.pinActual'),@nuevo
varchar(50)=JSON_VALUE(@Json,'$.pinNuevo');

   IF LEN(ISNULL(@actual,''))&lt;&gt;4 OR HASHBYTES('SHA2_256',CONVERT(varchar(200),@salt+@actual))&lt;&gt;@hash

   BEGIN SET @codigo='55';SET @mensaje=N'PIN actual incorrecto.';

    INSERT dbo.CambioPin(UsuarioId,Cajero,Origen,Resultado) VALUES(@u,@Cajero,'USUARIO',@mensaje);GOTO
AccesoFallido;END;

   IF LEN(ISNULL(@nuevo,''))&lt;&gt;4 OR @nuevo LIKE '%[^0-9]%' OR @actual=@nuevo

   BEGIN SET @codigo='30';SET @mensaje=N'El nuevo PIN debe tener 4 dígitos y ser distinto al actual.';GOTO
Terminar;END;

   SET @salt=CONVERT(varchar(64),CRYPT_GEN_RANDOM(32),2);

   UPDATE dbo.Tarjeta SET
Salt=@salt,PinHash=HASHBYTES('SHA2_256',CONVERT(varchar(200),@salt+@nuevo)),PinTemporal=0 WHERE Id=@t;

   INSERT dbo.CambioPin(UsuarioId,Cajero,Origen,Resultado) VALUES(@u,@Cajero,'USUARIO','00');

   UPDATE dbo.Token SET Usado=1 WHERE UsuarioId=@u;

   UPDATE dbo.Sesion SET Fin=@fecha WHERE UsuarioId=@u AND Id&lt;&gt;@Sesion AND Fin IS NULL;

   SET @datos=(SELECT @fecha fecha FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

   SET @mensaje=N'PIN actualizado en toda la red.';GOTO GuardarResultado;

  END;

  IF @Tipo NOT IN('RETIRO','DEPOSITO') BEGIN SET @codigo='30';SET @mensaje=N'Operación desconocida.';GOTO
Terminar;END;

  SET @monto=TRY_CONVERT(decimal(18,2),JSON_VALUE(@Json,'$.monto'));

  IF @monto IS NULL OR @monto&lt;=0 OR @monto&gt;30000 OR @monto&lt;&gt;FLOOR(@monto)

  BEGIN SET @codigo='30';SET @mensaje=N'Monto inválido; use quetzales enteros entre 1 y 30000.';GOTO
Terminar;END;

  IF JSON_QUERY(@Json,'$.desglose') IS NULL BEGIN SET @codigo='30';SET @mensaje=N'Falta desglose.';GOTO
```

```
Terminar;END;

  IF EXISTS(SELECT 1 FROM OPENJSON(@Json,'$.desglose') WHERE TRY_CONVERT(int,[key]) NOT
IN(200,100,50,20,10,5,1)

   OR TRY_CONVERT(int,[key]) IS NULL OR TRY_CONVERT(int,value) IS NULL OR TRY_CONVERT(int,value)&lt;0 OR
TRY_CONVERT(int,value)&gt;30000 OR type&lt;&gt;2)

   OR EXISTS(SELECT [key] FROM OPENJSON(@Json,'$.desglose') GROUP BY [key] HAVING COUNT(*)&gt;1)

  BEGIN SET @codigo='30';SET @mensaje=N'Denominaciones o cantidades inválidas.';GOTO Terminar;END;

  INSERT @billetes SELECT CONVERT(int,[key]),CONVERT(int,value) FROM OPENJSON(@Json,'$.desglose');

  IF ISNULL((SELECT SUM(CONVERT(bigint,Denominacion)*Cantidad) FROM @billetes),0)&lt;&gt;@monto

  BEGIN SET @codigo='30';SET @mensaje=N'El desglose no coincide con el monto.';GOTO Terminar;END;

  SELECT @efectivo=SUM(CONVERT(bigint,Denominacion)*Cantidad) FROM dbo.InventarioCajero WITH(UPDLOCK,ROWLOCK)
WHERE Cajero=@Cajero;

  IF @Tipo='RETIRO'

  BEGIN

   IF @saldo&lt;@monto BEGIN SET @codigo='51';SET @mensaje=N'Saldo insuficiente.';GOTO RegistrarMovimiento;END;

   IF @retirado+@monto&gt;@limite BEGIN SET @codigo='61';SET @mensaje=N'Excede el límite diario global
disponible.';GOTO RegistrarMovimiento;END;

   IF EXISTS(SELECT 1 FROM @billetes b LEFT JOIN dbo.InventarioCajero i ON i.Cajero=@Cajero AND
i.Denominacion=b.Denominacion WHERE b.Cantidad&gt;ISNULL(i.Cantidad,0))

   BEGIN

    SET @codigo='92';SET @mensaje=N'No hay billetes para ese desglose.';

    SET @datos=(SELECT JSON_QUERY((SELECT Denominacion denominacion,Cantidad cantidad FROM
dbo.InventarioCajero WHERE Cajero=@Cajero FOR JSON PATH)) inventario FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

    INSERT dbo.Alerta(Tipo,Cajero,Descripcion) VALUES('RETIRO_SIN_BILLETES',@Cajero,N'Retiro rechazado por
falta de billetes.');

    GOTO RegistrarMovimiento;

   END;

  END

  ELSE IF @efectivo+@monto&gt;(SELECT Valor FROM dbo.Parametro WHERE Clave='MaxEfectivo')

   BEGIN SET @codigo='93';SET @mensaje=N'El depósito supera la capacidad del cajero.';GOTO
RegistrarMovimiento;END;

  UPDATE dbo.Usuario SET Saldo=Saldo+CASE WHEN @Tipo='RETIRO' THEN -@monto ELSE @monto END WHERE Id=@u;

  UPDATE i SET Cantidad=i.Cantidad+CASE WHEN @Tipo='RETIRO' THEN -b.Cantidad ELSE b.Cantidad END

  FROM dbo.InventarioCajero i JOIN @billetes b ON i.Denominacion=b.Denominacion WHERE i.Cajero=@Cajero;

  SET @saldo=@saldo+CASE WHEN @Tipo='RETIRO' THEN -@monto ELSE @monto END;

  SET @retirado=@retirado+CASE WHEN @Tipo='RETIRO' THEN @monto ELSE 0 END;

  RegistrarMovimiento:

  INSERT dbo.Transaccion(UsuarioId,TarjetaId,Cajero,Solicitud,Tipo,Monto,CodigoRespuesta,Motivo)
VALUES(@u,@t,@Cajero,@Solicitud,@Tipo,@monto,@codigo,@mensaje);

  SET @id=SCOPE_IDENTITY();

  INSERT dbo.TransaccionDetalle SELECT @id,Denominacion,Cantidad FROM @billetes;

  IF @codigo='00'

  SET @datos=(SELECT @id idTransaccion,@saldo saldo,CASE WHEN @limite&gt;@retirado THEN @limite-@retirado ELSE 0
END disponibleHoy,@fecha fecha,@monto monto,@Tipo tipo,@Cajero cajero FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

  EXEC dbo.sp_Alertas @Cajero;

  GOTO GuardarResultado;

  AccesoFallido:

  INSERT dbo.Sesion(UsuarioId,TarjetaId,Cajero,Fin,Resultado) VALUES(@u,@t,@Cajero,@fecha,@codigo);

  IF @u IS NOT NULL

  BEGIN

   UPDATE dbo.Usuario SET Intentos=CASE WHEN Intentos&lt;3 THEN Intentos+1 ELSE 3 END,Bloqueado=CASE WHEN
Intentos&gt;=2 THEN 1 ELSE Bloqueado END WHERE Id=@u;
```

```
   IF @intentos&gt;=2

   BEGIN

    SET @codigo='75';SET @mensaje=N'Tarjeta bloqueada por tres intentos fallidos.';

    INSERT dbo.Alerta(Tipo,Cajero,Descripcion) VALUES('TARJETA_BLOQUEADA',@Cajero,CONCAT('Usuario ',@u,'
bloqueado.'));

    UPDATE dbo.Sesion SET Fin=@fecha WHERE UsuarioId=@u AND Fin IS NULL;

   END;

  END;

  GOTO Terminar;

  GuardarResultado:

  INSERT dbo.SolicitudOperacion(Cajero,Solicitud,UsuarioId,Tipo,Huella,Codigo,Mensaje,Datos)
VALUES(@Cajero,@Solicitud,@u,@Tipo,@huella,@codigo,@mensaje,@datos);

  Terminar:

  IF @codigo='00' AND @datos='{}' SET @datos=(SELECT @fecha fecha FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

  IF @Tipo NOT IN('HEARTBEAT') AND @replay=0

   INSERT dbo.Bitacora(Origen,Accion,Detalle) VALUES(@Cajero,@Tipo,CONCAT('Usuario ',@u,'; resultado
',@codigo,'; solicitud ',@Solicitud));

  COMMIT;

  SELECT @codigo Codigo,@mensaje Mensaje,ISNULL(@datos,'{}') Datos;

 END TRY

 BEGIN CATCH

  IF XACT_STATE()&lt;&gt;0 ROLLBACK;

  -- El servidor no debe anunciar éxito si SQL no confirmó. Sin datos sensibles en errores.

  SELECT '96' Codigo,N'Error de base de datos; consulte el estado de la solicitud antes de repetir.'
Mensaje,'{}' Datos;

 END CATCH;

END;

GO
```

10_TransferenciasCajero.sql

```
USE BD_CajeroRed_Fase2;

GO

IF OBJECT_ID('dbo.TransferenciaCajero') IS NULL

CREATE TABLE dbo.TransferenciaCajero(

 Id bigint IDENTITY PRIMARY KEY,Cajero varchar(24) NOT NULL REFERENCES dbo.Cajero(Codigo),

 Solicitud varchar(64) NOT NULL,UsuarioId int NOT NULL REFERENCES dbo.Usuario(Id),UsuarioDestino int NULL
REFERENCES dbo.Usuario(Id),

 SesionOrigen varchar(36) NOT NULL,BancoOrigen varchar(15) NOT NULL REFERENCES dbo.BancoRed(CodigoBanco),

 BancoDestino varchar(15) NOT NULL REFERENCES dbo.BancoRed(CodigoBanco),

 CuentaOrigen varchar(20) NOT NULL,CuentaDestino varchar(20) NOT NULL,NombreOrigen nvarchar(120) NOT
NULL,NombreDestino nvarchar(120) NOT NULL,

 Tipo varchar(10) NOT NULL CHECK(Tipo IN('INTERNA','EXTERNA')),Monto decimal(18,2) NOT NULL
CHECK(Monto&gt;0),Concepto nvarchar(200) NOT NULL,

 TokenTransferencia varchar(40) NOT NULL UNIQUE,Huella varbinary(32) NOT NULL,

 Estado varchar(20) NOT NULL CHECK(Estado IN('PENDIENTE','RESERVADA','INCIERTA','APROBADA','RECHAZADA')),

 Codigo varchar(2) NOT NULL DEFAULT '96',Mensaje nvarchar(240) NOT NULL DEFAULT N'En proceso.',

 Datos nvarchar(max) NOT NULL DEFAULT '{}',SaldoPosterior decimal(18,2) NULL,

 Fecha datetime2 NOT NULL DEFAULT SYSDATETIME(),UNIQUE(Cajero,Solicitud));

GO

CREATE OR ALTER FUNCTION dbo.fn_CuentaActiva(@Cuenta varchar(20))

RETURNS TABLE AS RETURN

 SELECT u.Id,u.Nombre FROM dbo.Usuario u

 WHERE u.Activo=1 AND u.Bloqueado=0 AND

 (u.NumeroCuenta=@Cuenta OR EXISTS(SELECT 1 FROM dbo.Tarjeta t WHERE t.UsuarioId=u.Id AND t.Numero=@Cuenta AND
 t.Activa=1));

GO

CREATE OR ALTER VIEW dbo.vw_TransferenciasUsuario AS

SELECT t.IdTransferencia,t.UsuarioId,t.FechaCreacion
Fecha,t.TokenTransferencia,t.Direccion,t.BancoOrigen,t.BancoDestino,

 '**** '+RIGHT(t.CuentaOrigen,4) CuentaOrigen,'**** '+RIGHT(t.CuentaDestino,4) CuentaDestino,

 t.NombreOrigen,t.NombreDestino,t.Monto,t.Estado,t.Concepto

FROM dbo.TransferenciaBancaria t

UNION ALL

SELECT -r.Id*2,r.UsuarioId,r.Fecha,r.TokenTransferencia,CAST('SALIENTE' AS
varchar(10)),r.BancoOrigen,r.BancoDestino,

 '**** '+RIGHT(r.CuentaOrigen,4),'****
'+RIGHT(r.CuentaDestino,4),r.NombreOrigen,r.NombreDestino,r.Monto,r.Estado,r.Concepto

FROM dbo.TransferenciaCajero r WHERE r.Tipo='INTERNA' OR NOT EXISTS(

 SELECT 1 FROM dbo.TransferenciaBancaria t WHERE t.BancoOrigen=r.BancoOrigen AND
t.TokenTransferencia=r.TokenTransferencia)

UNION ALL

SELECT -r.Id*2-1,r.UsuarioDestino,r.Fecha,r.TokenTransferencia,CAST('ENTRANTE' AS
varchar(10)),r.BancoOrigen,r.BancoDestino,

 '**** '+RIGHT(r.CuentaOrigen,4),'****
'+RIGHT(r.CuentaDestino,4),r.NombreOrigen,r.NombreDestino,r.Monto,r.Estado,r.Concepto

FROM dbo.TransferenciaCajero r WHERE r.Tipo='INTERNA' AND r.Estado='APROBADA';

GO

CREATE OR ALTER PROCEDURE dbo.sp_TransferenciaCajero

 @Tipo varchar(30),@Cajero varchar(24),@Sesion varchar(36),@Solicitud varchar(64),@Json nvarchar(max)

AS
```

```
BEGIN

 SET NOCOUNT ON; SET XACT_ABORT ON;

 DECLARE @codigo varchar(2)='00',@mensaje nvarchar(240)=N'Operación aprobada.',@datos
nvarchar(max)='{}',@accion varchar(30)='',@ruta nvarchar(max)='{}',

 @u int,@tarjeta int,@propio varchar(15),@banco varchar(15)=UPPER(JSON_VALUE(@Json,'$.bancoDestino')),

 @cuenta varchar(20)=JSON_VALUE(@Json,'$.cuentaDestino'),@m
decimal(18,2)=TRY_CONVERT(decimal(18,2),JSON_VALUE(@Json,'$.monto')),

 @concepto nvarchar(200)=ISNULL(JSON_VALUE(@Json,'$.concepto'),''),@esperado
nvarchar(120)=JSON_VALUE(@Json,'$.nombreDestino'),

 @id bigint,@token varchar(40),@estado varchar(20),@tipoTransferencia varchar(10),@origenCuenta
varchar(20),@no nvarchar(120),@nd nvarchar(120),

 @dest int,@cantidad int,@saldo decimal(18,2),@huella varbinary(32),@guardada varbinary(32),@lock int,@dummy
int,

 @ip varchar(50),@puerto int,@hoy date,@fecha datetime2;

 BEGIN TRY

 BEGIN TRAN;

 EXEC @lock=sys.sp_getapplock
@Resource='CajeroRed_Interbancario',@LockMode='Exclusive',@LockOwner='Transaction',@LockTimeout=2000;

 IF @lock&lt;0 THROW 50400,'Operaciones bancarias ocupadas.',1;

 SET @fecha=SYSDATETIME();SET @hoy=CAST(@fecha AS date);

 -- FINALIZAR puede terminar una operación autorizada aunque su sesión se cerrara mientras esperaba al banco.

 IF @Tipo='FINALIZAR'

 BEGIN

  SELECT @id=Id,@u=UsuarioId FROM dbo.TransferenciaCajero WHERE Cajero=@Cajero AND Solicitud=@Solicitud

   AND (SesionOrigen=@Sesion OR EXISTS(SELECT 1 FROM dbo.Sesion s WHERE s.Id=@Sesion AND s.Cajero=@Cajero AND
s.UsuarioId=TransferenciaCajero.UsuarioId AND s.Fin IS NULL AND
s.UltimaActividad&gt;=DATEADD(second,-120,@fecha)));

  IF @id IS NULL BEGIN SET @codigo='57';SET @mensaje=N'Operación no autorizada.';GOTO Fin;END;

  GOTO Finalizar;

 END;

 SELECT @u=s.UsuarioId,@tarjeta=s.TarjetaId FROM dbo.Sesion s

 JOIN dbo.Cajero c ON c.Codigo=s.Cajero

 JOIN dbo.Tarjeta t ON t.Id=s.TarjetaId AND t.Activa=1

 JOIN dbo.Usuario u ON u.Id=s.UsuarioId AND u.Activo=1 AND u.Bloqueado=0

 WHERE s.Id=@Sesion AND s.Cajero=@Cajero AND s.Fin IS NULL AND s.Resultado='00'

  AND s.UltimaActividad&gt;=DATEADD(second,-120,@fecha) AND c.Activo=1 AND c.Habilitado=1 AND
c.FechaInicializacion=@hoy;

 IF @u IS NULL BEGIN SET @codigo='57';SET @mensaje=N'Sesión vencida, tarjeta inactiva o cajero fuera de
servicio.';GOTO Fin;END;

 IF EXISTS(SELECT 1 FROM dbo.Tarjeta WHERE Id=@tarjeta AND PinTemporal=1)

 BEGIN SET @codigo='58';SET @mensaje=N'Cambie el PIN temporal antes de transferir.';GOTO Fin;END;

 UPDATE dbo.Sesion SET UltimaActividad=@fecha WHERE Id=@Sesion;

 SELECT @propio=CodigoBanco FROM dbo.BancoRed WHERE EsPropio=1 AND Activo=1;

 IF @propio IS NULL BEGIN SET @codigo='91';SET @mensaje=N'El administrador debe configurar NETGEAR.';GOTO
Fin;END;

 IF @Tipo='BANCOS'

 BEGIN

  SET @datos=ISNULL((SELECT CodigoBanco codigoBanco,NombreBanco nombreBanco,EsPropio esPropio FROM
dbo.BancoRed

   WHERE Activo=1 AND (EsPropio=1 OR Confiable=1) ORDER BY EsPropio DESC,NombreBanco FOR JSON PATH),'[]');GOTO
 Fin;

 END;

 IF @Tipo='HISTORIAL'
```

```
 BEGIN

  SET @datos=ISNULL((SELECT TOP(30) Fecha fecha,TokenTransferencia tokenTransferencia,Direccion
direccion,BancoOrigen bancoOrigen,BancoDestino bancoDestino,

   CuentaOrigen cuentaOrigen,CuentaDestino cuentaDestino,NombreOrigen nombreOrigen,NombreDestino
nombreDestino,Monto monto,Estado estado,Concepto concepto

   FROM dbo.vw_TransferenciasUsuario WHERE UsuarioId=@u ORDER BY Fecha DESC,IdTransferencia DESC FOR JSON
PATH),'[]');GOTO Fin;

 END;

 IF @Tipo='ESTADO'

 BEGIN

  SELECT @id=Id FROM dbo.TransferenciaCajero WHERE Cajero=@Cajero AND
Solicitud=JSON_VALUE(@Json,'$.idOriginal') AND UsuarioId=@u;

  IF @id IS NULL BEGIN SET @codigo='25';SET @mensaje=N'No hay resultado confirmado de ese usuario. No repita
el envío.';GOTO Fin;END;

  GOTO Leer;

 END;

 IF @Tipo IN('PREPARAR','DESTINO')

 BEGIN

  IF @banco IS NULL OR @cuenta IS NULL OR LEN(JSON_VALUE(@Json,'$.cuentaDestino')) NOT BETWEEN 1 AND 20 OR
@cuenta LIKE '%[^0-9]%'

  BEGIN SET @codigo='30';SET @mensaje=N'Banco o cuenta inválidos.';GOTO Fin;END;

  SELECT @ip=IP,@puerto=Puerto FROM dbo.BancoRed WHERE CodigoBanco=@banco AND Activo=1 AND (EsPropio=1 OR
Confiable=1);

  IF @ip IS NULL BEGIN SET @codigo='14';SET @mensaje=N'Banco no disponible o sin autorización del
administrador.';GOTO Fin;END;

 END;

 IF @Tipo='DESTINO'

 BEGIN

  IF @banco=@propio

  BEGIN

   SELECT @cantidad=COUNT(*),@dest=MAX(Id),@nd=MAX(Nombre) FROM dbo.fn_CuentaActiva(@cuenta);

   IF @cantidad&lt;&gt;1 BEGIN SET @codigo='14';SET @mensaje=N'Cuenta inexistente, bloqueada o ambigua.';GOTO
Fin;END;

   IF @dest=@u BEGIN SET @codigo='30';SET @mensaje=N'Elija la cuenta de otra persona.';GOTO Fin;END;

   SET @datos=(SELECT CAST(1 AS bit) existe,@banco codigoBanco,@nd nombreTitular,'**** '+RIGHT(@cuenta,4)
cuentaDestino FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

  END

  ELSE

  BEGIN

   SET @accion='VERIFICAR';

   SET @ruta=(SELECT @ip ip,@puerto puerto,@propio bancoOrigen,@banco bancoDestino,@cuenta numeroCuenta FOR
JSON PATH,WITHOUT_ARRAY_WRAPPER);

  END;

  GOTO Fin;

 END;

 IF @Tipo='PREPARAR'

 BEGIN

  IF @m IS NULL OR @m&lt;=0 OR TRY_CONVERT(decimal(38,18),JSON_VALUE(@Json,'$.monto'))&lt;&gt;@m

   OR LEN(ISNULL(JSON_VALUE(@Json,'$.concepto'),''))&gt;200 OR
LEN(ISNULL(JSON_VALUE(@Json,'$.nombreDestino'),'')) NOT BETWEEN 1 AND 120

  BEGIN SET @codigo='30';SET @mensaje=N'Monto o concepto inválidos.';GOTO Fin;END;

  SET @huella=HASHBYTES('SHA2_256',CONCAT(@banco,'|',@cuenta,'|',@m,'|',@concepto,'|',@esperado));
```

```
  SELECT @id=Id,@guardada=Huella FROM dbo.TransferenciaCajero WHERE Cajero=@Cajero AND Solicitud=@Solicitud;

  IF @id IS NOT NULL

  BEGIN

   IF NOT EXISTS(SELECT 1 FROM dbo.TransferenciaCajero WHERE Id=@id AND UsuarioId=@u AND Huella=@huella)

   BEGIN SET @codigo='94';SET @mensaje=N'Referencia ya utilizada con datos distintos.';GOTO Fin;END;

   GOTO Leer;

  END;

  SELECT @saldo=u.Saldo,@origenCuenta=COALESCE(u.NumeroCuenta,t.Numero),@no=u.Nombre FROM dbo.Usuario u JOIN
dbo.Tarjeta t ON t.Id=@tarjeta WHERE u.Id=@u;

  IF @saldo&lt;@m BEGIN SET @codigo='51';SET @mensaje=N'Saldo insuficiente.';GOTO Fin;END;

  IF @banco=@propio

  BEGIN

   SELECT @cantidad=COUNT(*),@dest=MAX(Id),@nd=MAX(Nombre) FROM dbo.fn_CuentaActiva(@cuenta);

   IF @cantidad&lt;&gt;1 BEGIN SET @codigo='14';SET @mensaje=N'Cuenta inexistente, bloqueada o ambigua.';GOTO
Fin;END;

   IF @dest=@u BEGIN SET @codigo='30';SET @mensaje=N'No se puede transferir a la misma cuenta.';GOTO Fin;END;

   IF @nd&lt;&gt;@esperado BEGIN SET @codigo='30';SET @mensaje=N'El destinatario cambió. Verifique nuevamente.';GOTO
 Fin;END;

   -- Orden consistente, sin esperar a la red mientras se bloquean saldos.

   SELECT @dummy=Id FROM dbo.Usuario WITH(UPDLOCK,ROWLOCK) WHERE Id IN(@u,@dest) ORDER BY Id;

   IF NOT EXISTS(SELECT 1 FROM dbo.Usuario WHERE Id=@u AND Saldo&gt;=@m AND Activo=1 AND Bloqueado=0)

   BEGIN SET @codigo='51';SET @mensaje=N'Saldo insuficiente.';GOTO Fin;END;

  END;

  WHILE 1=1

  BEGIN

   SET @token='TRF-'+CONVERT(char(8),@fecha,112)+'-
'+RIGHT('000000'+CONVERT(varchar(6),CONVERT(bigint,CRYPT_GEN_RANDOM(4))%1000000),6);

   IF NOT EXISTS(SELECT 1 FROM dbo.TransferenciaCajero WHERE TokenTransferencia=@token)

    AND NOT EXISTS(SELECT 1 FROM dbo.TransferenciaBancaria WHERE BancoOrigen=@propio AND
TokenTransferencia=@token) BREAK;

  END;

  INSERT dbo.TransferenciaCajero(Cajero,Solicitud,UsuarioId,UsuarioDestino,SesionOrigen,BancoOrigen,BancoDesti
no,CuentaOrigen,CuentaDestino,

   NombreOrigen,NombreDestino,Tipo,Monto,Concepto,TokenTransferencia,Huella,Estado)

  VALUES(@Cajero,@Solicitud,@u,@dest,@Sesion,@propio,@banco,@origenCuenta,@cuenta,@no,COALESCE(@nd,@esperado),

   CASE WHEN @banco=@propio THEN 'INTERNA' ELSE 'EXTERNA' END,@m,@concepto,@token,@huella,'PENDIENTE');

  SET @id=SCOPE_IDENTITY();

  IF @banco=@propio

  BEGIN

   UPDATE dbo.Usuario SET Saldo=Saldo-@m WHERE Id=@u;

   UPDATE dbo.Usuario SET Saldo=Saldo+@m WHERE Id=@dest;

   UPDATE dbo.TransferenciaCajero SET Estado='APROBADA',Codigo='00',Mensaje=N'Transferencia NETGEAR
aprobada.',

    SaldoPosterior=(SELECT Saldo FROM dbo.Usuario WHERE Id=@u) WHERE Id=@id;

   INSERT dbo.Bitacora(Origen,Accion,Detalle) VALUES(@Cajero,'TRANSFERENCIA_INTERNA',@token);

   GOTO Recibo;

  END;

  SET @accion='VERIFICAR_ENVIO';

  SET @ruta=(SELECT @ip ip,@puerto puerto,@propio bancoOrigen,@banco bancoDestino,@cuenta
numeroCuenta,@esperado nombreDestino FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);
```

```
  SET @codigo='96';SET @mensaje=N'Transferencia en validación.';GOTO Fin;

 END;

 IF @Tipo='RESERVAR'

 BEGIN

  SELECT @id=Id,@token=TokenTransferencia,@banco=BancoDestino,@m=Monto FROM dbo.TransferenciaCajero WHERE
Cajero=@Cajero AND Solicitud=@Solicitud AND UsuarioId=@u;

  IF @id IS NULL BEGIN SET @codigo='25';SET @mensaje=N'Solicitud no encontrada.';GOTO Fin;END;

  IF NOT EXISTS(SELECT 1 FROM dbo.TransferenciaCajero WHERE Id=@id AND Estado='PENDIENTE') GOTO Leer;

  SELECT @ip=IP,@puerto=Puerto FROM dbo.BancoRed WHERE CodigoBanco=@banco AND Activo=1 AND Confiable=1 AND
EsPropio=0;

  IF @ip IS NULL BEGIN SET @codigo='57';SET @mensaje=N'Banco destino no autorizado.';GOTO RechazarPrevio;END;

  UPDATE dbo.BancoRed SET TransferidoHoy=0,FechaUltimoReinicio=@hoy WHERE FechaUltimoReinicio&lt;&gt;@hoy;

  SELECT @saldo=Saldo FROM dbo.Usuario WITH(UPDLOCK,ROWLOCK) WHERE Id=@u;

  IF @saldo&lt;@m BEGIN SET @codigo='51';SET @mensaje=N'Saldo insuficiente.';GOTO RechazarPrevio;END;

  IF EXISTS(SELECT 1 FROM dbo.BancoRed WHERE CodigoBanco IN(@propio,@banco) AND LimiteDiario-
TransferidoHoy&lt;@m)

  BEGIN SET @codigo='65';SET @mensaje=N'Cupo diario bancario insuficiente.';GOTO RechazarPrevio;END;

  UPDATE dbo.Usuario SET Saldo=Saldo-@m WHERE Id=@u;

  UPDATE dbo.BancoRed SET TransferidoHoy=TransferidoHoy+@m WHERE CodigoBanco IN(@propio,@banco);

  INSERT dbo.TransferenciaBancaria(TokenTransferencia,Direccion,BancoOrigen,BancoDestino,UsuarioId,CuentaOrige
n,CuentaDestino,NombreOrigen,NombreDestino,Monto,Concepto,Estado,Huella)

  SELECT TokenTransferencia,'SALIENTE',BancoOrigen,BancoDestino,UsuarioId,CuentaOrigen,CuentaDestino,NombreOri
gen,NombreDestino,Monto,Concepto,'RESERVADA',Huella FROM dbo.TransferenciaCajero WHERE Id=@id;

  UPDATE dbo.TransferenciaCajero SET Estado='RESERVADA',SaldoPosterior=(SELECT Saldo FROM dbo.Usuario WHERE
Id=@u) WHERE Id=@id;

  SET @accion='ENVIAR';

  SET @ruta=(SELECT @ip ip,@puerto puerto,'IB_TRANSFERIR' tipo,'SERVER'
cajero,REPLACE(CONVERT(varchar(36),NEWID()),'-','') idMensaje,

    CONVERT(varchar(19),@fecha,120) timestamp,TokenTransferencia tokenTransferencia,BancoOrigen
bancoOrigen,BancoDestino bancoDestino,

    CuentaOrigen cuentaOrigen,CuentaDestino cuentaDestino,NombreOrigen nombreOrigen,NombreDestino
nombreDestino,Monto monto,Concepto concepto FROM dbo.TransferenciaCajero WHERE Id=@id FOR JSON
PATH,WITHOUT_ARRAY_WRAPPER);

  SET @codigo='96';SET @mensaje=N'Enviando transferencia.';GOTO Fin;

 END;

 SET @codigo='30';SET @mensaje=N'Operación no reconocida.';GOTO Fin;

 RechazarPrevio:

 UPDATE dbo.TransferenciaCajero SET Estado='RECHAZADA',Codigo=@codigo,Mensaje=@mensaje,SaldoPosterior=(SELECT
Saldo FROM dbo.Usuario WHERE Id=@u) WHERE Id=@id;

 GOTO Recibo;

 Finalizar:

 SELECT @token=TokenTransferencia,@estado=Estado,@m=Monto,@propio=BancoOrigen,@banco=BancoDestino FROM
dbo.TransferenciaCajero WHERE Id=@id;

 IF EXISTS(SELECT 1 FROM dbo.TransferenciaBancaria WHERE BancoOrigen=@propio AND TokenTransferencia=@token AND
 Estado='APROBADA')

 BEGIN

  UPDATE dbo.TransferenciaCajero SET Estado='APROBADA',Codigo='00',Mensaje=N'Transferencia confirmada por el
banco destino.' WHERE Id=@id;

  SET @estado='APROBADA';

 END;

 IF @estado IN('APROBADA','RECHAZADA') GOTO Recibo;

 SET @codigo=ISNULL(JSON_VALUE(@Json,'$.codigo'),'96');

 SET @mensaje=LEFT(COALESCE(JSON_VALUE(@Json,'$.mensaje'),N'Transferencia sin confirmar.'),240);
```

```
 IF EXISTS(SELECT 1 FROM dbo.TransferenciaBancaria WHERE BancoOrigen=@propio AND TokenTransferencia=@token)

 BEGIN

  IF @codigo='00' SET @estado='APROBADA';

  ELSE IF @codigo IN('14','65','57','30')

  BEGIN

   SET @estado='RECHAZADA';

   IF EXISTS(SELECT 1 FROM dbo.TransferenciaBancaria WHERE BancoOrigen=@propio AND TokenTransferencia=@token
AND Estado IN('RESERVADA','INCIERTA'))

   BEGIN

    UPDATE dbo.Usuario SET Saldo=Saldo+@m WHERE Id=@u;

    UPDATE dbo.BancoRed SET TransferidoHoy=TransferidoHoy-@m WHERE CodigoBanco IN(@propio,@banco)

     AND FechaUltimoReinicio=CAST((SELECT Fecha FROM dbo.TransferenciaCajero WHERE Id=@id) AS date);

   END;

  END

  ELSE BEGIN SET @estado='INCIERTA';SET @codigo='96';END;

  UPDATE dbo.TransferenciaBancaria SET Estado=@estado,FechaActualizacion=@fecha WHERE BancoOrigen=@propio AND
TokenTransferencia=@token;

 END

 ELSE

 BEGIN

  -- Nunca se envió IB_TRANSFERIR: se puede rechazar sin devolver fondos inexistentes.

  SET @estado='RECHAZADA';IF @codigo IN('00','96')SET @codigo='91';

 END;

 UPDATE dbo.TransferenciaCajero SET Estado=@estado,Codigo=@codigo,Mensaje=@mensaje WHERE Id=@id;

 INSERT dbo.Bitacora(Origen,Accion,Detalle) VALUES(@Cajero,'TRANSFERENCIA_'+@estado,@token);

 GOTO Recibo;

 Leer:

 SELECT @estado=r.Estado,@token=r.TokenTransferencia,@propio=r.BancoOrigen,@banco=r.BancoDestino,@m=r.Monto
FROM dbo.TransferenciaCajero r WHERE Id=@id;

 IF @estado='PENDIENTE' AND EXISTS(SELECT 1 FROM dbo.TransferenciaCajero WHERE Id=@id AND
Fecha&lt;DATEADD(second,-40,@fecha))

 BEGIN

  UPDATE dbo.TransferenciaCajero SET Estado='RECHAZADA',Codigo='91',Mensaje=N'Cancelada antes del envío. No se
 cargó dinero.' WHERE Id=@id;

 END;

 IF EXISTS(SELECT 1 FROM dbo.TransferenciaBancaria t WHERE t.BancoOrigen=@propio AND
t.TokenTransferencia=@token AND t.Estado IN('APROBADA','RECHAZADA'))

 BEGIN

  UPDATE r SET Estado=t.Estado,Codigo=CASE WHEN t.Estado='APROBADA' THEN '00' ELSE '14'
END,Mensaje=N'Resultado confirmado por el banco.'

  FROM dbo.TransferenciaCajero r JOIN dbo.TransferenciaBancaria t ON t.BancoOrigen=r.BancoOrigen AND
t.TokenTransferencia=r.TokenTransferencia

  WHERE r.Id=@id AND r.Estado NOT IN('APROBADA','RECHAZADA');

 END;

 IF @Tipo='ESTADO' AND EXISTS(SELECT 1 FROM dbo.TransferenciaCajero WHERE Id=@id AND Estado
IN('RESERVADA','INCIERTA'))

 BEGIN

  SELECT @ip=IP,@puerto=Puerto FROM dbo.BancoRed WHERE CodigoBanco=@banco AND Activo=1 AND Confiable=1;

  IF @ip IS NOT NULL

  BEGIN

   SET @accion='CONCILIAR';
```

```
   SET @ruta=(SELECT @ip ip,@puerto puerto,'IB_ESTADO_TRANSFERENCIA'
tipo,REPLACE(CONVERT(varchar(36),NEWID()),'-','') idMensaje,

    @propio bancoOrigen,@banco bancoDestino,@token tokenTransferencia,@m monto,CuentaDestino cuentaDestino
FROM dbo.TransferenciaCajero WHERE Id=@id FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

  END;

 END;

 Recibo:

 SELECT @codigo=Codigo,@mensaje=Mensaje,@estado=Estado FROM dbo.TransferenciaCajero WHERE Id=@id;

 IF @estado IN('APROBADA','RECHAZADA') AND EXISTS(SELECT 1 FROM dbo.TransferenciaCajero WHERE Id=@id AND
Datos='{}')

 BEGIN

  UPDATE r SET Datos=(SELECT r.TokenTransferencia tokenTransferencia,r.Estado estado,r.Monto
monto,r.BancoDestino bancoDestino,

   r.NombreDestino nombreDestino,'**** '+RIGHT(r.CuentaDestino,4) cuentaDestino,r.Concepto concepto,r.Fecha
fecha,

   COALESCE(r.SaldoPosterior,(SELECT Saldo FROM dbo.Usuario WHERE Id=r.UsuarioId)) saldo FOR JSON
PATH,WITHOUT_ARRAY_WRAPPER) FROM dbo.TransferenciaCajero r WHERE r.Id=@id;

 END;

 SELECT @datos=CASE WHEN Estado IN('APROBADA','RECHAZADA') THEN Datos ELSE

  (SELECT TokenTransferencia tokenTransferencia,Estado estado,Monto monto,Fecha fecha FOR JSON
PATH,WITHOUT_ARRAY_WRAPPER) END FROM dbo.TransferenciaCajero WHERE Id=@id;

 IF @estado NOT IN('APROBADA','RECHAZADA') BEGIN SET @codigo='96';SET @mensaje=N'Transferencia pendiente. No
repita el envío.';END;

 IF @Tipo='ESTADO' AND @estado IN('APROBADA','RECHAZADA')

 BEGIN

  SET @datos=(SELECT Solicitud idOriginal,UsuarioId usuarioId,Codigo codigo,Mensaje mensaje,JSON_QUERY(Datos)
datos FROM dbo.TransferenciaCajero WHERE Id=@id FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

  SET @codigo='00';SET @mensaje=N'Resultado confirmado.';

 END;

 Fin:

 COMMIT;

 SELECT @codigo Codigo,@mensaje Mensaje,@datos Datos,@accion Accion,@ruta Ruta;

 END TRY

 BEGIN CATCH

 IF @@TRANCOUNT&gt;0 ROLLBACK;

 THROW;

 END CATCH;

END;

GO


-- Recepción externa: rechazar identificadores ambiguos.

CREATE OR ALTER PROCEDURE dbo.sp_IB_Operacion

 @Tipo varchar(40),@Json nvarchar(max),@IP varchar(50)='',@Administrador int=NULL

AS

BEGIN

 SET NOCOUNT ON;SET XACT_ABORT ON;

 DECLARE @codigo varchar(2)='00',@mensaje nvarchar(240)=N'Operación aprobada.',@datos nvarchar(max)='{}',

 @propio varchar(15),@nombre nvarchar(100),@bo varchar(15)=JSON_VALUE(@Json,'$.bancoOrigen'),

 @bd varchar(15)=JSON_VALUE(@Json,'$.bancoDestino'),@token
varchar(40)=JSON_VALUE(@Json,'$.tokenTransferencia'),

 @co varchar(20)=JSON_VALUE(@Json,'$.cuentaOrigen'),@cd
varchar(20)=COALESCE(JSON_VALUE(@Json,'$.cuentaDestino'),JSON_VALUE(@Json,'$.numeroCuenta')),
```

```
 @no nvarchar(120)=ISNULL(JSON_VALUE(@Json,'$.nombreOrigen'),''),@nd nvarchar(120),

 @concepto nvarchar(200)=ISNULL(JSON_VALUE(@Json,'$.concepto'),''),

 @m decimal(18,2)=TRY_CONVERT(decimal(18,2),JSON_VALUE(@Json,'$.monto')),

 @u int=TRY_CONVERT(int,JSON_VALUE(@Json,'$.usuarioId')),@hoy date=CAST(SYSDATETIME() AS date),

 @huella varbinary(32),@anterior varbinary(32),@estado varchar(20),@id bigint,@lock int;

 BEGIN TRY

 BEGIN TRAN;

 EXEC @lock=sys.sp_getapplock
@Resource='CajeroRed_Interbancario',@LockMode='Exclusive',@LockOwner='Transaction',@LockTimeout=2000;

 IF @lock&lt;0 THROW 50200,'Integración ocupada; reintente.',1;

 IF @Tipo IN('CONFIGURAR','ASIGNAR_CUENTA','RESERVAR','FINALIZAR')

 AND NOT EXISTS(SELECT 1 FROM dbo.Administrador WHERE Id=@Administrador AND Activo=1 AND Rol='Administrador')

 THROW 50201,'Administrador no autorizado.',1;

 IF @Tipo='CONFIGURAR'

 BEGIN

  DECLARE @c varchar(15)=JSON_VALUE(@Json,'$.codigoBanco'),@n nvarchar(100)=JSON_VALUE(@Json,'$.nombreBanco'),

  @ipb varchar(50)=JSON_VALUE(@Json,'$.ip'),@p int=TRY_CONVERT(int,JSON_VALUE(@Json,'$.puerto')),

  @es bit=TRY_CONVERT(bit,JSON_VALUE(@Json,'$.esPropio')),@conf
bit=TRY_CONVERT(bit,JSON_VALUE(@Json,'$.confiable')),

  @lim decimal(18,2)=TRY_CONVERT(decimal(18,2),JSON_VALUE(@Json,'$.limiteDiario'));

  IF NULLIF(@c,'') IS NULL OR @c LIKE '%[^A-Z0-9_-]%' OR NULLIF(TRIM(@n),'') IS NULL OR NULLIF(@ipb,'') IS
NULL

   OR @p IS NULL OR @p NOT BETWEEN 1 AND 65535 OR @es IS NULL OR @conf IS NULL OR @lim IS NULL OR @lim&lt;=0

   THROW 50202,'Configuración bancaria inválida.',1;

  IF @es=1 AND EXISTS(SELECT 1 FROM dbo.BancoRed WHERE EsPropio=1 AND CodigoBanco&lt;&gt;@c)

  BEGIN

   IF EXISTS(SELECT 1 FROM dbo.TransferenciaBancaria) OR EXISTS(SELECT 1 FROM dbo.BancoRed WHERE
CodigoBanco=@c)

    THROW 50203,'No se puede cambiar un código con historial ni usar uno ya registrado.',1;

   UPDATE dbo.BancoRed SET CodigoBanco=@c WHERE EsPropio=1;

  END;

  IF EXISTS(SELECT 1 FROM dbo.BancoRed WHERE CodigoBanco=@c AND EsPropio&lt;&gt;@es)

   THROW 50204,'No se puede cambiar la propiedad de un banco existente.',1;

  IF EXISTS(SELECT 1 FROM dbo.BancoRed WHERE CodigoBanco=@c AND FechaUltimoReinicio=@hoy AND
TransferidoHoy&gt;@lim)

   THROW 50205,'El límite no puede ser menor al cupo ya consumido.',1;

  IF EXISTS(SELECT 1 FROM dbo.BancoRed WHERE CodigoBanco=@c)

   UPDATE dbo.BancoRed SET NombreBanco=@n,IP=@ipb,Puerto=@p,Confiable=@conf,LimiteDiario=@lim,Activo=1 WHERE
CodigoBanco=@c;

  ELSE INSERT dbo.BancoRed(CodigoBanco,NombreBanco,IP,Puerto,EsPropio,Confiable,LimiteDiario)
VALUES(@c,@n,@ipb,@p,@es,@conf,@lim);

  INSERT dbo.Bitacora(Origen,Accion,Detalle)
VALUES(CONCAT('ADMIN-',@Administrador),'IB_CONFIGURAR',CONCAT('Banco ',@c));

  GOTO Fin;

 END;

 IF @Tipo='ASIGNAR_CUENTA'

 BEGIN

  SET @cd=JSON_VALUE(@Json,'$.numeroCuenta');

  IF @cd IS NULL OR LEN(@cd) NOT BETWEEN 1 AND 20 OR @cd LIKE '%[^0-9]%' THROW 50206,'Cuenta inválida: 1 a 20
dígitos.',1;

  -- Evita que una cuenta de otro usuario se confunda con una tarjeta.
```

```
  IF EXISTS(SELECT 1 FROM dbo.Tarjeta WHERE Numero=@cd AND UsuarioId&lt;&gt;@u) THROW 50207,'La cuenta coincide con
la tarjeta de otro usuario.',1;

  UPDATE dbo.Usuario SET NumeroCuenta=@cd WHERE Id=@u AND Activo=1;

  IF @@ROWCOUNT=0 THROW 50208,'Usuario no disponible.',1;

  INSERT dbo.Bitacora(Origen,Accion,Detalle)
VALUES(CONCAT('ADMIN-',@Administrador),'IB_ASIGNAR_CUENTA',CONCAT('Usuario ',@u));

  GOTO Fin;

 END;

 SELECT @propio=CodigoBanco,@nombre=NombreBanco FROM dbo.BancoRed WHERE EsPropio=1 AND Activo=1;

 IF @propio IS NULL BEGIN SET @codigo='91';SET @mensaje=N'Configure primero el banco propio.';GOTO Fin;END;

 UPDATE dbo.BancoRed SET TransferidoHoy=0,FechaUltimoReinicio=@hoy WHERE FechaUltimoReinicio&lt;&gt;@hoy;

 IF @Tipo='IB_PING'

 BEGIN

  SET @datos=(SELECT CodigoBanco codigoBanco,NombreBanco nombreBanco,LimiteDiario-TransferidoHoy
cupoDisponibleHoy,'EN_LINEA' estado FROM dbo.BancoRed WHERE CodigoBanco=@propio FOR JSON
PATH,WITHOUT_ARRAY_WRAPPER);

  SET @mensaje=N'Banco '+@propio+N' en línea';GOTO Fin;

 END;

 IF @Tipo IN('IB_VERIFICAR_CUENTA','IB_TRANSFERIR','IB_ESTADO_TRANSFERENCIA') AND ISNULL(@bd,@propio)&lt;&gt;@propio

 BEGIN SET @codigo='14';SET @mensaje=N'Banco destino incorrecto.';GOTO Fin;END;

 IF @Tipo='IB_VERIFICAR_CUENTA'

 BEGIN

  SET @u=NULL;

  IF (SELECT COUNT(*) FROM dbo.fn_CuentaActiva(@cd))=1

   SELECT @u=u.Id,@nd=u.Nombre FROM dbo.Usuario u WITH(UPDLOCK,ROWLOCK) JOIN dbo.fn_CuentaActiva(@cd) f ON
f.Id=u.Id;

  SET @datos=(SELECT CAST(CASE WHEN @u IS NULL THEN 0 ELSE 1 END AS bit) existe,@propio codigoBanco,@nombre
nombreBanco,

   @cd numeroCuenta,ISNULL(@nd,'') nombreTitular,LimiteDiario-TransferidoHoy cupoDisponibleBanco FROM
dbo.BancoRed WHERE CodigoBanco=@propio FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

  IF @u IS NULL BEGIN SET @codigo='14';SET @mensaje=N'La cuenta no existe en este banco.';END;

  GOTO Fin;

 END;

 IF @Tipo IN('IB_TRANSFERIR','IB_ESTADO_TRANSFERENCIA')

 AND NOT EXISTS(SELECT 1 FROM dbo.BancoRed WHERE CodigoBanco=@bo AND EsPropio=0 AND Activo=1 AND Confiable=1
AND IP=@IP)

 BEGIN SET @codigo='57';SET @mensaje=N'Banco origen o IP no autorizado.';GOTO Fin;END;

 IF @Tipo='IB_ESTADO_TRANSFERENCIA'

 BEGIN

  SELECT @estado=Estado,@datos=RespuestaDatos FROM dbo.TransferenciaBancaria WHERE BancoOrigen=@bo AND
TokenTransferencia=@token AND Direccion='ENTRANTE';

  IF @estado IS NULL BEGIN SET @codigo='25';SET @mensaje=N'Transferencia no registrada; no confirma rechazo ni
 autoriza devolución.';END;

  GOTO Fin;

 END;

 IF @Tipo='IB_TRANSFERIR'

 BEGIN

  IF @token IS NULL OR LEN(@token)&lt;&gt;19 OR @token NOT LIKE
'TRF-[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]-[0-9][0-9][0-9][0-9][0-9][0-9]'

   OR @m IS NULL OR @m&lt;=0 OR @co IS NULL OR @cd IS NULL OR @bo=@propio

  BEGIN SET @codigo='30';SET @mensaje=N'Token, cuentas o monto inválidos.';GOTO Fin;END;

  SET @huella=HASHBYTES('SHA2_256',CONCAT(@bo,'|',@bd,'|',@co,'|',@cd,'|',@m,'|',@no,'|',@concepto));
```

```
  SELECT @anterior=Huella,@datos=RespuestaDatos FROM dbo.TransferenciaBancaria WHERE BancoOrigen=@bo AND
TokenTransferencia=@token;

  IF @anterior IS NOT NULL

  BEGIN

   IF @anterior&lt;&gt;@huella BEGIN SET @codigo='94';SET @mensaje=N'Token reutilizado con contenido distinto.';SET
@datos='{}';END;

   GOTO Fin;

  END;

  SET @u=NULL;

  SET @u=NULL;

  IF (SELECT COUNT(*) FROM dbo.fn_CuentaActiva(@cd))=1

   SELECT @u=u.Id,@nd=u.Nombre FROM dbo.Usuario u WITH(UPDLOCK,ROWLOCK) JOIN dbo.fn_CuentaActiva(@cd) f ON
f.Id=u.Id;

  IF @u IS NULL BEGIN SET @codigo='14';SET @mensaje=N'La cuenta no existe en este banco.';GOTO Fin;END;

  IF EXISTS(SELECT 1 FROM dbo.BancoRed WHERE CodigoBanco IN(@propio,@bo) AND LimiteDiario-TransferidoHoy&lt;@m)

  BEGIN SET @codigo='65';SET @mensaje=N'Cupo diario bancario insuficiente.';GOTO Fin;END;

  UPDATE dbo.Usuario SET Saldo=Saldo+@m WHERE Id=@u;

  UPDATE dbo.BancoRed SET TransferidoHoy=TransferidoHoy+@m WHERE CodigoBanco IN(@propio,@bo);

  SET @datos=(SELECT @token tokenTransferencia,@propio bancoDestino,@cd cuentaDestino,@nd nombreDestino,@m
monto,LimiteDiario-TransferidoHoy cupoRestanteBancoDestino FROM dbo.BancoRed WHERE CodigoBanco=@propio FOR
JSON PATH,WITHOUT_ARRAY_WRAPPER);

  INSERT dbo.TransferenciaBancaria(TokenTransferencia,Direccion,BancoOrigen,BancoDestino,UsuarioId,CuentaOrige
n,CuentaDestino,NombreOrigen,NombreDestino,Monto,Concepto,Estado,Huella,RespuestaDatos)

  VALUES(@token,'ENTRANTE',@bo,@propio,@u,@co,@cd,@no,@nd,@m,@concepto,'APROBADA',@huella,@datos);

  INSERT dbo.Bitacora(Origen,Accion,Detalle) VALUES(@bo,'IB_RECIBIDA',@token);

  SET @mensaje=N'Transferencia interbancaria acreditada y token registrado.';GOTO Fin;

 END;

 IF @Tipo='RESERVAR'

 BEGIN

  IF @m IS NULL OR @m&lt;=0 OR @bd=@propio OR NOT EXISTS(SELECT 1 FROM dbo.BancoRed WHERE CodigoBanco=@bd AND
Activo=1 AND EsPropio=0 AND Confiable=1)

   THROW 50209,'Monto o banco destino no autorizado.',1;

  SELECT @co=COALESCE(u.NumeroCuenta,t.Numero),@no=u.Nombre FROM dbo.Usuario u WITH(UPDLOCK,ROWLOCK)

  LEFT JOIN dbo.Tarjeta t ON t.UsuarioId=u.Id AND t.Activa=1 WHERE u.Id=@u AND u.Activo=1 AND u.Bloqueado=0
AND u.Saldo&gt;=@m;

  IF @co IS NULL BEGIN SET @codigo='51';SET @mensaje=N'Usuario sin cuenta activa o saldo insuficiente.';GOTO
Fin;END;

  IF EXISTS(SELECT 1 FROM dbo.BancoRed WHERE CodigoBanco IN(@propio,@bd) AND LimiteDiario-TransferidoHoy&lt;@m)

  BEGIN SET @codigo='65';SET @mensaje=N'Cupo diario bancario insuficiente.';GOTO Fin;END;

  IF EXISTS(SELECT 1 FROM dbo.TransferenciaBancaria WHERE BancoOrigen=@propio AND TokenTransferencia=@token)

  BEGIN SET @codigo='94';SET @mensaje=N'Token ya registrado.';GOTO Fin;END;

  SET @nd=ISNULL(JSON_VALUE(@Json,'$.nombreDestino'),'');

  UPDATE dbo.Usuario SET Saldo=Saldo-@m WHERE Id=@u;

  UPDATE dbo.BancoRed SET TransferidoHoy=TransferidoHoy+@m WHERE CodigoBanco IN(@propio,@bd);

  SET @huella=HASHBYTES('SHA2_256',CONCAT(@propio,'|',@bd,'|',@co,'|',@cd,'|',@m,'|',@no,'|',@concepto));

  INSERT dbo.TransferenciaBancaria(TokenTransferencia,Direccion,BancoOrigen,BancoDestino,UsuarioId,CuentaOrige
n,CuentaDestino,NombreOrigen,NombreDestino,Monto,Concepto,Estado,Huella)

  VALUES(@token,'SALIENTE',@propio,@bd,@u,@co,@cd,@no,@nd,@m,@concepto,'RESERVADA',@huella);

  SET @datos=(SELECT 'IB_TRANSFERIR' tipo,'SERVER' cajero,REPLACE(CONVERT(varchar(36),NEWID()),'-','')
idMensaje,

   CONVERT(varchar(19),SYSDATETIME(),120) timestamp,@token tokenTransferencia,@propio bancoOrigen,@co
cuentaOrigen,@no nombreOrigen,
```

```
   @bd bancoDestino,@cd cuentaDestino,@nd nombreDestino,@m monto,@concepto concepto FOR JSON
PATH,WITHOUT_ARRAY_WRAPPER);

  INSERT dbo.Bitacora(Origen,Accion,Detalle) VALUES(CONCAT('ADMIN-',@Administrador),'IB_RESERVADA',@token);

  GOTO Fin;

 END;

 IF @Tipo='FINALIZAR'

 BEGIN

  SELECT @id=IdTransferencia,@estado=Estado,@u=UsuarioId,@m=Monto,@bo=BancoOrigen,@bd=BancoDestino FROM
dbo.TransferenciaBancaria

   WHERE TokenTransferencia=@token AND BancoOrigen=@propio AND Direccion='SALIENTE';

  IF @id IS NULL THROW 50210,'Transferencia saliente no encontrada.',1;

  IF @estado IN('APROBADA','RECHAZADA') GOTO Fin;

  DECLARE @resultado varchar(2)=JSON_VALUE(@Json,'$.codigo');

  IF @resultado='00' SET @estado='APROBADA';

  ELSE IF @resultado IN('14','65','57','30')

  BEGIN

   SET @estado='RECHAZADA';

   UPDATE dbo.Usuario SET Saldo=Saldo+@m WHERE Id=@u;

   UPDATE dbo.BancoRed SET TransferidoHoy=TransferidoHoy-@m

   WHERE CodigoBanco IN(@bo,@bd) AND FechaUltimoReinicio=CAST((SELECT FechaCreacion FROM
dbo.TransferenciaBancaria WHERE IdTransferencia=@id) AS date);

  END

  ELSE SET @estado='INCIERTA';

  UPDATE dbo.TransferenciaBancaria SET Estado=@estado,FechaActualizacion=SYSDATETIME() WHERE
IdTransferencia=@id;

  INSERT dbo.Bitacora(Origen,Accion,Detalle) VALUES(CONCAT('ADMIN-',@Administrador),'IB_'+@estado,@token);

  SET @datos=(SELECT @token tokenTransferencia,@estado estado FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);

  GOTO Fin;

 END;

 SET @codigo='30';SET @mensaje=N'Operación interbancaria no soportada.';

 Fin:

 COMMIT;

 SELECT @codigo Codigo,@mensaje Mensaje,@datos Datos,CONVERT(varchar(19),SYSDATETIME(),120) Fecha;

 END TRY

 BEGIN CATCH

 IF @@TRANCOUNT&gt;0 ROLLBACK;

 THROW;

 END CATCH;

END;

GO
```

11_MonitoreoIndependiente.sql

```
USE BD_CajeroRed_Fase2;
GO
-- El latido no espera los bloqueos de una operación financiera.
CREATE OR ALTER PROCEDURE dbo.sp_Heartbeat
 @Json nvarchar(max),@Cajero varchar(24),@Sesion varchar(36)=NULL,@Solicitud varchar(64)
AS
BEGIN
 SET NOCOUNT ON; SET LOCK_TIMEOUT 200;
 DECLARE @activo bit,@habilitado bit,@inicializado date,@datos nvarchar(max);
 BEGIN TRY
  SELECT @activo=Activo,@habilitado=Habilitado,@inicializado=FechaInicializacion
  FROM dbo.Cajero WITH(READPAST,READCOMMITTEDLOCK) WHERE Codigo=@Cajero;
  IF @activo IS NULL
   SELECT '96' Codigo,N'Conexión activa; estado SQL ocupado.' Mensaje,'{}' Datos;
  ELSE IF @activo=0
   SELECT '91' Codigo,N'Cajero inactivo.' Mensaje,'{}' Datos;
  ELSE
  BEGIN
   UPDATE dbo.Cajero SET UltimoHeartbeat=SYSDATETIME() WHERE Codigo=@Cajero;
   SET @datos=(SELECT @habilitado habilitado,CAST(CASE WHEN @inicializado=CAST(SYSDATETIME() AS date) THEN 1
ELSE 0 END AS bit) inicializado FOR JSON PATH,WITHOUT_ARRAY_WRAPPER);
   SELECT '00' Codigo,N'Conexión activa.' Mensaje,@datos Datos;
  END;
 END TRY
 BEGIN CATCH
  IF ERROR_NUMBER()=1222 SELECT '96' Codigo,N'Conexión activa; estado SQL ocupado.' Mensaje,'{}' Datos;
  ELSE THROW;
 END CATCH;
END;
GO
```